

到的分组是否一次重传。对于停等协议这种简单情况，1 比特序号就足够了，因为它可让接收方知道发送方是否正在重传前一个发送分组（接收到的分组序号与最近收到的分组序号相同），或是一个新分组（序号变化了，用模 2 运算“前向”移动）。因为目前我们假定信道不丢分组，ACK 和 NAK 分组本身不需要指明它们要确认的分组序号。发送方知道所接收到的 ACK 和 NAK 分组（无论是否是含糊不清的）是为响应其最近发送的数据分组而生成的。

图 3-11 和图 3-12 给出了对 rdt 2.1 的 FSM 描述，这是 rdt 2.0 的修订版。rdt 2.1 的发送方和接收方 FSM 的状态数都是以前的两倍。这是因为协议状态此时必须反映出目前（由发送方）正发送的分组或（在接收方）希望接收的分组的序号是 0 还是 1。值得注意的是，发送或期望接收 0 号分组的状态中的动作与发送或期望接收 1 号分组的状态中的动作是相似的；唯一的不同是序号处理的方法不同。

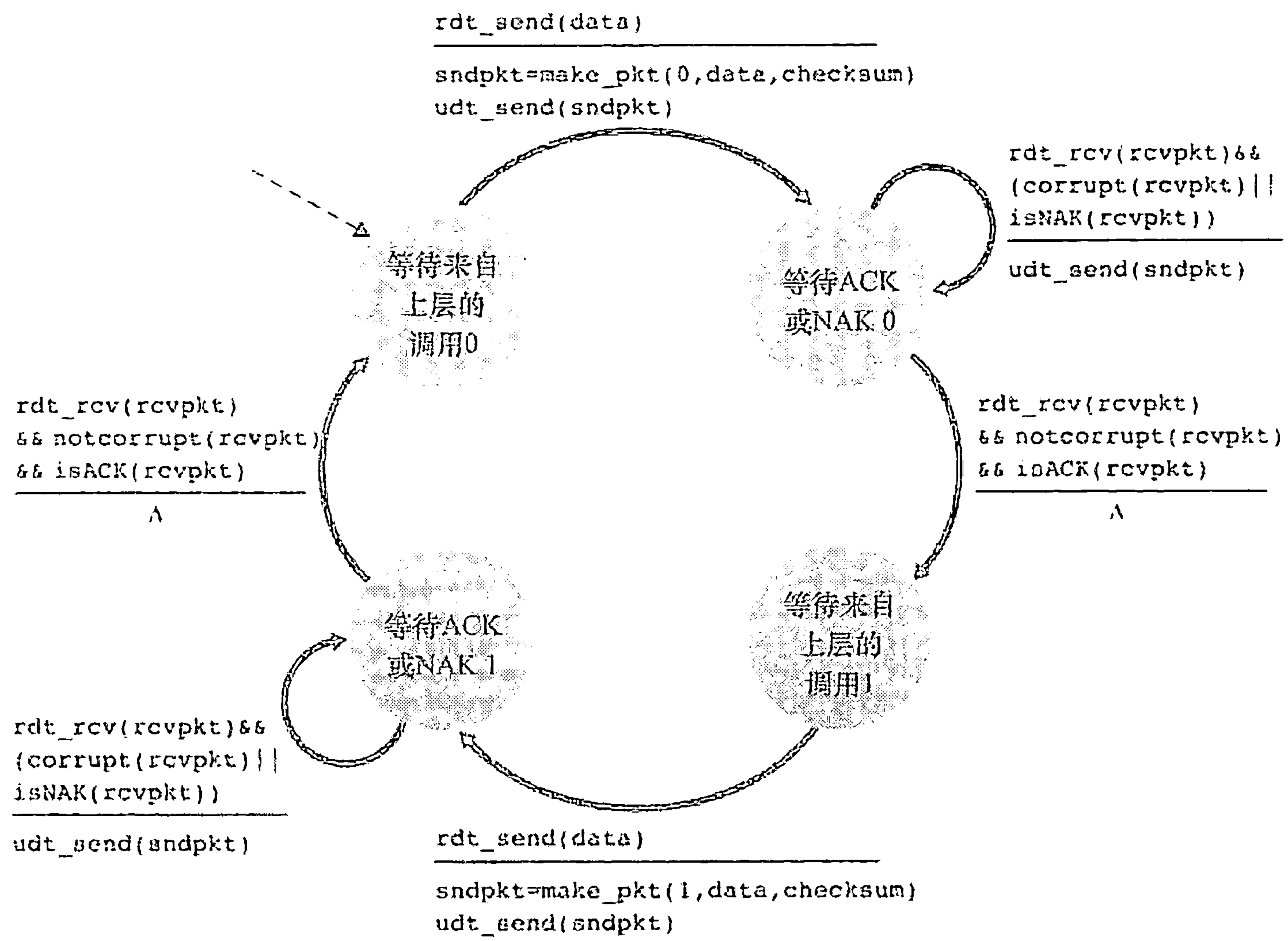


图 3-11 rdt 2.1 发送方

协议 rdt 2.1 使用了从接收方到发送方的肯定确认和否定确认。当接收到失序的分组时，接收方对所接收的分组发送一个肯定确认。如果收到受损的分组，则接收方将发送一个否定确认。如果不发送 NAK，而是对上次正确接收的分组发送一个 ACK，我们也能实现与 NAK 一样的效果。发送方接收到对同一个分组的两个 ACK（即接收冗余 ACK（duplicate ACK））后，就知道接收方没有正确接收到跟在被确认两次的分组后面的分组。rdt 2.2 是在有比特差错信道上实现的一个无 NAK 的可靠数据传输协议，如图 3-13 和图 3-14 所示。rdt2.1 和 rdt2.2 之间的细微变化在于，接收方此时必须包括由一个 ACK 报文所确认的分组序号（这可以通过在接收方 FSM 中，在 make_pkt() 中包括参数 ACK 0 或 ACK 1 来实现），发送方此时必须检查接收到的 ACK 报文中被确认的分组序号（这可通过在发送方 FSM 中，在 isACK() 中包括参数 0 或 1 来实现）。

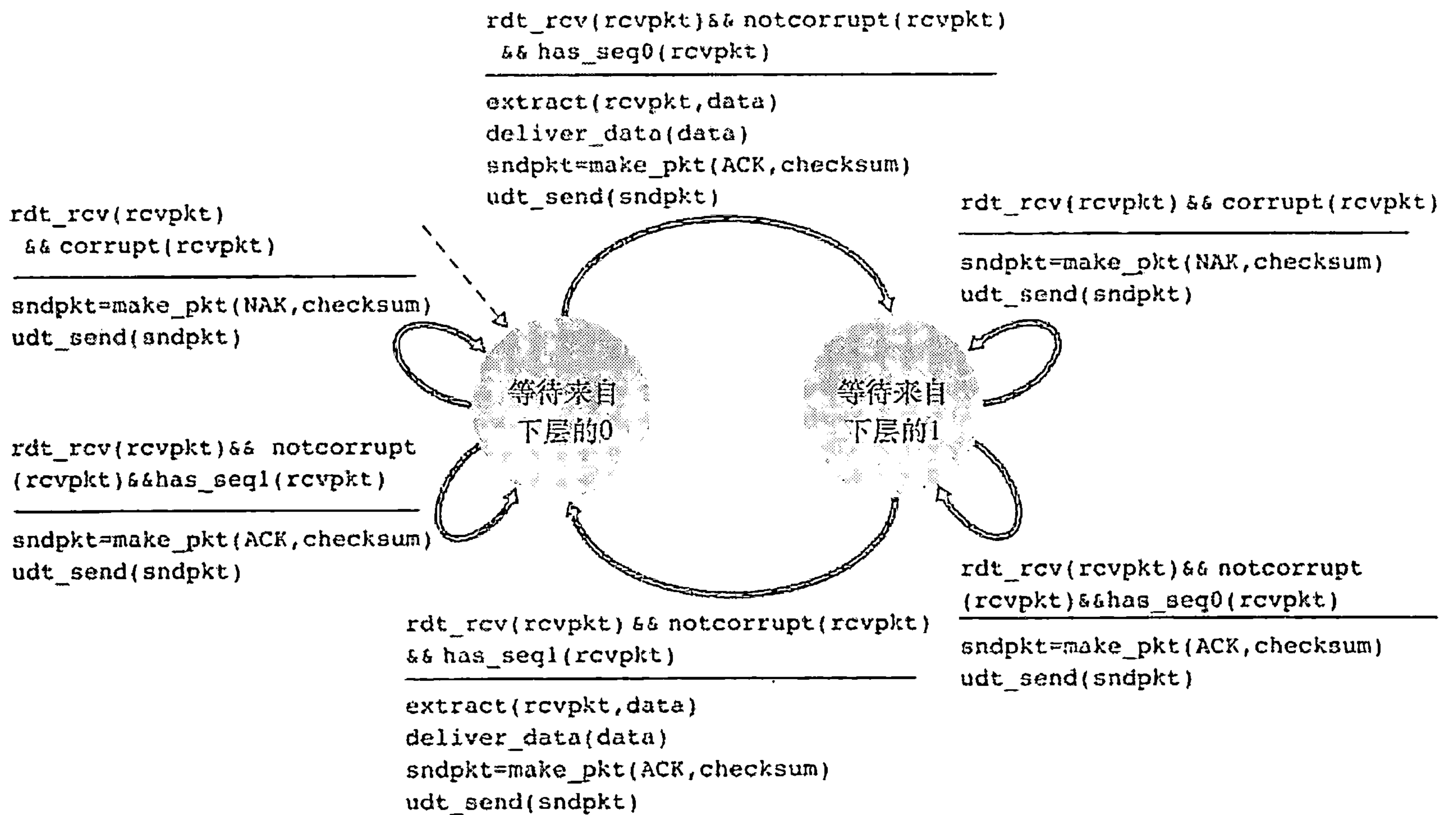


图 3-12 rdt 2.1 接收方

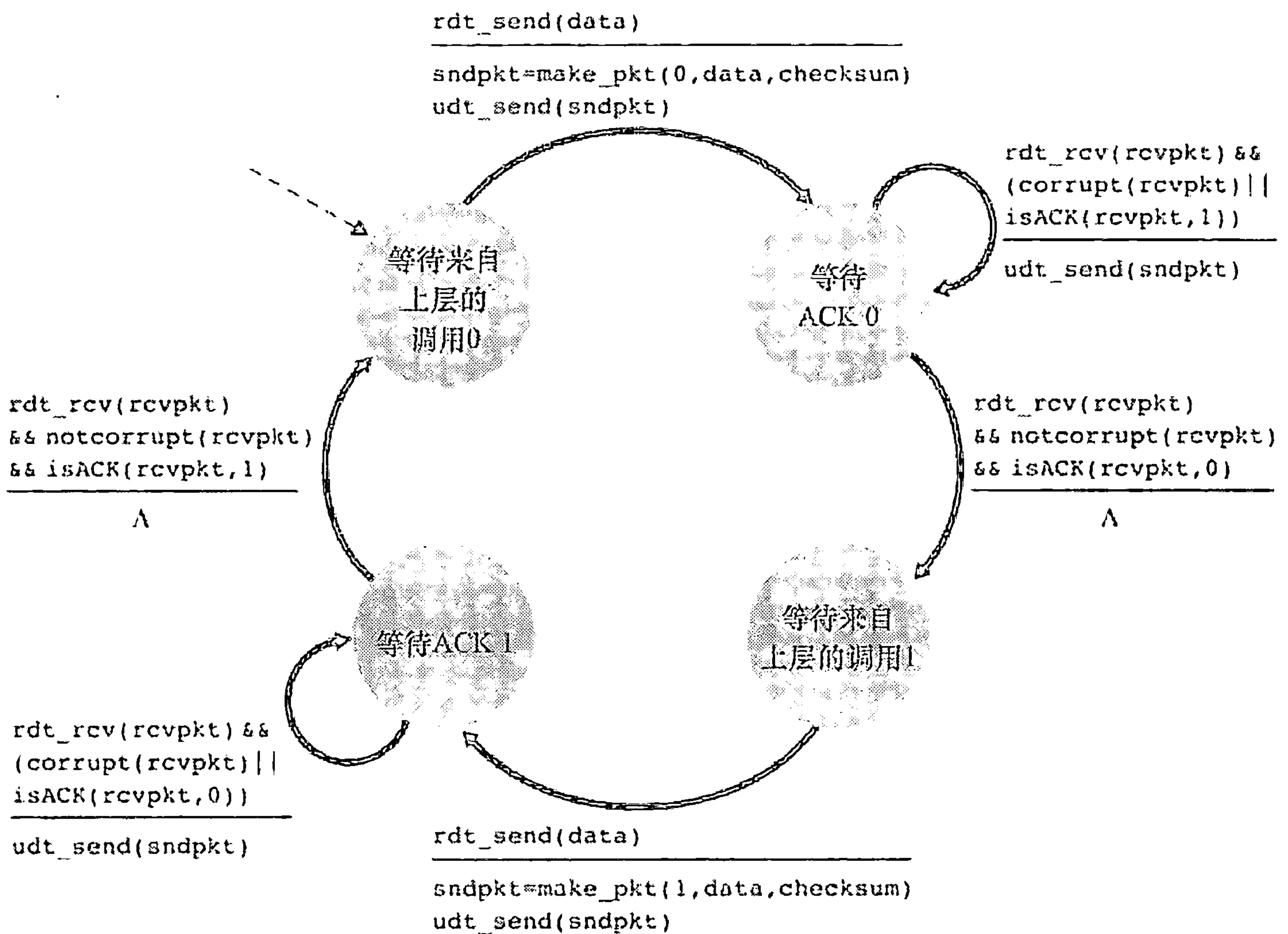


图 3-13 rdt 2.2 发送方

3. 经具有比特差错的丢包信道的可靠数据传输：rdt 3.0

现在假定除了比特受损外，底层信道还会丢包，这在今天的计算机网络（包括因特网）中并不罕见。协议现在必须处理另外两个关注的问题：怎样检测丢包以及发生丢包后

该做些什么。在 rdt 2.2 中已经研发的技术，如使用检验和、序号、ACK 分组和重传等，使我们能给出后一个问题的答案。为解决第一个关注的问题，还需增加一种新的协议机制。

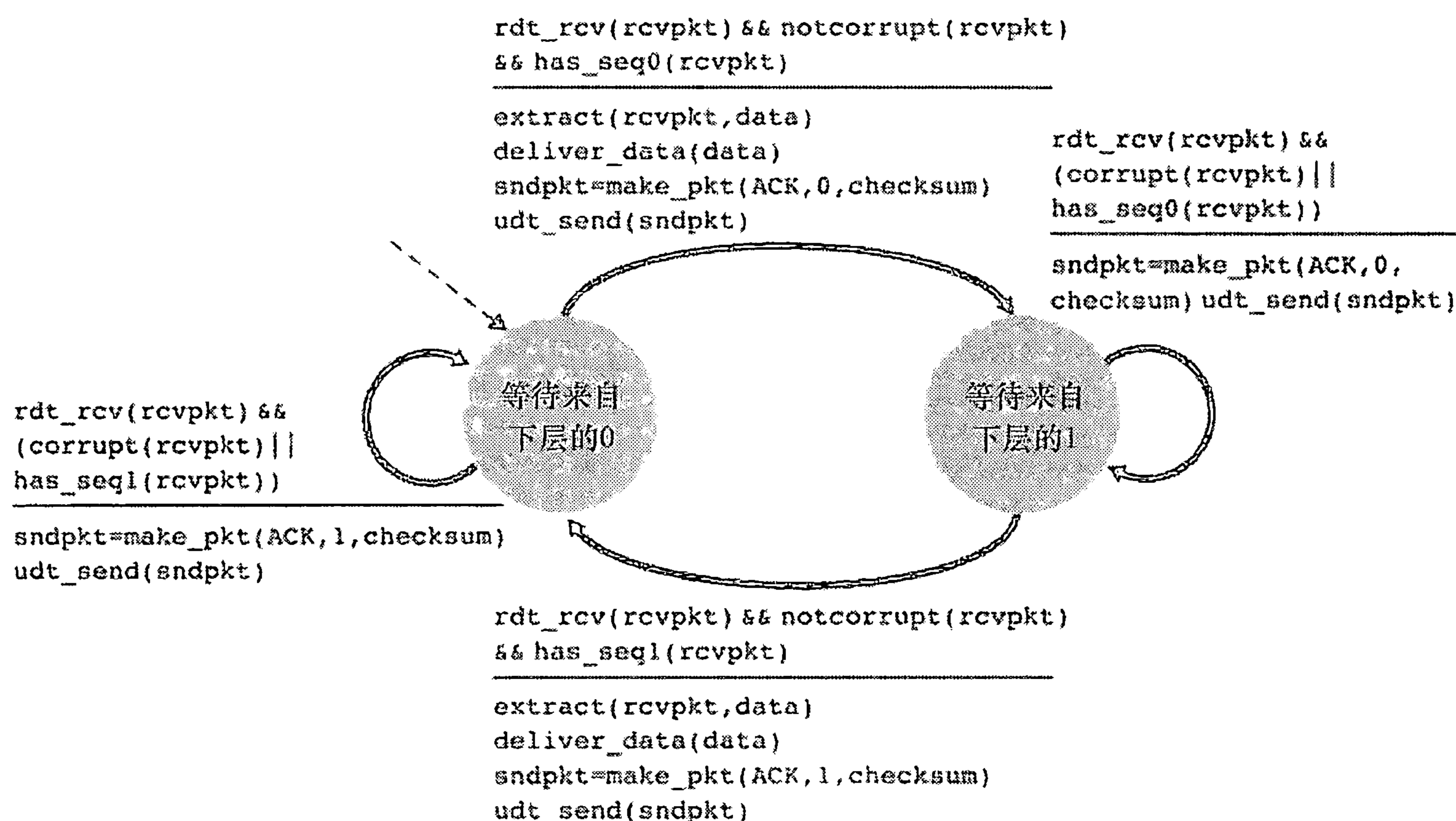


图 3-14 rdt 2.2 接收方

有很多可能的方法用于解决丢包问题（在本章结尾的习题中研究了几种其他方法）。这里，我们让发送方负责检测和恢复丢包工作。假定发送方传输一个数据分组，该分组或者接收方对该分组的 ACK 发生了丢失。在这两种情况下，发送方都收不到应当到来的接收方的响应。如果发送方愿意等待足够长的时间以便确定分组已丢失，则它只需重传该数据分组即可。你应该相信该协议确实有效。

但是发送方需要等待多久才能确定已丢失了某些东西呢？很明显发送方至少需要等待这样长的时间：即发送方与接收方之间的一个往返时延（可能会包括在中间路由器的缓冲时延）加上接收方处理一个分组所需的时间。在很多网络中，最坏情况下的最大时延是很难估算的，确定的因素非常少。此外，理想的协议应尽可能快地从丢包中恢复出来；等待一个最坏情况的时延可能意味着要等待一段较长的时间，直到启动差错恢复为止。因此实践中采取的方法是发送方明智地选择一个时间值，以判定可能发生了丢包（尽管不能确保）。如果在这个时间内没有收到 ACK，则重传该分组。注意到如果一个分组经历了一个特别大的时延，发送方可能会重传该分组，即使该数据分组及其 ACK 都没有丢失。这就在发送方到接收方的信道中引入了冗余数据分组（duplicate data packet）的可能性。幸运的是，rdt 2.2 协议已经有足够的功能（即序号）来处理冗余分组情况。

从发送方的观点来看，重传是一种万能灵药。发送方不知道是一个数据分组丢失，还是一个 ACK 丢失，或者只是该分组或 ACK 过度延时。在所有这些情况下，动作是同样的：重传。为了实现基于时间的重传机制，需要一个倒计时定时器（countdown timer），在一个给定的时间量过期后，可中断发送方。因此，发送方需要能做到：①每次发送一个分组（包括第一次分组和重传分组）时，便启动一个定时器。②响应定时器中断（采取适当的动作）。③终止定时器。

图 3-15 给出了 rdt 3.0 的发送方 FSM，这是一个在可能出错和丢包的信道上可靠传输

数据的协议；在课后习题中，将请你提供 rdt 3.0 的接收方 FSM。图 3-16 显示了在没有丢包和延迟分组情况下协议运作的情况，以及它是如何处理数据分组丢失的。在图 3-16 中，时间从图的顶部朝底部移动；注意到一个分组的接收时间必定迟于一个分组的发送时间，这是因为发送时延与传播时延之故。在图 3-16b ~ d 中，发送方括号部分表明了定时器的设置时刻以及随后的超时。本章后面的习题探讨了该协议几个更细微的方面。因为分组序号在 0 和 1 之间交替，因此 rdt 3.0 有时被称为比特交替协议（alternating-bit protocol）。

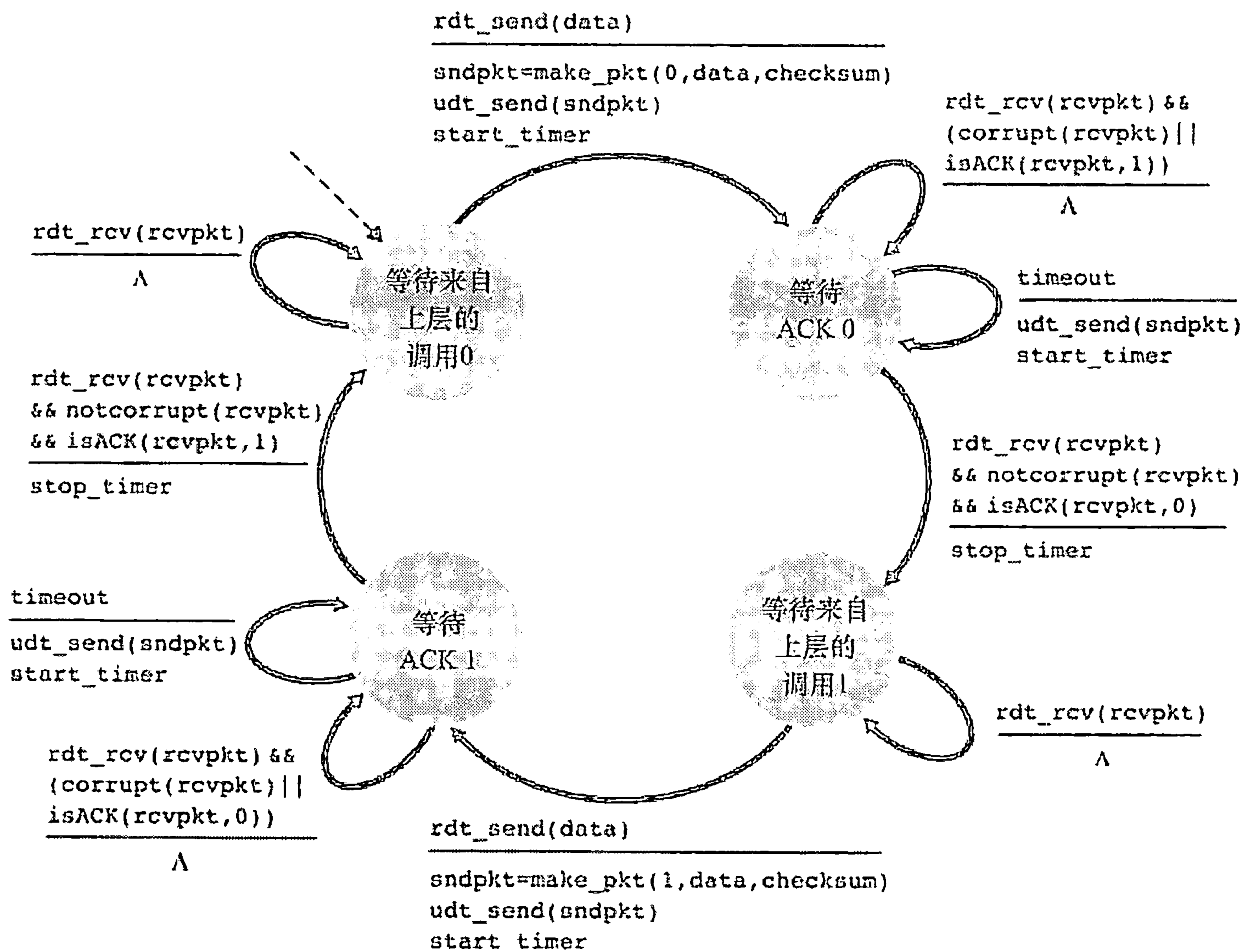


图 3-15 rdt 3.0 发送方

现在我们归纳一下数据传输协议的要点。在检验和、序号、定时器、肯定和否定确认分组这些技术中，每种机制都在协议的运行中起到了必不可少的作用。至此，我们得到了一个可靠数据传输协议！

3.4.2 流水线可靠数据传输协议

rdt 3.0 是一个功能正确的协议，但并非人人都对它的性能满意，特别是在今天的高速网络中更是如此。rdt 3.0 性能问题的核心在于它是一个停等协议。

为了评价该停等行为对性能的影响，可考虑一种具有两台主机的理想化场合，一台主机位于美国西海岸，另一台位于美国东海岸，如图 3-17 所示。在这两个端系统之间的光速往返传播时延 RTT 大约为 30 毫秒。假定彼此通过一条发送速率 R 为 1Gbps（每秒 10^9 比特）的信道相连。包括首部字段和数据的分组长 L 为 1000 字节（8000 比特），发送一个分组进入 1Gbps 链路实际所需时间是：

$$t_{\text{trans}} = \frac{L}{R} = \frac{8000\text{bit}/\text{pkt}}{10^9\text{bit/s}} = 8\mu\text{s}/\text{pkt}$$

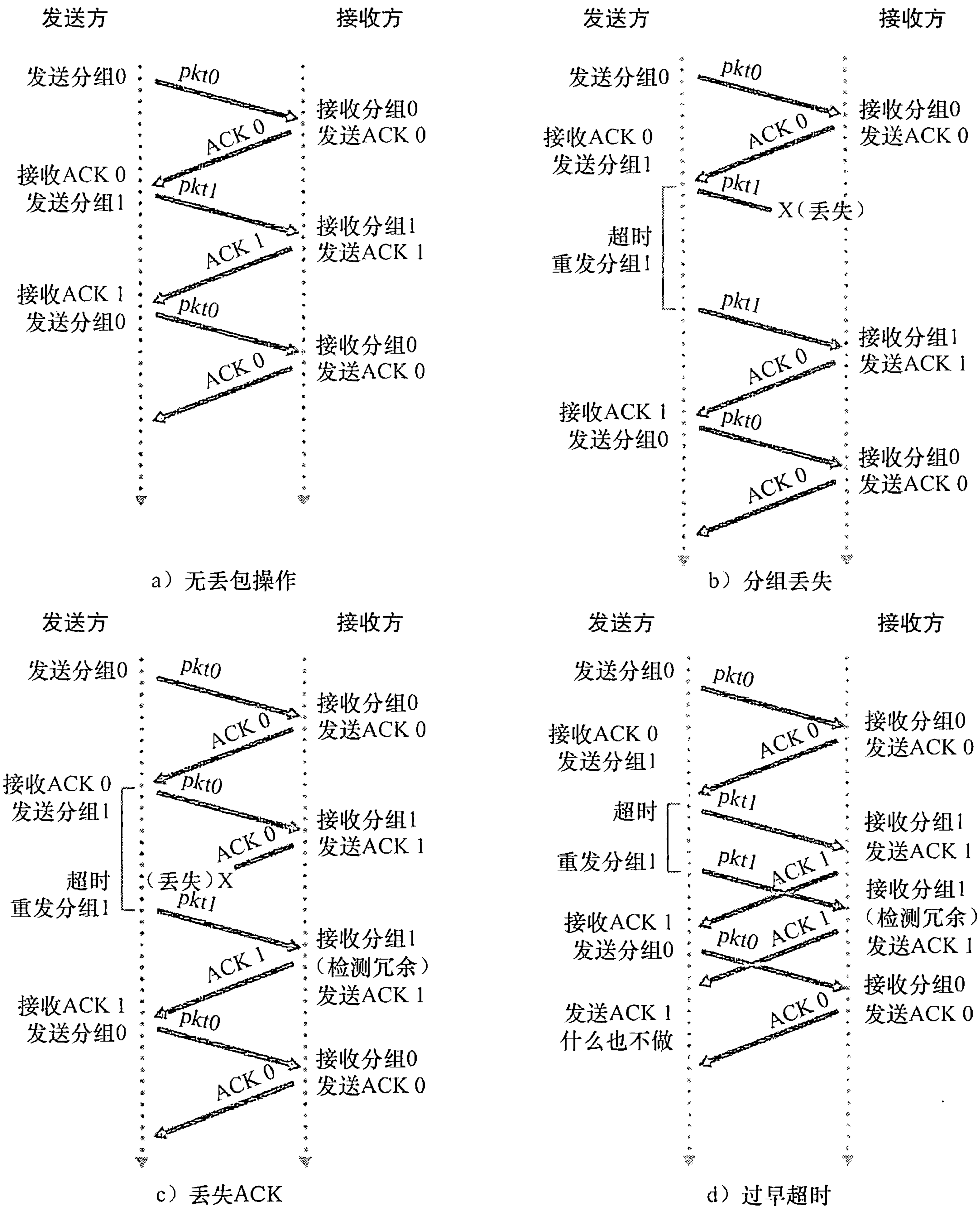


图 3-16 rdt 3.0 的运行，比特交替协议

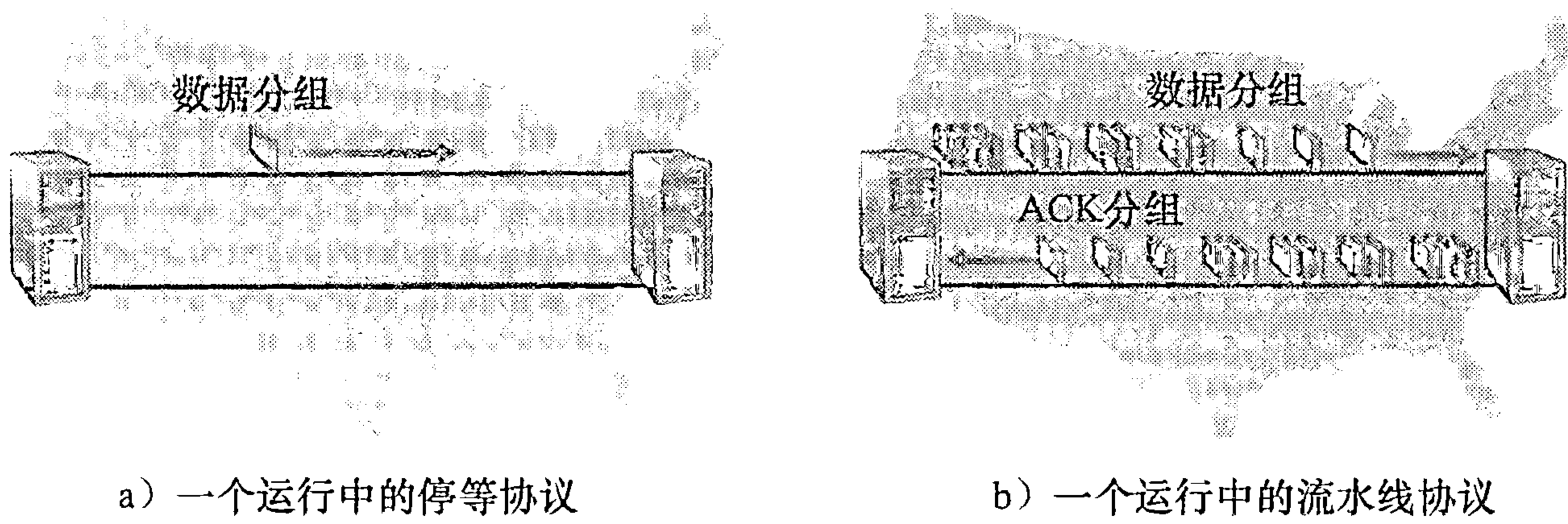
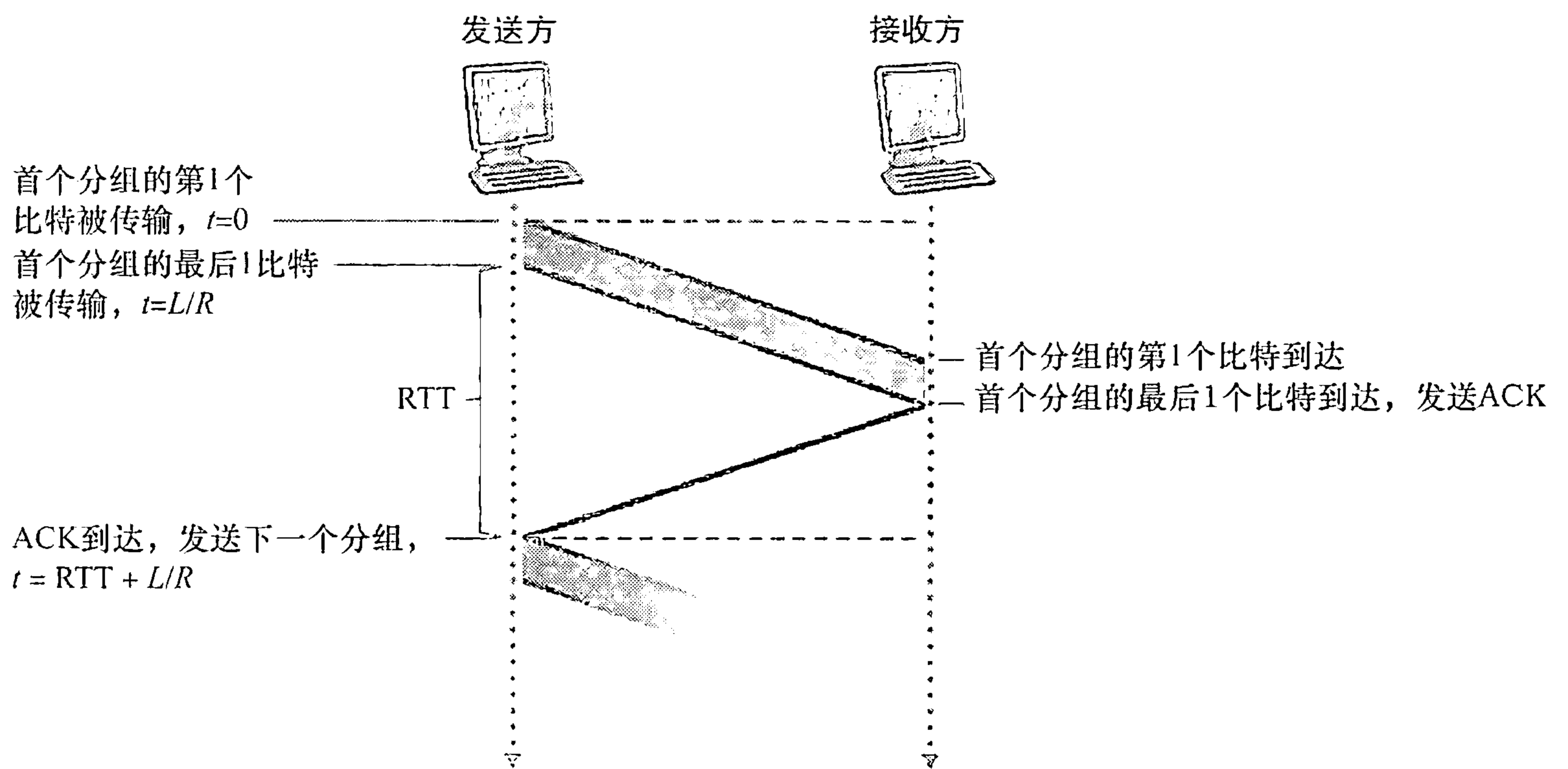


图 3-17 停等协议与流水线协议

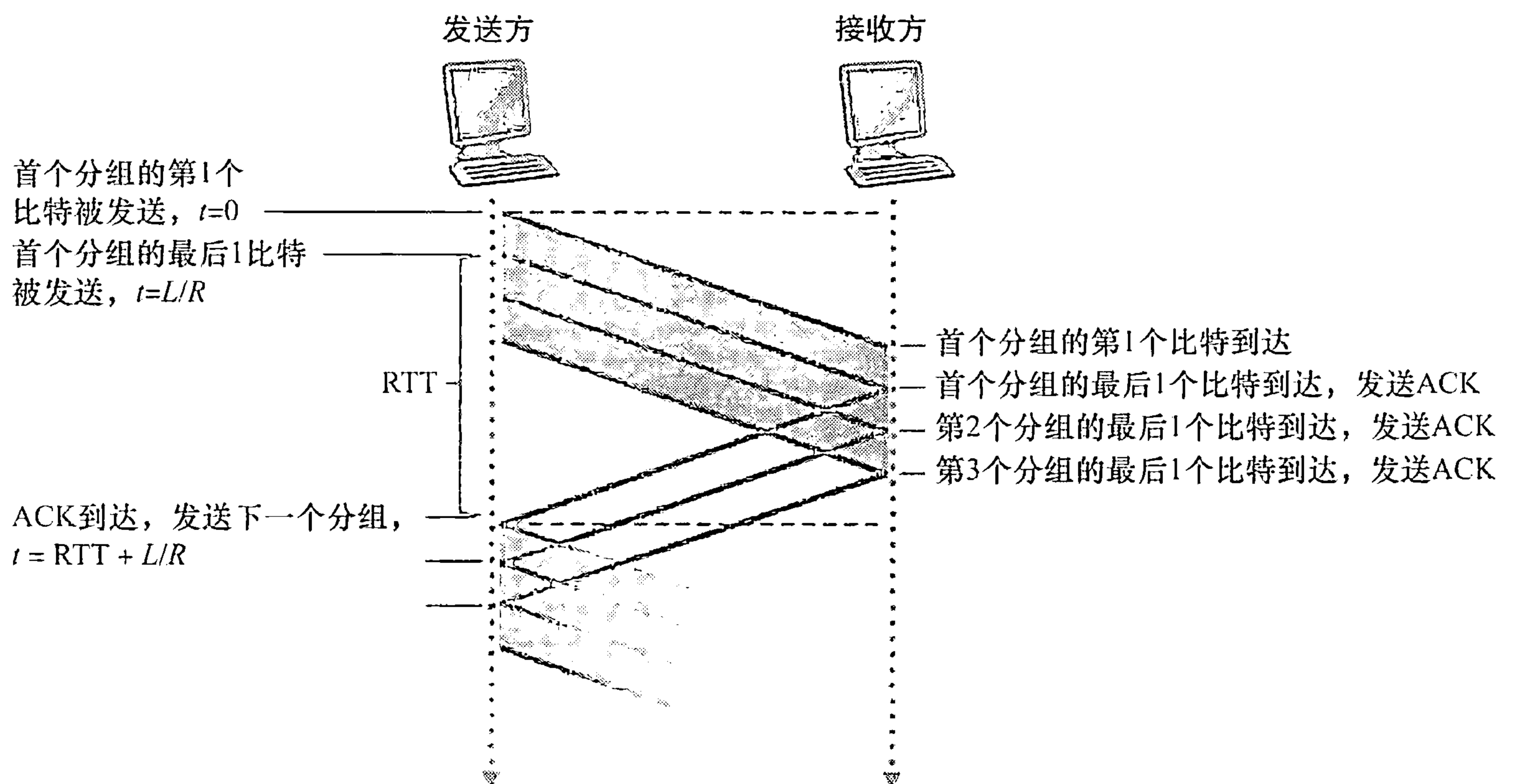
图 3-18a 显示了对于该停等协议，如果发送方在 $t = 0$ 时刻开始发送分组，则在 $t = L/R = 8\mu s$ 后，最后 1 比特数据进入了发送端信道。该分组经过 15ms 的穿越国家的旅途后到达接

收端，该分组的最后 1 比特在时刻 $t = \text{RTT}/2 + L/R = 15.008\text{ms}$ 时到达接收方。为了简化起见，假设 ACK 分组很小（以便我们可以忽略其发送时间），接收方一旦收到一个数据分组的最后 1 比特后立即发送 ACK，ACK 在时刻 $t = \text{RTT} + L/R = 30.008\text{ms}$ 时在发送方出现。此时，发送方可以发送下一个报文。因此，在 30.008ms 内，发送方的发送只用了 0.008ms 。如果我们定义发送方（或信道）的利用率（utilization）为：发送方实际忙于将发送比特送进信道的那部分时间与发送时间之比，图 3-18a 中的分析表明了停等协议有着非常低的发送方利用率 U_{sender} ：

$$U_{\text{sender}} = \frac{L/R}{\text{RTT} + L/R} = \frac{0.008}{30.008} = 0.00027$$



a) 停等操作



b) 流水线操作

图 3-18 停等和流水线发送

这就是说，发送方只有万分之 2.7 时间是忙的。从其他角度来看，发送方在 30.008ms 内只能发送 1000 字节，有效的吞吐量仅为 267kbps，即使有 1Gbps 的链路可用！想象一个不幸的网络经理购买了一条千兆比容量的链路，但他仅能得到 267kbps 吞吐量的情况！这是一个形象的网络协议限制底层网络硬件所提供的能力的示例。而且，我们还忽略了在发送方和接收方的底层协议处理时间，以及可能出现在发送方与接收方之间的任何中间路由器上的处理与排队时延。考虑到这些因素，将进一步增加时延，使其性能更糟糕。

解决这种特殊的性能问题的一个简单方法是：不使用停等方式运行，允许发送方发送多个分组而无需等待确认，如在图 3-17b 图示的那样。图 3-18b 显示了如果发送方可以在等待确认之前发送 3 个报文，其利用率也基本上提高 3 倍。因为许多从发送方向接收方输送的分组可以被看成是填充到一条流水线中，故这种技术被称为流水线（pipelining）。流水线技术对可靠数据传输协议可带来如下影响：

- 必须增加序号范围，因为每个输送中的分组（不计算重传的）必须有一个唯一的序号，而且也许有多个在输送中未确认的报文。
- 协议的发送方和接收方两端也许必须缓存多个分组。发送方最低限度应当能缓冲那些已发送但没有确认的分组。如下面讨论的那样，接收方或许也需要缓存那些已正确接收的分组。
- 所需序号范围和对缓冲的要求取决于数据传输协议如何处理丢失、损坏及延时过大的分组。解决流水线的差错恢复有两种基本方法是：回退 N 步（Go-Back- N ，GBN）和选择重传（Selective Repeat，SR）。

3.4.3 回退 N 步

在回退 N 步（GBN）协议中，允许发送方发送多个分组（当有多个分组可用时）而不需等待确认，但它也受限于在流水线中未确认的分组数不能超过某个最大允许数 N 。在本节中我们较为详细地描述 GBN。但在继续阅读之前，建议你操作本书配套 Web 网站上的 GBN Java 小程序（这是一个非常好的 Java 程序）。

图 3-19 显示了发送方看到的 GBN 协议的序号范围。如果我们将基序号（base）定义为最早的未确认分组的序号，将下一个序号（nextseqnum）定义为最小的未使用序号（即下一个待发分组的序号），则可将序号范围分割成 4 段。在 $[0, \text{base} - 1]$ 段内的序号对应于已经发送并被确认的分组。 $[\text{base}, \text{nextseqnum} - 1]$ 段内对应已经发送但未被确认的分组。 $[\text{nextseqnum}, \text{base} + N - 1]$ 段内的序号能用于那些要被立即发送的分组，如果有数据来自上层的话。最后，大于或等于 $\text{base} + N$ 的序号是不能使用的，直到当前流水线中未被确认的分组（特别是序号为 base 的分组）已得到确认为止。

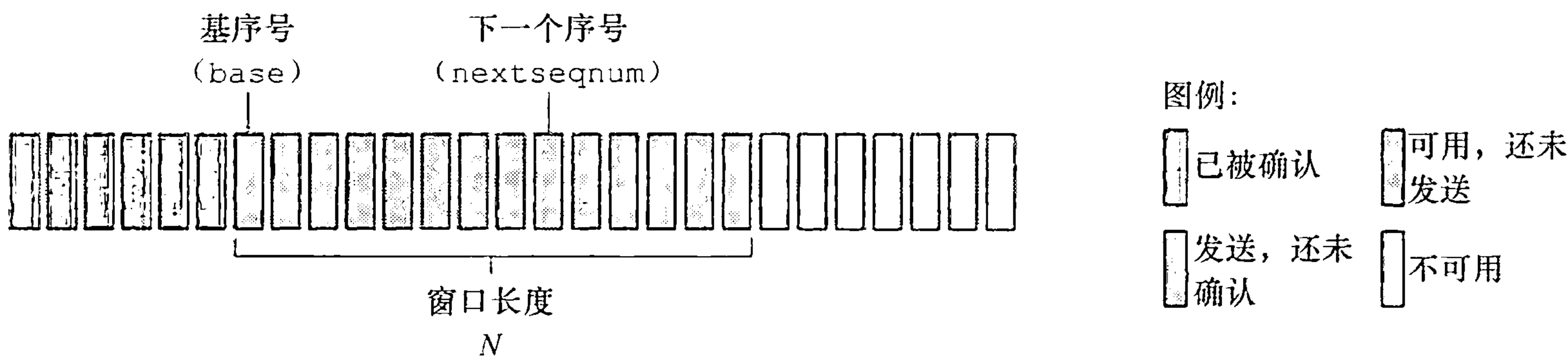


图 3-19 在 GBN 中发送方看到的序号

如图 3-19 所提示的那样，那些已被发送但还未被确认的分组的许可序号范围可以被看成是一个在序号范围内长度为 N 的窗口。随着协议的运行，该窗口在序号空间向前滑动。因此， N 常被称为窗口长度 (window size)，GBN 协议也常被称为滑动窗口协议 (sliding-window protocol)。你也许想知道，我们为什么先要限制这些被发送的、未被确认的分组的数目为 N 呢？为什么不允许这些分组为无限制的数目呢？我们将在 3.5 节看到，流量控制是对发送方施加限制的原因之一。我们将在 3.7 节学习 TCP 拥塞控制时分析另一个原因。

在实践中，一个分组的序号承载在分组首部的一个固定长度的字段中。如果分组序号字段的比特数是 k ，则该序号范围是 $[0, 2^k - 1]$ 。在一个有限的序号范围内，所有涉及序号的运算必须使用模 2^k 运算。（即序号空间可被看作是一个长度为 2^k 的环，其中序号 $2^k - 1$ 紧接着序号 0。）前面讲过，rdt 3.0 有一个 1 比特的序号，序号范围是 $[0, 1]$ 。在本章末的几道习题中探讨了一个有限的序号范围所产生的结果。我们将在 3.5 节看到，TCP 有一个 32 比特的序号字段，其中的 TCP 序号是按字节流中的字节进行计数的，而不是按分组计数。

图 3-20 和图 3-21 给出了一个基于 ACK、无 NAK 的 GBN 协议的发送方和接收方这两端的扩展 FSM 描述。我们称该 FSM 描述为扩展 FSM，是因为我们已经增加了变量（类似于编程语言中的变量）base 和 nextseqnum，还增加了对这些变量的操作以及与这些变量有关的条件动作。注意到该扩展的 FSM 规约现在变得有点像编程语言规约。[Bochman 1984] 对 FSM 扩展技术提供了一个很好的综述，也提供了用于定义协议的其他基于编程语言的技术。

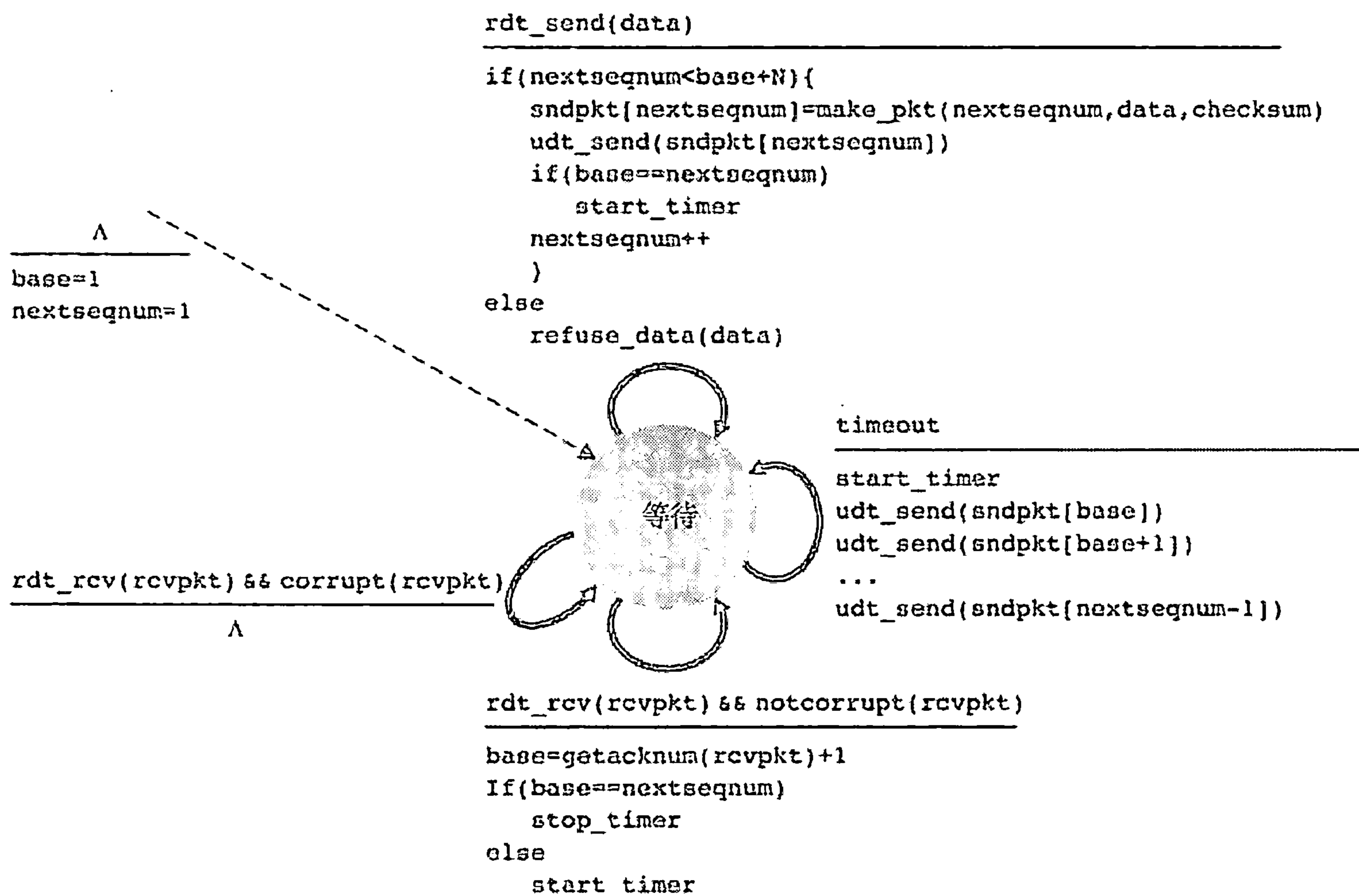


图 3-20 GBN 发送方的扩展 FSM 描述

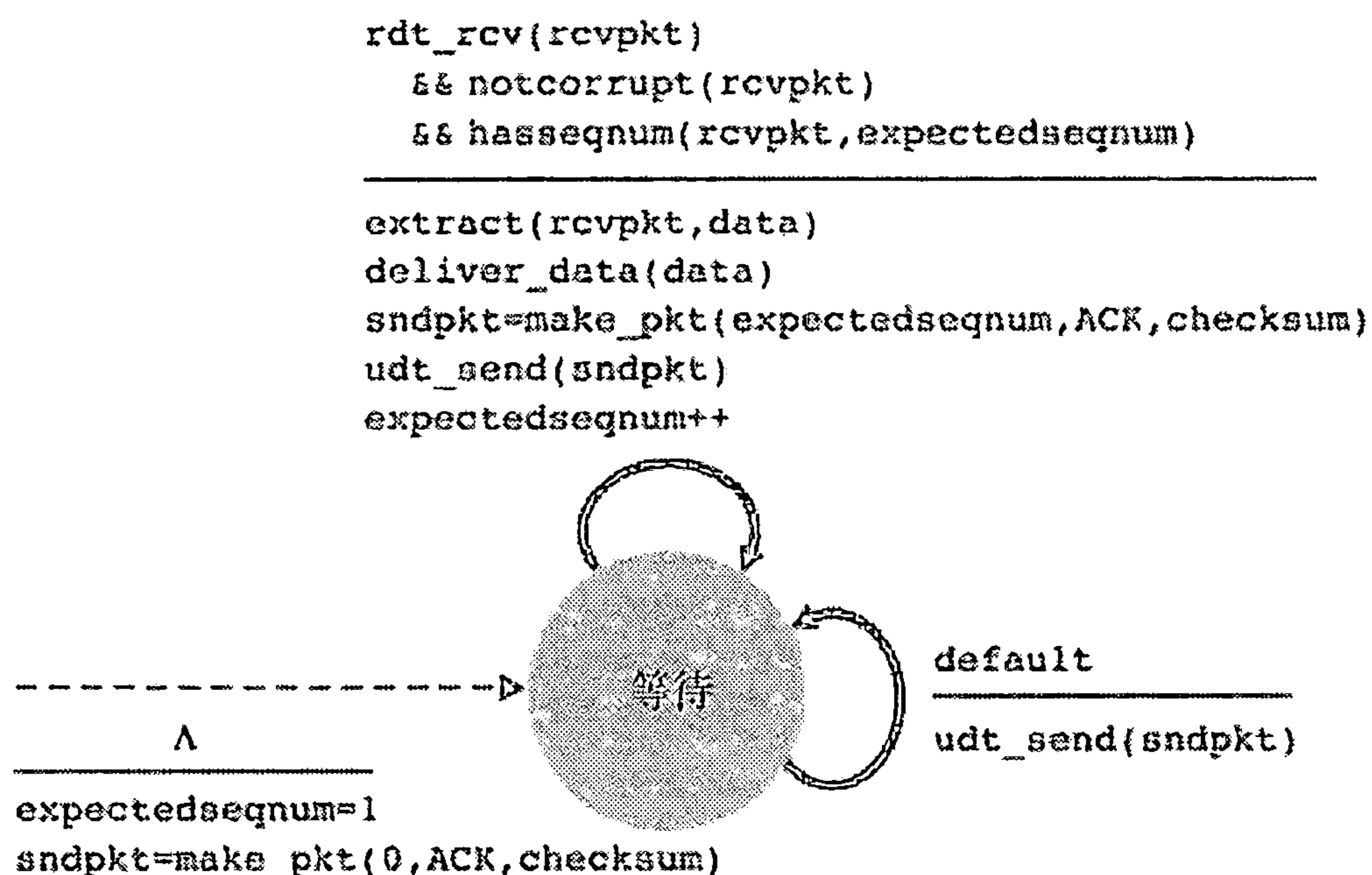


图 3-21 GBN 接收方的扩展 FSM 描述

GBN 发送方必须响应三种类型的事件：

- 上层的调用。当上层调用 `rdt_send()` 时，发送方首先检查发送窗口是否已满，即是否有 N 个已发送但未被确认的分组。如果窗口未满，则产生一个分组并将其发送，并相应地更新变量。如果窗口已满，发送方只需将数据返回给上层，隐式地指示上层该窗口已满。然后上层可能会过一会儿再试。在实际实现中，发送方更可能缓存（并不立刻发送）这些数据，或者使用同步机制（如一个信号量或标志）允许上层在仅当窗口不满时才调用 `rdt_send()`。
- 收到一个 ACK。在 GBN 协议中，对序号为 n 的分组的确认采取累积确认（cumulative acknowledgment）的方式，表明接收方已正确接收到序号为 n 的以前且包括 n 在内的所有分组。稍后讨论 GBN 接收方一端时，我们将再次研究这个主题。
- 超时事件。协议的名字“回退 N 步”来源于出现丢失和时延过长分组时发送方的行为。就像在停等协议中那样，定时器将再次用于恢复数据或确认分组的丢失。如果出现超时，发送方重传所有已发送但还未被确认过的分组。图 3-20 中的发送方仅使用一个定时器，它可被当作是最早的已发送但未被确认的分组所使用的定时器。如果收到一个 ACK，但仍有已发送但未被确认的分组，则定时器被重新启动。如果没有已发送但未被确认的分组，该定时器被终止。

在 GBN 中，接收方的动作也很简单。如果一个序号为 n 的分组被正确接收到，并且按序（即上次交付给上层的数据是序号为 $n-1$ 的分组），则接收方为分组 n 发送一个 ACK，并将该分组中的数据部分交付到上层。在所有其他情况下，接收方丢弃该分组，并为最近按序接收的分组重新发送 ACK。注意到因为一次交付给上层一个分组，如果分组 k 已接收并交付，则所有序号比 k 小的分组也已经交付。因此，使用累积确认是 GBN 一个自然的选择。

在 GBN 协议中，接收方丢弃所有失序分组。尽管丢弃一个正确接收（但失序）的分组有点愚蠢和浪费，但这样做是有理由的。前面讲过，接收方必须按序将数据交付给上层。假定现在期望接收分组 n ，而分组 $n+1$ 却到了。因为数据必须按序交付，接收方可能缓存（保存）分组 $n+1$ ，然后，在它收到并交付分组 n 后，再将该分组交付到上层。然而，如果分组 n 丢失，则该分组及分组 $n+1$ 最终将在发送方根据 GBN 重传规则而被重

传。因此，接收方只需丢弃分组 $n + 1$ 即可。这种方法的优点是接收缓存简单，即接收方不需要缓存任何失序分组。因此，虽然发送方必须维护窗口的上下边界及 `nextseqnum` 在该窗口中的位置，但是接收方需要维护的唯一信息就是下一个按序接收的分组的序号。该值保存在 `expectedseqnum` 变量中，如图 3-21 中接收方 FSM 所示。当然，丢弃一个正确接收的分组的缺点是随后对该分组的重传也许会丢失或出错，因此甚至需要更多的重传。

图 3-22 给出了窗口长度为 4 个分组的 GBN 协议的运行情况。因为该窗口长度的限制，发送方发送分组 0~3，然后在继续发送之前，必须等待直到一个或多个分组被确认。当接收到每一个连续的 ACK（例如 ACK 0 和 ACK 1）时，该窗口便向前滑动，发送方便可以发送新的分组（分别是分组 4 和分组 5）。在接收方，分组 2 丢失，因此分组 3、4 和 5 被发现是失序分组并被丢弃。

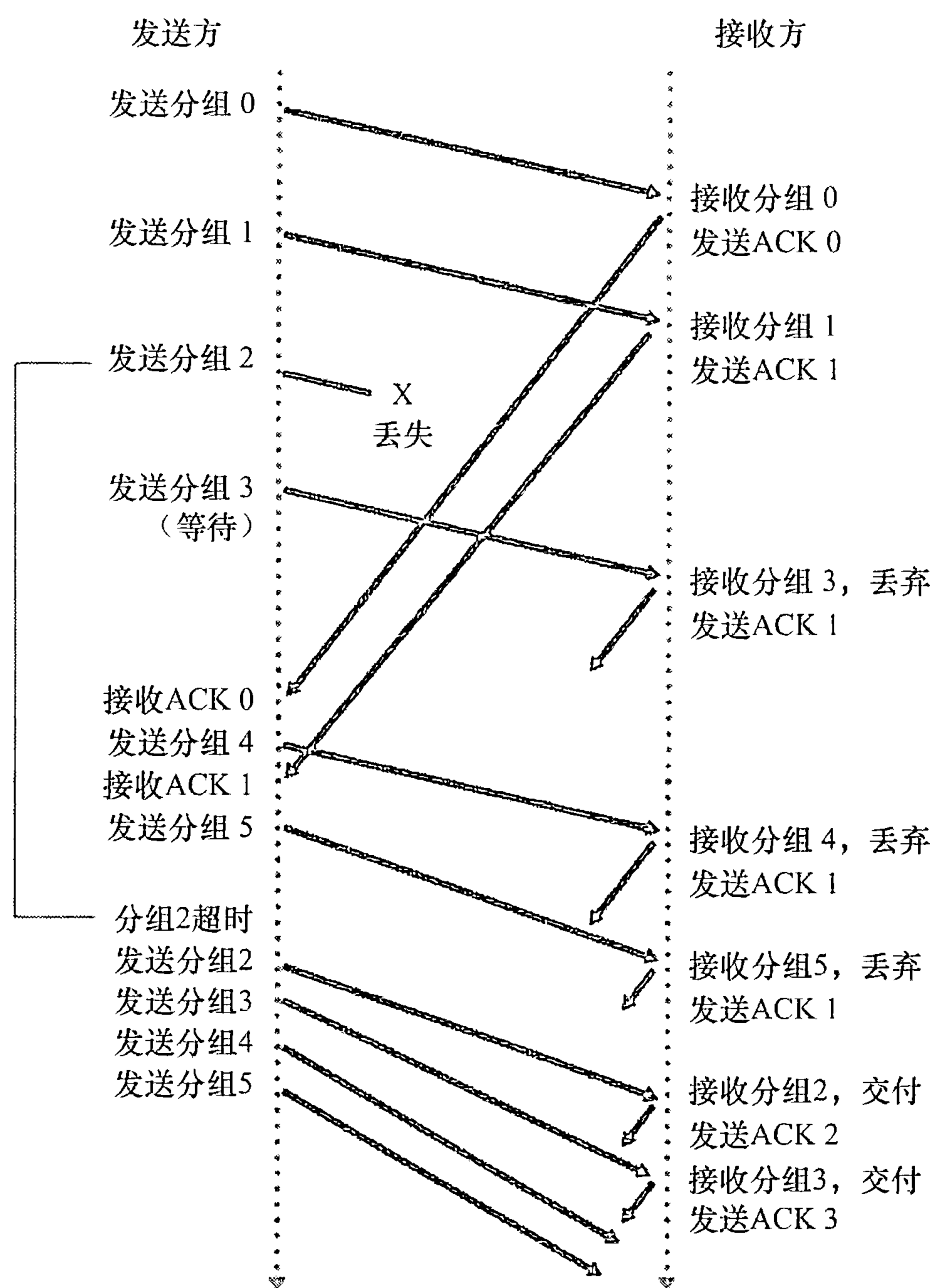


图 3-22 运行中的 GBN

在结束对 GBN 的讨论之前，需要提请注意的是，在协议栈中实现该协议可能与图 3-20 中的扩展 FSM 有相似的结构。该实现也可能是以各种过程形式出现，每个过程实现了在响应各种可能出现的事件时要采取的动作。在这种基于事件的编程（event-based programming）方式中，这些过程要么被协议栈中的其他过程调用，要么作为一次中断的结果。在发送方，这些事件包括：①来自上层实体的调用去调用 `rdt_send()`；②定时器中断；③报文到达时，来自下层的调用去调用 `rdt_rcv()`。本章后面的编程作业会使你有一个机会在一个模拟而真实的网络环境中实际实现这些例程。

这里我们注意到，GBN 协议中综合了我们将在 3.5 节中学习 TCP 可靠数据传输构件时遇到的所有技术。这些技术包括使用序号、累积确认、检验和以及超时/重传操作。

3.4.4 选择重传

在图 3-17 中，GBN 协议潜在地允许发送方用多个分组“填充流水线”，因此避免了停等协议中所提到的信道利用率问题。然而，GBN 本身也有一些情况存在着性能问题。尤其是当窗口长度和带宽时延积都很大时，在流水线中会有很多分组更是如此。单个分组的差错就能够引起 GBN 重传大量分组，许多分组根本没有必要重传。随着信道差错率的增加，流水线可能会被这些不必要重传的分组所充斥。想象一下，在我们口述消息的例子中，如果每次有一个单词含糊不清，其前后 1000 个单词（例如，窗口长度为 1000 个单词）不得不被重传的情况。此次口述会由于这些反复述说的单词而变慢。

顾名思义，选择重传（SR）协议通过让发送方仅重传那些它怀疑在接收方出错（即丢失或受损）的分组而避免了不必要的重传。这种个别的、按需的重传要求接收方逐个地确认正确接收的分组。再次用窗口长度 N 来限制流水线中未完成、未被确认的分组数。然而，与 GBN 不同的是，发送方已经收到了对窗口中某些分组的 ACK。图 3-23 显示了 SR 发送方看到的序号空间。图 3-24 详细描述了 SR 发送方所采取的动作。

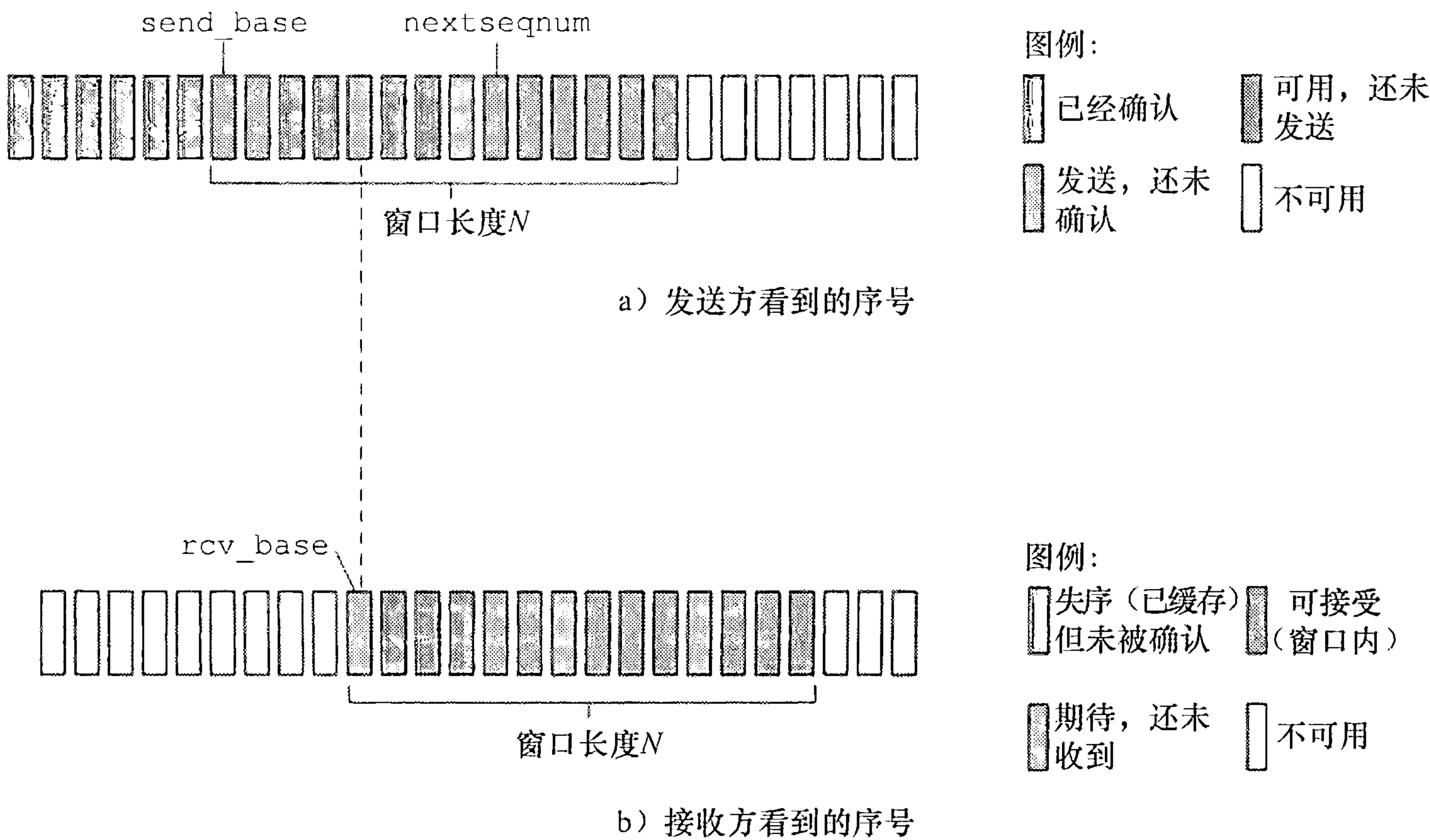


图 3-23 选择重传（SR）发送方与接收方的序号空间

1. 从上层收到数据。当从上层接收到数据后，SR 发送方检查下一个可用于该分组的序号。如果序号位于发送方的窗口内，则将数据打包并发送；否则就像在 GBN 中一样，要么将数据缓存，要么将其返回给上层以便以后传输。
 2. 超时。定时器再次被用来防止丢失分组。然而，现在每个分组必须拥有其自己的逻辑定时器，因为超时发生后只能发送一个分组。可以使用单个硬件定时器模拟多个逻辑定时器的操作 [Varghese 1997]。
 3. 收到 ACK。如果收到 ACK，倘若该分组序号在窗口内，则 SR 发送方将那个被确认的分组标记为已接收。如果该分组的序号等于 `send_base`，则窗口基序号向前移动到具有最小序号的未确认分组处。如果窗口移动了并且有序号落在窗口内的未发送分组，则发送这些分组。

图 3-24 SR 发送方的事件与动作

SR 接收方将确认一个正确接收的分组而不管其是否按序。失序的分组将被缓存直到所有丢失分组（即序号更小的分组）皆被收到为止，这时才可以将一批分组按序交付给上层。图 3-25 详细列出了 SR 接收方所采用的各种动作。图 3-26 给出了一个例子以说明出现丢包时 SR 的操作。值得注意的是，在图 3-26 中接收方初始时缓存了分组 3、4、5，并在最终收到分组 2 时，才将它们一并交付给上层。

1. 序号在 $[\text{rcv_base}, \text{rcv_base} + N - 1]$ 内的分组被正确接收。在此情况下，收到的分组落在接收方的窗口内，一个选择 ACK 被回送给发送方。如果该分组以前没收到过，则缓存该分组。如果该分组的序号等于接收窗口的基序号（图 3-23 中的 rcv_base ），则该分组以及以前缓存的序号连续的（起始于 rcv_base 的）分组交付给上层。然后，接收窗口按向前移动分组的编号向上交付这些分组。举个例子来说，考虑一下图 3-26。当收到一个序号为 $\text{rcv_base} = 2$ 的分组时，该分组及分组 3、4、5 可被交付给上层。
2. 序号在 $[\text{rcv_base} - N, \text{rcv_base} - 1]$ 内的分组被正确收到。在此情况下，必须产生一个 ACK，即使该分组是接收方以前已确认过的分组。
3. 其他情况。忽略该分组。

图 3-25 SR 接收方的事件与动作

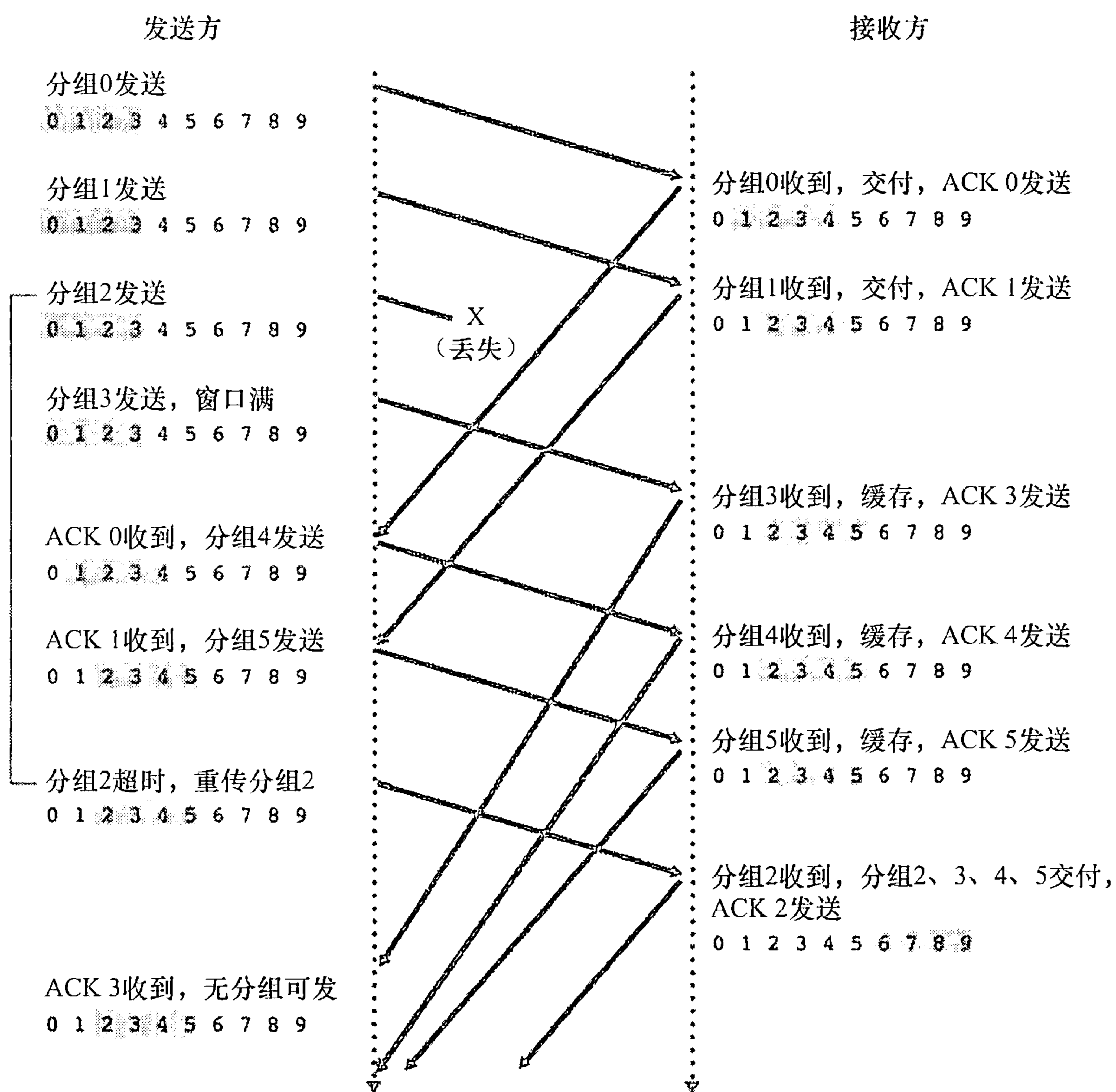
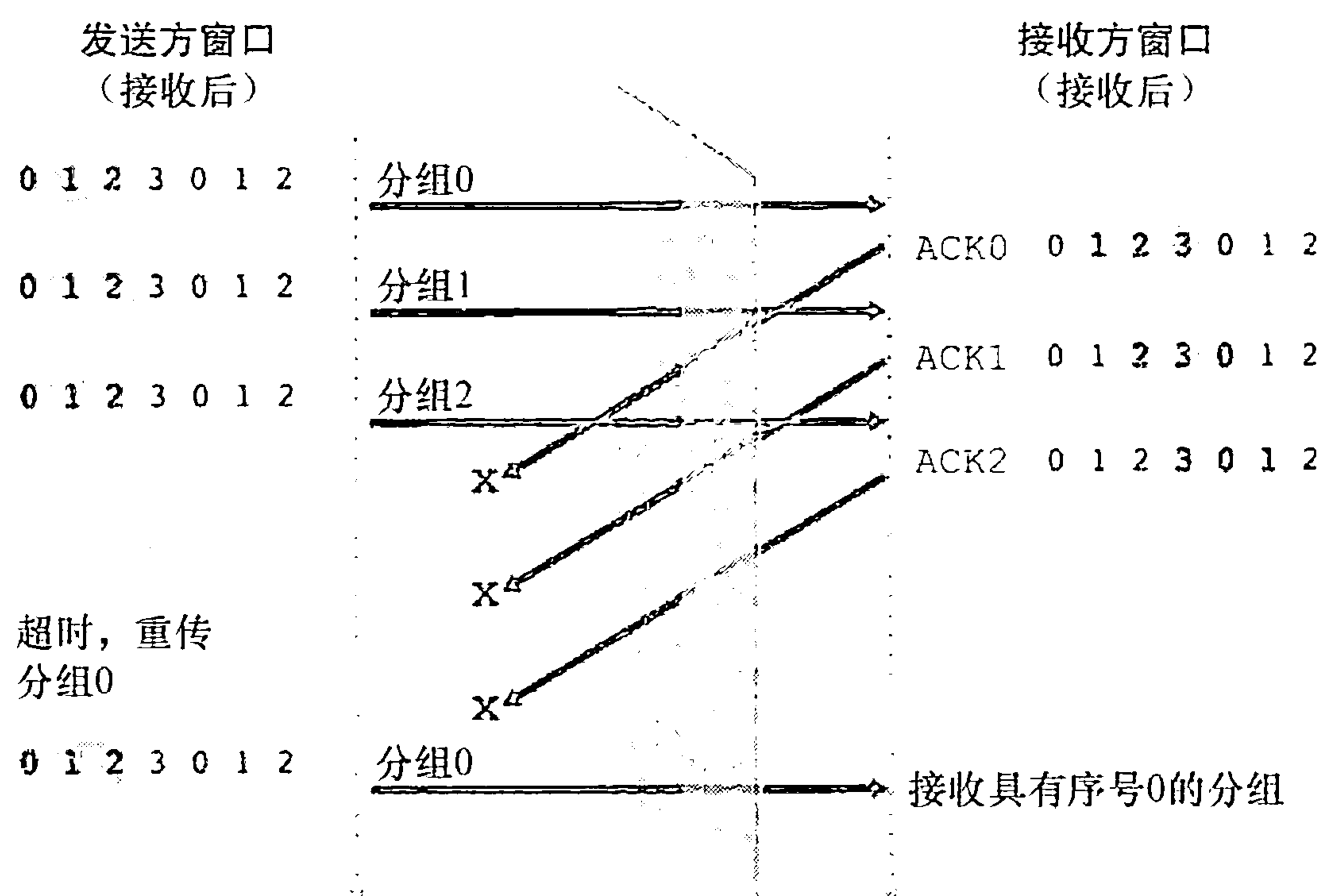


图 3-26 SR 操作

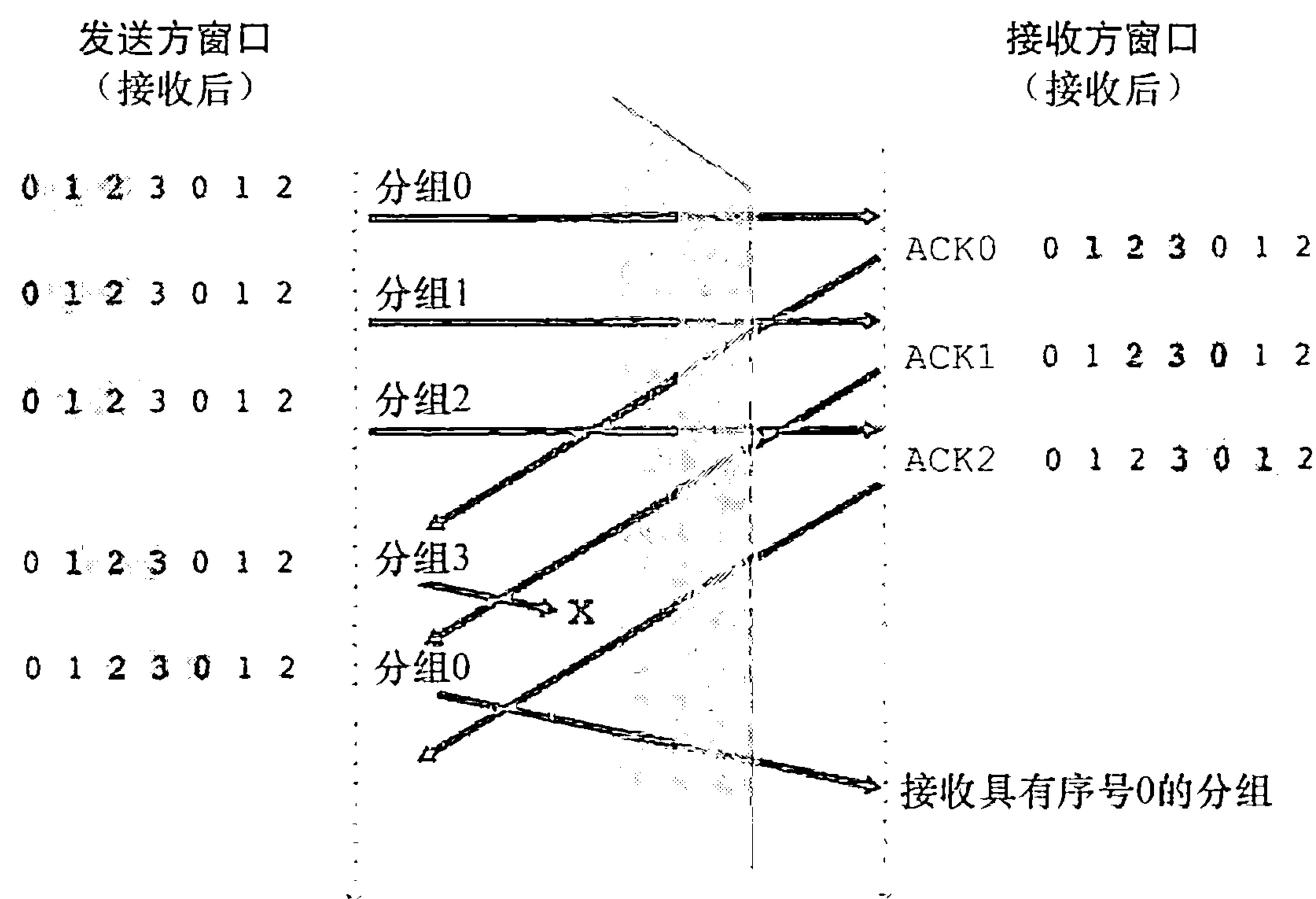
注意到图 3-25 中的第二步很重要，接收方重新确认（而不是忽略）已收到过的那些序号小于当前窗口基序号的分组。你应该理解这种重新确认确实是需要。例如，给定在图 3-23 中所示的发送方和接收方的序号空间，如果分组 send_base 的 ACK 没有从接收方传

播回发送方，则发送方最终将重传分组 `send_base`，即使显然（对我们而不是对发送方来说！）接收方已经收到了该分组。如果接收方不确认该分组，则发送方窗口将永远不能向前滑动！这个例子说明了 SR 协议（和很多其他协议一样）的一个重要方面。对于哪些分组已经被正确接收，哪些没有，发送方和接收方并不总是能看到相同的结果。对 SR 协议而言，这就意味着发送方和接收方的窗口并不总是一致。

当我们面对有限序号范围的现实时，发送方和接收方窗口间缺乏同步会产生严重的后果。考虑下面例子中可能发生的情况，该例有包括 4 个分组序号 0、1、2、3 的有限序号范围且窗口长度为 3。假定发送了分组 0 至 2，并在接收方被正确接收且确认了。此时，接收方窗口落在第 4、5、6 个分组上，其序号分别为 3、0、1。现在考虑两种情况。在第一种情况下，如图 3-27a 所示，对前 3 个分组的 ACK 丢失，因此发送方重传这些分组。因此，接收方下一步要接收序号为 0 的分组，即第一个发送分组的副本。



a)



b)

图 3-27 SR 接收方窗口太大的困境：是一个新分组还是一次重传

在第二种情况下，如图 3-27b 所示，对前 3 个分组的 ACK 都被正确交付。因此发送方向前移动窗口并发送第 4、5、6 个分组，其序号分别为 3、0、1。序号为 3 的分组丢失，但序号为 0 的分组到达（一个包含新数据的分组）。

现在考虑一下图 3-27 中接收方的观点，在发送方和接收方之间有一副假想的帘子，因为接收方不能“看见”发送方采取的动作。接收方所能观察到的是它从信道中收到的以及它向信道中发出报文序列。就其所关注的而言，图 3-27 中的两种情况是等同的。没有办法区分是第一个分组的重传还是第 5 个分组的初次传输。显然，窗口长度比序号空间小 1 时协议无法工作。但窗口必须多小呢？本章后面的一道习题请你说明为何对于 SR 协议而言，窗口长度比须小于或等于序号空间大小的一半。

在本书配套的网站上，可以找到一个模仿 SR 协议运行的 Java 小程序。尝试进行你以前对 GBN Java 小程序所进行的相同的实验。这些结果与你期望的一致吗？

至此我们结束了对可靠数据传输协议的讨论。我们已涵盖许多基础知识，并介绍了多种机制，这些机制可一起提供可靠数据传输。表 3-1 总结这些机制。既然我们已经学习了所有这些运行中的机制，并能看到“全景”，我们建议你再复习一遍本节内容，看看这些机制是怎样逐步被添加进来，以涵盖复杂性渐增的（现实的）连接发送方与接收方的各种信道模型的，或者如何改善协议性能的。

表 3-1 可靠数据传输机制及其用途的总结

机制	用途和说明
检验和	用于检测在一个传输分组中的比特错误
定时器	用于超时/重传一个分组，可能因为该分组（或其 ACK）在信道中丢失了。由于当一个分组延时但未丢失（过早超时），或当一个分组已被接收方收到但从接收方到发送方的 ACK 丢失时，可能产生超时事件，所以接收方可能会收到一个分组的多个冗余副本
序号	用于为从发送方流向接收方的数据分组按顺序编号。所接收分组的序号间的空隙可使接收方检测出丢失的分组。具有相同序号的分组可使接收方检测出一个分组的冗余副本
确认	接收方用于告诉发送方一个分组或一组分组已被正确地接收到了。确认报文通常携带着被确认的分组或多个分组的序号。确认可以是逐个的或累积的，这取决于协议
否定确认	接收方用于告诉发送方某个分组未被正确地接收。否定确认报文通常携带着未被正确接收的分组的序号
窗口、流水线	发送方也许被限制仅发送那些序号落在一个指定范围内的分组。通过允许一次发送多个分组但未被确认，发送方的利用率可在停等操作模式的基础上得到增加。我们很快将会看到，窗口长度可根据接收方接收和缓存报文的能力、网络中的拥塞程度或两者情况来进行设置

我们通过考虑在底层信道模型中的一个遗留假设来结束对可靠数据传输协议的讨论。前面讲过，我们曾假定分组在发送方与接收方之间的信道中不能被重新排序。这在发送方与接收方由单段物理线路相连的情况下，通常是一个合理的假设。然而，当连接两端的“信道”是一个网络时，分组重新排序是可能会发生的。分组重新排序的一个表现就是，一个具有序号或确认号 x 的分组的旧副本可能会出现，即使发送方或接收方的窗口中都没有包含 x 。对于分组重新排序，信道可被看成基本上是在缓存分组，并在将来任意时刻自然地释放出这些分组。由于序号可以被重新使用，那么必须小心，以免出现这样的冗余分组。实际应用中采用的方法是，确保一个序号不被重新使用，直到发送方“确信”任何先前发送的序号为 x 的分组都不再在网络中为止。通过假定一个分组在网络中的“存活”时间不会超过某个固定最大时间量来做到这一点。在高速网络的 TCP 扩展中，最长的分组寿

命被假定为大约 3 分钟 [RFC 1323]。[Sunshine 1978] 描述了一种使用序号的方法，它可使重新排序问题完全避免。

3.5 面向连接的运输：TCP

既然我们已经学习了可靠数据传输的基本原理，我们就可以转而学习 TCP 了。TCP 是因特网运输层的面向连接的可靠的运输协议。我们在本节中将看到，为了提供可靠数据传输，TCP 依赖于前一节所讨论的许多基本原理，其中包括差错检测、重传、累积确认、定时器以及用于序号和确认号的首部字段。TCP 定义在 RFC 793、RFC 1122、RFC 1323、RFC 2018 以及 RFC 2581 中。

3.5.1 TCP 连接

TCP 被称为是面向连接的 (connection-oriented)，这是因为在一个应用进程可以开始向另一个应用进程发送数据之前，这两个进程必须先相互“握手”，即它们必须相互发送某些预备报文段，以建立确保数据传输的参数。作为 TCP 连接建立的一部分，连接的双方都将初始化与 TCP 连接相关的许多 TCP 状态变量（其中的许多状态变量将在本节和 3.7 节中讨论）。

历史事件

Vinton Cerf 和 Robert Kahn 与 TCP/IP

在 20 世纪 70 年代早期，分组交换网开始飞速增长，而因特网的前身 ARPAnet 也只是当时众多分组交换网中的一个。这些网络都有它们各自的协议。Vinton Cerf 和 Robert Kahn 这两个研究人员认识到互联这些网络的重要性，发明了沟通网络的 TCP/IP 协议，该协议代表传输控制协议/网际协议 (Transmission Control Protocol/Internet Protocol)。虽然 Cerf 和 Kahn 开始时把该协议看成是单一的实体，但是后来将它分成单独运行的两个部分：TCP 和 IP。Cerf 和 Kahn 在 1974 年 5 月的《IEEE Transactions on Communications Technology》杂志上发表了一篇关于 TCP/IP 的论文。

TCP/IP 协议是当今因特网的支柱性协议，但它的发明先于 PC、工作站、智能手机和平板电脑，先于以太网、DSL、WiFi 和其他接入网技术的激增，先于 Web、社交媒体和流式视频等。Cerf 和 Kahn 看到了对于联网协议的需求，一方面为行将定义的应用提供广泛的支持，另一方面允许任何主机与链路层协议互操作。

2004 年，Cerf 和 Kahn 由于“联网方面的开创性工作（包括因特网的基本通信协议 TCP/IP 的设计和实现）以及联网方面富有才能的领导”而获得 ACM 图灵奖，该奖项被认为是“计算机界的诺贝尔奖”。

这种 TCP “连接”不是一条像在电路交换网络中的端到端 TDM 或 FDM 电路，也不是一条虚电路（参见第 1 章），因为其连接状态完全保留在两个端系统中。由于 TCP 协议只在端系统中运行，而不在中间的网络元素（路由器和链路层交换机）中运行，所以中间的网络元素不会维持 TCP 连接状态。事实上，中间路由器对 TCP 连接完全视而不见，它们

看到的是数据报，而不是连接。

TCP 连接提供的是**全双工服务**（full-duplex service）：如果一台主机上的进程 A 与另一台主机上的进程 B 存在一条 TCP 连接，那么应用层数据就可在从进程 B 流向进程 A 的同时，也从进程 A 流向进程 B。TCP 连接也总是**点对点**（point-to-point）的，即在单个发送方与单个接收方之间的连接。所谓“多播”（参见 4.7 节），即在一次发送操作中，从一个发送方将数据传送给多个接收方，对 TCP 来说这是不可能的。对于 TCP 而言，两台主机是一对，而 3 台主机则太多！

我们现在来看看 TCP 连接是怎样建立的。假设运行在某台主机上的一个进程想与另一台主机上的一个进程建立一条连接。前面讲过，发起连接的这个进程被称为**客户进程**，而另一个进程被称为**服务器进程**。该客户应用进程首先要通知客户运输层，它想与服务器上的一个进程建立一条连接。2.7.2 节讲过，一个 Python 客户程序通过发出下面的命令来实现此目的。

```
clientSocket.connect((serverName,serverPort))
```

其中 serverName 是服务器的名字，serverPort 标识了服务器上的进程。客户上的 TCP 便开始与服务器上的 TCP 建立一条 TCP 连接。我们将在本节后面更为详细地讨论连接建立的过程。现在知道下列事实就可以了：客户首先发送一个特殊的 TCP 报文段，服务器用另一个特殊的 TCP 报文段来响应，最后，客户再用第三个特殊报文段作为响应。前两个报文段不承载“有效载荷”，也就是不包含应用层数据；而第三个报文段可以承载有效载荷。由于在这两台主机之间发送了 3 个报文段，所以这种连接建立过程常被称为**三次握手**（three-way handshake）。

一旦建立起一条 TCP 连接，两个应用进程之间就可以相互发送数据了。我们考虑一下从客户进程向服务器进程发送数据的情况。如 2.7 节中所述，客户进程通过套接字（该进程之门）传递数据流。数据一旦通过该门，它就由客户中运行的 TCP 控制了。如图 3-28 所示，TCP 将这些数据引导到该连接的**发送缓存**（send buffer）里，发送缓存是在三次握手初期设置的缓存之一。接下来 TCP 就会不时从发送缓存里取出一块数据。有趣的是，在 TCP 规范 [RFC 793] 中却没提及 TCP 应何时实际发送缓存里的数据，只是描述为“TCP 应该在它方便的时候以报文段的形式发送数据”。TCP 可从缓存中取出并放入报文段中的数据数量受限于**最大报文段长度**（Maximum Segment Size, MSS）。MSS 通常根据最初确定的由本地发送主机发送的最大链路层帧长度（即所谓的**最大传输单元**（Maximum Transmission Unit, MTU））来设置。设置该 MSS 要保证一个 TCP 报文段（当封装在一个 IP 数据报中）加上 TCP/IP 首部长度（通常 40 字节）将适合单个链路层帧。以太网和 PPP 链路层协议都具有 1500 字节的 MTU，因此 MSS 的典型值为 1460 字节。已经提出了多种发现路径 MTU 的方法，并基于路径 MTU 值设置 MSS（路径 MTU 是指能在从源到目的地的所有链路上发送的最大链路层帧 [RFC 1191]）。注意到 MSS 是指在报文段里应用层数据的最大长度，而不是指包括 TCP 首部的 TCP 报文段的最大长度。（该术语很容易混淆，但是我们不得不采用它，因为它已经根深蒂固了。）

TCP 为每块客户数据配上一个 TCP 首部，从而形成多个 TCP 报文段（TCP segment）。这些报文段被下传给网络层，网络层将其分别封装在网络层 IP 数据报中。然后这些 IP 数据报被发送到网络中。当 TCP 在另一端接收到一个报文段后，该报文段的数据就被放入该 TCP 连接的接收缓存中，如图 3-28 中所示。应用程序从此缓存中读取数据流。TCP 连接的

每一端都有各自的发送缓存和接收缓存。（读者可以参见 <http://www.awl.com/kurose-ross> 处的在线流控制 Java 小程序，它提供了关于发送缓存和接收缓存的一个动画演示。）

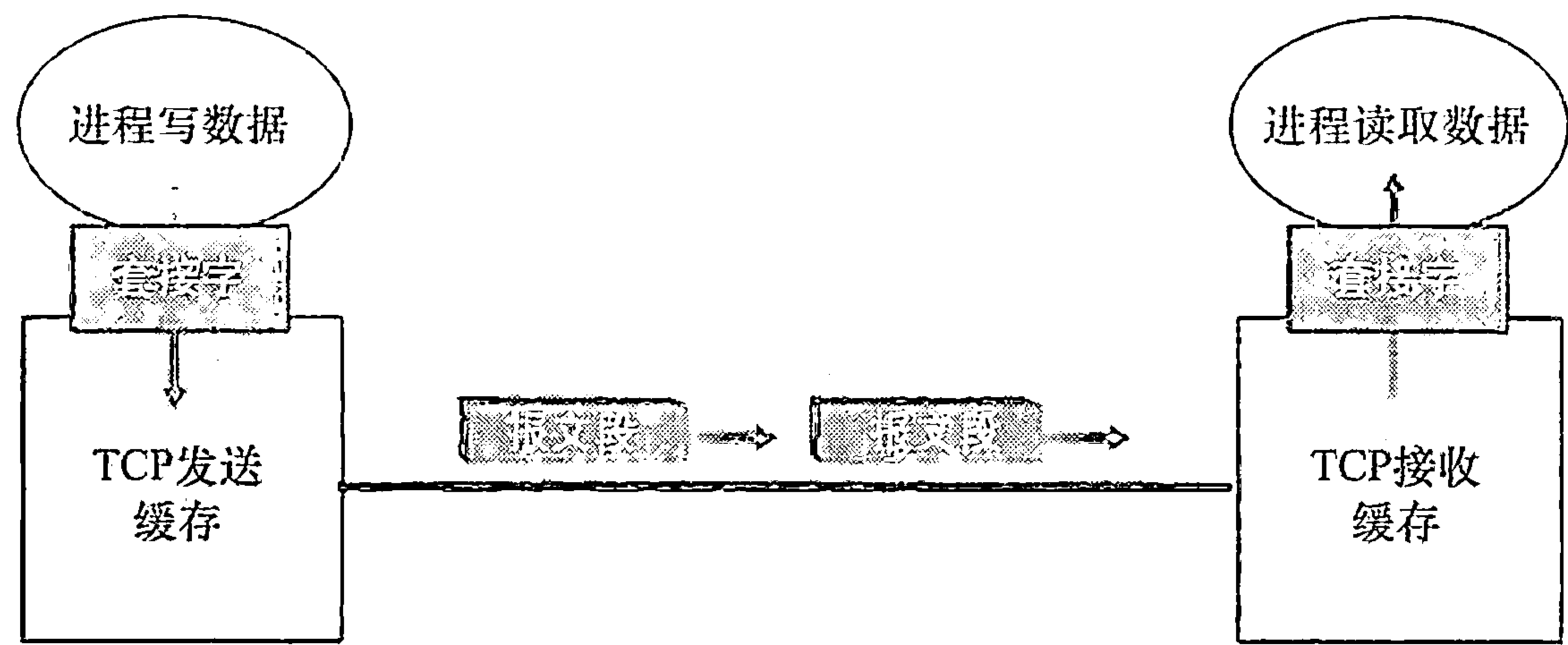


图 3-28 TCP 发送缓存和接收缓存

从以上讨论中我们可以看出，TCP 连接的组成包括：一台主机上的缓存、变量和与进程连接的套接字，以及另一台主机上的另一组缓存、变量和与进程连接的套接字。如前面讲过的那样，在这两台主机之间的网络元素（路由器、交换机和中继器）中，没有为该连接分配任何缓存和变量。

3.5.2 TCP 报文段结构

简要地了解了 TCP 连接后，我们研究一下 TCP 报文段结构。TCP 报文段由首部字段和一个数据字段组成。数据字段包含一块应用数据。如前所述，MSS 限制了报文段数据字段的最大长度。当 TCP 发送一个大文件，例如某 Web 页面上的一个图像时，TCP 通常是 将该文件划分成长度为 MSS 的若干块（最后一块除外，它通常小于 MSS）。然而，交互式应用通常传送长度小于 MSS 的数据块。例如，对于像 Telnet 这样的远程登录应用，其 TCP 报文段的数据字段经常只有一个字节。由于 TCP 的首部一般是 20 字节（比 UDP 首部多 12 字节），所以 Telnet 发送的报文段也许只有 21 字节长。

图 3-29 显示了 TCP 报文段的结构。与 UDP 一样，首部包括源端口号和目的端口号，它被用于多路复用/分解来自或送到上层应用的数据。另外，同 UDP 一样，TCP 首部也包括检验和字段（checksum field）。TCP 报文段首部还包含下列字段：

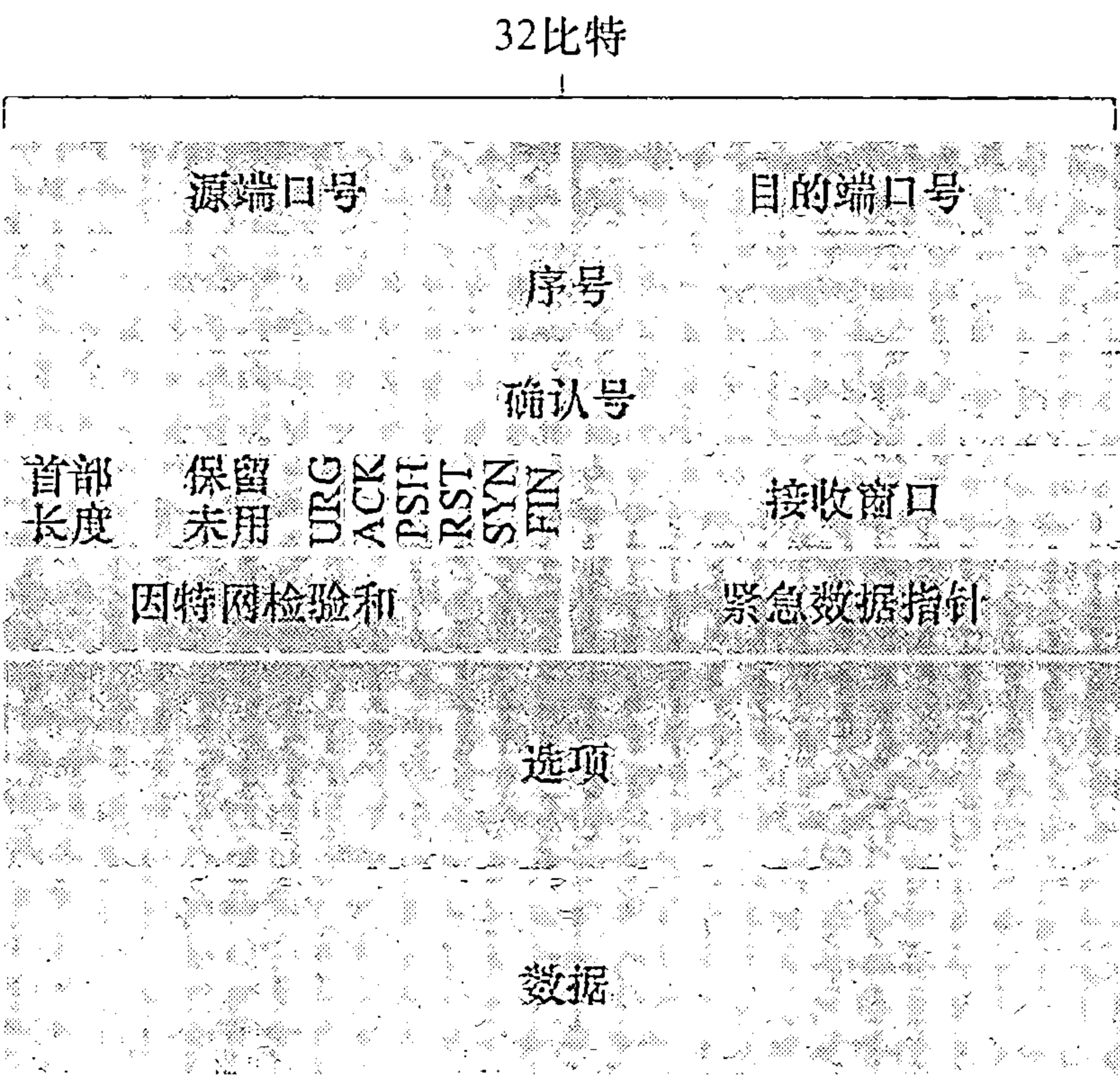


图 3-29 TCP 报文段结构

- 32 比特的序号字段（sequence number field）和 32 比特的确认号字段（acknowledgment number field）。这些字段被 TCP 发送方和接收方用来实现可靠数据传输服务，讨论见后。
- 16 比特的接收窗口字段（receive window field），该字段用于流量控制。我们很快就会看到，该字段用于指示接收方愿意接受的字节数量。

- 4 比特的首部长度字段 (header length field)，该字段指示了以 32 比特的字为单位的 TCP 首部长度。由于 TCP 选项字段的原因，TCP 首部的长度是可变的。(通常，选项字段为空，所以 TCP 首部的典型长度就是 20 字节。)
- 可选与变长的选项字段 (options field)，该字段用于发送方与接收方协商最大报文段长度 (MSS) 时，或在高速网络环境下用作窗口调节因子时使用。首部字段中还定义了一个时间戳选项。可参见 RFC 854 和 RFC 1323 了解其他细节。
- 6 比特的标志字段 (flag field)。ACK 比特用于指示确认字段中的值是有效的，即该报文段包括一个对已被成功接收报文段的确认。RST、SYN 和 FIN 比特用于连接建立和拆除，我们将在本节后面讨论该问题。当 PSH 比特被设置的时候，就指示接收方应立即将数据交给上层。最后，URG 比特用来指示报文段里存在着被发送端的上层实体置为“紧急”的数据。紧急数据的最后一个字节由 16 比特的紧急数据指针字段指出。当紧急数据存在并给出指向紧急数据尾的指针的时候，TCP 必须通知接收端的上层实体。(在实践中，PSH、URG 和紧急数据指针并没有使用。为了完整性起见，我们才提到这些字段。)

1. 序号和确认号

TCP 报文段首部中两个最重要的字段是序号字段和确认号字段。这两个字段是 TCP 可靠传输服务的关键部分。但是在讨论这两个字段是如何用于提供可靠数据传输之前，我们首先来解释一下 TCP 在这两个字段中究竟放置了什么。

TCP 把数据看成一个无结构的、有序的字节流。我们从 TCP 对序号的使用上可以看出这一点，因为序号是建立在传送的字节流之上，而不是建立在传送的报文段的序列之上。一个报文段的序号 (sequence number for a segment) 因此是该报文段首字节的字节流编号。举例来说，假设主机 A 上的一个进程想通过一条 TCP 连接向主机 B 上的一个进程发送一个数据流。主机 A 中的 TCP 将隐式地对数据流中的每一个字节编号。假定数据流由一个包含 500 000 字节的文件组成，其 MSS 为 1000 字节，数据流的首字节编号是 0。如图 3-30 所示，该 TCP 将为该数据流构建 500 个报文段。给第一个报文段分配序号 0，第二个报文段分配序号 1000，第三个报文段分配序号 2000，以此类推。每一个序号被填入到相应 TCP 报文段首部的序号字段中。

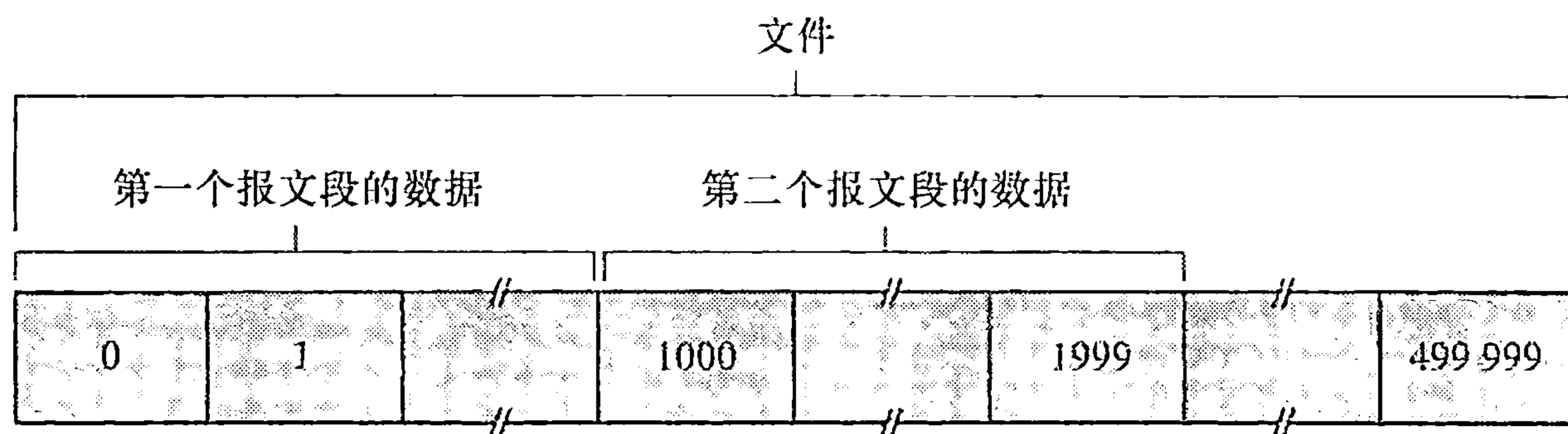


图 3-30 文件数据划分成 TCP 报文段

现在我们考虑一下确认号。确认号要比序号难处理一些。前面讲过，TCP 是全双工的，因此主机 A 在向主机 B 发送数据的同时，也许也接收来自主机 B 的数据（都是同一条 TCP 连接的一部分）。从主机 B 到达的每个报文段中都有一个序号用于从 B 流向 A 的数据。主机 A 填充进报文段的确认号是主机 A 期望从主机 B 收到的下一字节的序号。看一些例子有助于理解实际发生的事情。假设主机 A 已收到了来自主机 B 的编号为 0 ~ 535 的

所有字节，同时假设它打算发送一个报文段给主机 B。主机 A 等待主机 B 的数据流中字节 536 及之后的所有字节。所以主机 A 就会在它发往主机 B 的报文段的确认号字段中填上 536。

再举一个例子，假设主机 A 已收到一个来自主机 B 的包含字节 0 ~ 535 的报文段，以及另一个包含字节 900 ~ 1000 的报文段。由于某种原因，主机 A 还没有收到字节 536 ~ 899 的报文段。在这个例子中，主机 A 为了重新构建主机 B 的数据流，仍在等待字节 536（和其后的字节）。因此，A 到 B 的下一个报文段将在确认号字段中包含 536。因为 TCP 只确认该流中至第一个丢失字节为止的字节，所以 TCP 被称为提供累积确认（cumulative acknowledgment）。

最后一个例子也会引发一个重要而微妙的问题。主机 A 在收到第二个报文段（字节 536 ~ 899）之前收到第三个报文段（字节 900 ~ 1000）。因此，第三个报文段失序到达。该微妙的问题是：当主机在一条 TCP 连接中收到失序报文段时该怎么办？有趣的是，TCP RFC 并没有为此明确规定任何规则，而是把这一问题留给实现 TCP 的编程人员去处理。他们有两个基本的选择：①接收方立即丢弃失序报文段（如前所述，这可以简化接收方的设计）；②接收方保留失序的字节，并等待缺少的字节以填补该间隔。显然，后一种选择对网络带宽而言更为有效，是实践中采用的方法。

在图 3-30 中，我们假设初始序号为 0。事实上，一条 TCP 连接的双方均可随机地选择初始序号。这样做可以减少将那些仍在网络中存在的来自两台主机之间先前已终止的连接的报文段，误认为是后来这两台主机之间新建连接所产生的有效报文段的可能性（它碰巧与旧连接使用了相同的端口号）[Sunshine 1978]。

2. Telnet：序号和确认号的一个学习案例

Telnet 由 RFC 854 定义，它现在是一个用于远程登录的流行应用层协议。它运行在 TCP 之上，被设计成可在任意一对主机之间工作。Telnet 与我们第 2 章讨论的批量数据传输应用不同，它是一个交互式应用。我们在此讨论一个 Telnet 例子，因为该例子很好地阐述 TCP 的序号与确认号。我们注意到许多用户现在更愿意采用 SSH 协议而不是 Telnet，因为在 Telnet 连接中发送的数据（包括口令！）是没有加密的，使得 Telnet 易于受到窃听攻击（如在 8.7 节中讨论的那样）。

假设主机 A 发起一个与主机 B 的 Telnet 会话。因为是主机 A 发起该会话，因此它被标记为客户，而主机 B 被标记为服务器。（在客户端的）用户键入的每个字符都会被发送至远程主机；远程主机将回送每个字符的副本给客户，并将这些字符显示在 Telnet 用户的屏幕上。这种“回显”（echo back）用于确保由 Telnet 用户发送的字符已经被远程主机收到并在远程站点上得到处理。因此，在从用户击键到字符被显示在用户屏幕上这段时间内，每个字符在网络中传输了两次。

现在假设用户输入了一个字符‘C’，然后喝起了咖啡。我们考察一下在客户与服务器之间发送的 TCP 报文段。如图 3-31 所示，假设客户和服务器的起始序号分别是 42 和 79。前面讲过，一个报文段的序号就是该报文段数据字段首字节的序号。因此，客户发送的第一个报文段的序号为 42，服务器发送的第一个报文段的序号为 79。前面讲过，确认号就是主机正在等待的数据的下一个字节序号。在 TCP 连接建立后但没有发送任何数据之前，该客户等待字节 79，而该服务器等待字节 42。

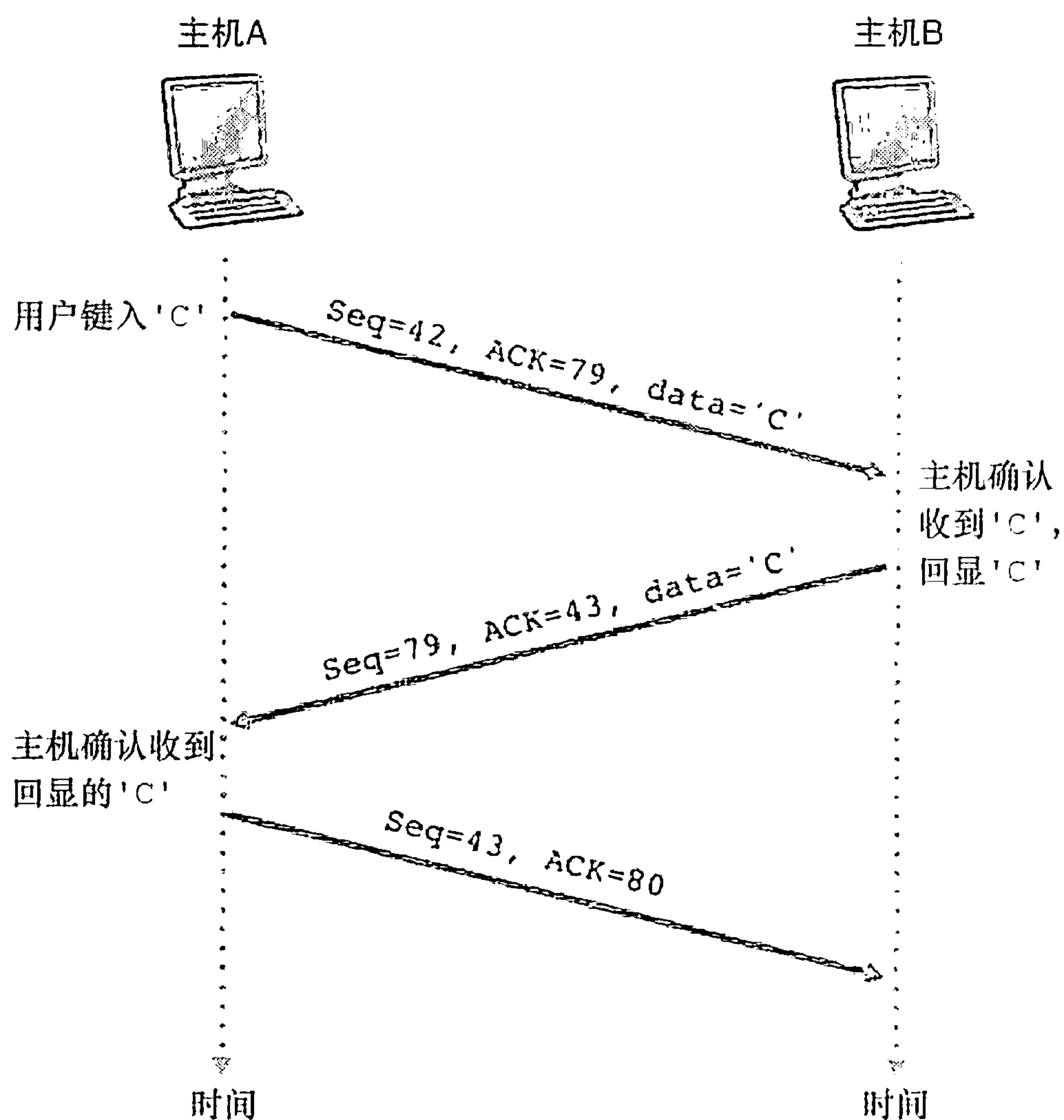


图 3-31 一个经 TCP 的简单 Telnet 应用的确认号和序号

如图 3-31 中所示，共发送 3 个报文段。第一个报文段是由客户发往服务器，在它的 数据字段里包含一字节的字符 ‘C’ 的 ASCII 码。如我们刚讲到的那样，第一个报文段的 序号字段里是 42。另外，由于客户还没有接收到来自服务器的任何数据，因此该第一个报 文段中的确认号字段中是 79。

第二个报文段是由服务器发往客户。它有两个目的：首先它是为该服务器所收到数据 提供一个确认。通过在确认号字段中填入 43，服务器告诉客户它已经成功地收到字节 42 及以前的所有字节，现在正等待着字节 43 的出现。该报文段的第二个目的是回显字符 ‘C’。因此，在第二个报文段的数据字段里填入的是字符 ‘C’ 的 ASCII 码。第二个报文 段的序号为 79，它是该 TCP 连接上从服务器到客户的数据流的起始序号，这也正是服务 器要发送的第一个字节的数据。值得注意的是，对客户到服务器的数据的确认被装载在一个 承载服务器到客户的数据的报文段中；这种确认被称为是被捎带（piggybacked）在服务 器到客户的数据报文段中的。

第三个报文段是从客户发往服务器的。它的唯一目的是确认已从服务器收到的数据。（前面讲过，第二个报文段中包含的数据是字符 ‘C’，是从服务器到客户的。）该报文段的 数据字段为空（即确认信息没有被任何从客户到服务器的数据所捎带）。该报文段的 确认号字段填入的是 80，因为客户已经收到了字节流中序号为 79 及以前的字节，它现在正 等待着字节 80 的出现。你可能认为这有点奇怪，即使该报文段里没有数据还仍有序号。 这是因为 TCP 存在序号字段，报文段需要填入某个序号。

3.5.3 往返时间的估计与超时

TCP 如同前面 3.4 节所讲的 rdt 协议一样，它采用超时/重传机制来处理报文段的丢失

问题。尽管这在概念上简单，但是当在如 TCP 这样的实际协议中实现超时/重传机制时还是会产生许多微妙的问题。也许最明显的一个问题就是超时间隔长度的设置。显然，超时间隔必须大于该连接的往返时间（RTT），即从一个报文段发出到它被确认的时间。否则会造成不必要的重传。但是这个时间间隔到底应该是多大呢？刚开始时应如何估计往返时间呢？是否应该为所有未确认的报文段各设一个定时器？问题如此之多！我们在本节中的讨论基于 [Jacobson 1988] 中有关 TCP 的工作以及 IETF 关于管理 TCP 定时器的建议 [RFC 6298]。

1. 估计往返时间

我们开始学习 TCP 定时器的管理问题，要考虑一下 TCP 是如何估计发送方与接收方之间往返时间的。这是通过如下方法完成的。报文段的样本 RTT（表示为 SampleRTT）就是从某报文段被发出（即交给 IP）到对该报文段的确认被收到之间的时间量。大多数 TCP 的实现仅在某个时刻做一次 SampleRTT 测量，而不是为每个发送的报文段测量一个 SampleRTT。这就是说，在任意时刻，仅为一个已发送的但目前尚未被确认的报文段估计 SampleRTT，从而产生一个接近每个 RTT 的新 SampleRTT 值。另外，TCP 决不为已被重传的报文段计算 SampleRTT；它仅为传输一次的报文段测量 SampleRTT [Kan 1987]。（本章后面的一个习题请你考虑一下为什么要这么做。）

显然，由于路由器的拥塞和端系统负载的变化，这些报文段的 SampleRTT 值会随之波动。由于这种波动，任何给定的 SampleRTT 值也许都是非典型的。因此，为了估计一个典型的 RTT，自然要采取某种对 SampleRTT 取平均的办法。TCP 维持一个 SampleRTT 均值（称为 EstimatedRTT）。一旦获得一个新 SampleRTT 时，TCP 就会根据下列公式来更新 EstimatedRTT：

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

上面的公式是以编程语言的语句方式给出的，即 EstimatedRTT 的新值是由以前的 EstimatedRTT 值与 SampleRTT 新值加权组合而成的。在 [RFC 6298] 中给出的 α 参考值是 $\alpha = 0.125$ （即 $1/8$ ），这时上面的公式变为：

$$\text{EstimatedRTT} = 0.875 \cdot \text{EstimatedRTT} + 0.125 \cdot \text{SampleRTT}$$

值得注意的是，EstimatedRTT 是一个 SampleRTT 值的加权平均值。如在本章后面习题中讨论的那样，这个加权平均对最近的样本赋予的权值要大于对老样本赋予的权值。这是很自然的，因为越近的样本越能更好地反映网络的当前拥塞情况。从统计学观点讲，这种平均被称为指数加权移动平均（Exponential Weighted Moving Average, EWMA）。在 EWMA 中的“指数”一词看起来是指一个给定的 SampleRTT 的权值在更新的过程中呈指数型快速衰减。在课后习题中，将要求你推导出 EstimatedRTT 的指数表达形式。

图 3-32 显示了当 $\alpha = 1/8$ 时，在 gaia.cs.umass.edu（在美国马萨诸塞州的 Amherst）与 fantasia.eurecom.fr（在法国南部）之间的一条 TCP 连接上的 SampleRTT 值与 EstimatedRTT 值。显然，SampleRTT 的变化在计算 EstimatedRTT 的过程中趋于平缓。

除了估算 RTT 外，测量 RTT 的变化也是有价值的。[RFC 6298] 定义了 RTT 偏差 DevRTT，用于估算 SampleRTT 一般会偏离 EstimatedRTT 的程度：

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

注意到 DevRTT 是一个 SampleRTT 与 EstimatedRTT 之间差值的 EWMA。如果 SampleRTT

值波动较小,那么 DevRTT 的值就会很小;另一方面,如果波动很大,那么 DevRTT 的值就会很大。 β 的推荐值为 0.25。

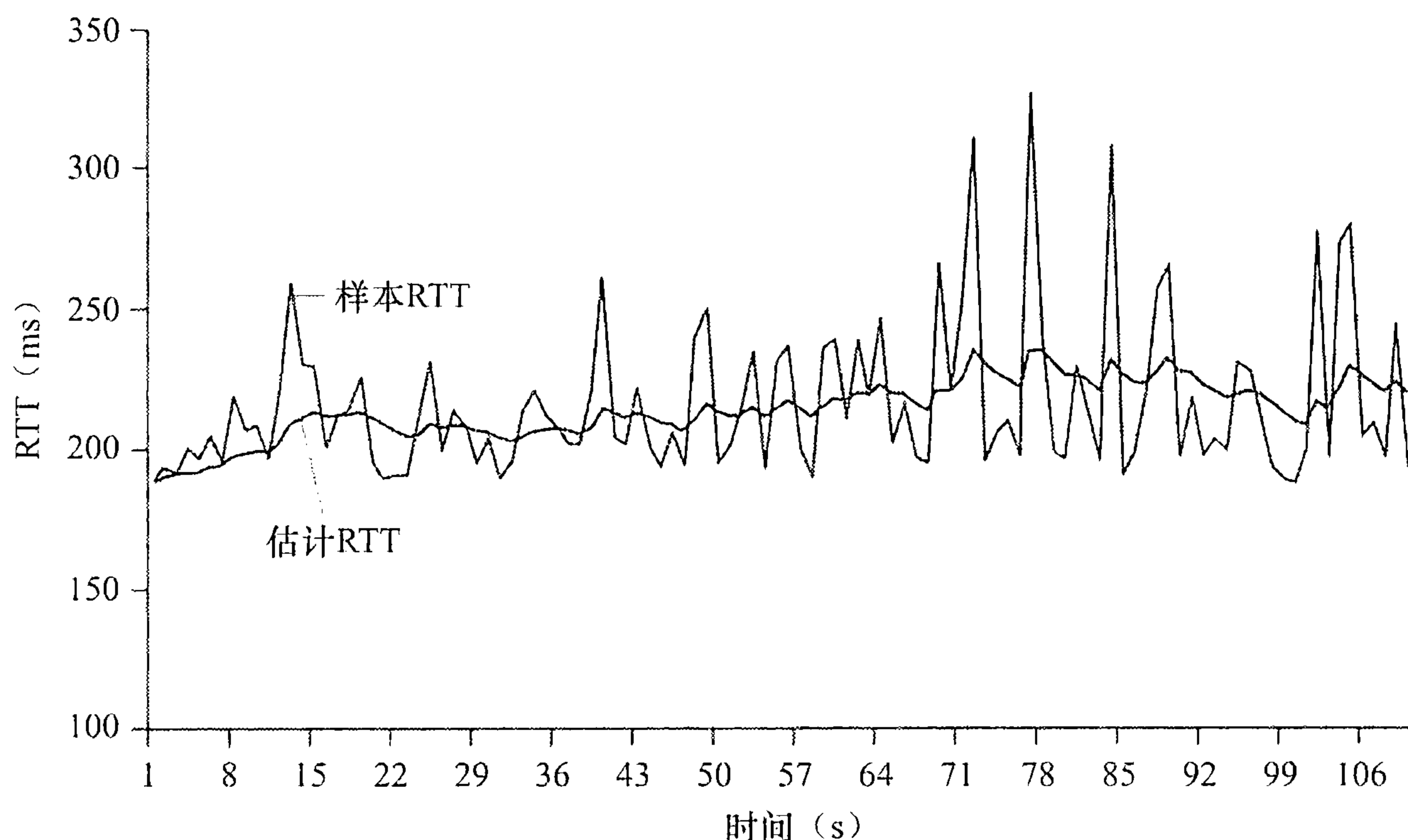


图 3-32 RTT 样本和 RTT 估计

2. 设置和管理重传超时间隔

假设已经给出了 EstimatedRTT 值和 DevRTT 值,那么 TCP 超时间隔应该用什么值呢?很明显,超时间隔应该大于等于 EstimatedRTT,否则,将造成不必要的重传。但是超时间隔也不应该比 EstimatedRTT 大太多,否则当报文段丢失时,TCP 不能很快地重传该报文段,导致数据传输时延大。因此要求将超时间隔设为 EstimatedRTT 加上一定余量。当 SampleRTT 值波动较大时,这个余量应该大些;当波动较小时,这个余量应该小些。因此,DevRTT 值应该在这里发挥作用了。在 TCP 的确定重传超时间隔的方法中,所有这些因素都考虑到了:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

推荐的初始 TimeoutInterval 值为 1 秒 [RFC 6298]。同样,当出现超时时,TimeoutInterval 值将加倍,以免即将被确认的后继报文段过早出现超时。不管怎样,一旦报文段收到并更新 EstimatedRTT 后,TimeoutInterval 就又使用上述公式计算了。

实践原则

与我们在 3.4 节中所学的方法很像,TCP 通过使用肯定确认与定时器来提供可靠数据传输。TCP 确认正确接收到的数据,而当认为报文段或其确认报文丢失或受损时,TCP 会重传这些报文段。有些版本的 TCP 还有一个隐式 NAK 机制(在 TCP 的快速重传机制下,收到对一个特定报文段的 3 个冗余 ACK 就可作为对后面报文段的一个隐式 NAK,从而在超时之前触发对该报文段的重传。TCP 使用一系列编号以使接收方能识别丢失或重复的报文段。像可靠数据传输协议 rdt3.0 的情况一样,TCP 自己也无法确认一个报文段或其 ACK 是丢失了还是受损了,或是时延过长了。在发送方,TCP 的响应是相同的:重传有问题的报文段。

TCP 也使用流水线，使得发送方在任意时刻都可以有多个已发出但还未被确认的报文段存在。我们在前面已经看到，当报文段长度与往返时延之比很小时，流水线可显著地增加一个会话的吞吐量。一个发送方可以具有的未被确认报文段的确切个数是由 TCP 的流量控制和拥塞控制机制决定的。TCP 流量控制将在本节后面讨论；TCP 拥塞控制将在 3.7 节中讨论。我们暂时只需知道 TCP 发送方使用了流水线即可。

3.5.4 可靠数据传输

前面讲过，因特网的网络层服务（IP 服务）是不可靠的。IP 不保证数据报的交付，不保证数据报的按序交付，也不保证数据报中数据的完整性。对于 IP 服务，数据报能够溢出路由器缓存而永远不能到达目的地，数据报也可能是乱序到达，而且数据报中的比特可能损坏（由 0 变为 1 或者相反）。由于运输层报文段是被 IP 数据报携带着在网络中传输的，所以运输层的报文段也会遇到这些问题。

TCP 在 IP 不可靠的尽力而为服务之上创建了一种可靠数据传输服务（reliable data transfer service）。TCP 的可靠数据传输服务确保一个进程从其接收缓存中读出的数据流是无损坏、无间隔、非冗余和按序的数据流；即该字节流与连接的另一方端系统发送出的字节流是完全相同。TCP 提供可靠数据传输的方法涉及我们在 3.4 节中所学的许多原理。

在我们前面研发可靠数据传输技术时，曾假定每一个已发送但未被确认的报文段都与一个定时器相关联，这在概念上是最简单的。虽然这在理论上很好，但定时器的管理却需要相当大的开销。因此，推荐的定时器管理过程 [RFC 6298] 仅使用单一的重传定时器，即使有多个已发送但还未被确认的报文段。在本节中描述的 TCP 协议遵循了这种单一定时器的推荐。

我们将以两个递增的步骤来讨论 TCP 是如何提供可靠数据传输的。我们先给出一个 TCP 发送方的高度简化的描述，该发送方只用超时来恢复报文段的丢失；然后再给出一个更全面的描述，该描述中除了使用超时机制外，还使用冗余确认技术。在接下来的讨论中，我们假定数据仅向一个方向发送，即从主机 A 到主机 B，且主机 A 在发送一个大文件。

图 3-33 给出了一个 TCP 发送方高度简化的描述。我们看到在 TCP 发送方有 3 个与发送和重传有关的主要事件：从上层应用程序接收数据；定时器超时和收到 ACK。一旦第一个主要事件发生，TCP 从应用程序接收数据，将数据封装在一个报文段中，并把该报文段交给 IP。注意到每一个报文段都包含一个序号，如 3.5.2 节所讲的那样，这个序号就是该报文段第一个数据字节的字节流编号。还要注意到如果定时器还没有为某些其他报文段而运行，则当报文段被传给 IP 时，TCP 就启动该定时器。（将定时器想象为与最早的未被确认的报文段相关联是有帮助的。）该定时器的过期间隔是 TimeoutInterval，它是由 3.5.3 节中所描述的 EstimatedRTT 和 DevRTT 计算得出的。

第二个主要事件是超时。TCP 通过重传引起超时的报文段来响应超时事件。然后 TCP 重启定时器。

TCP 发送方必须处理的第三个主要事件是一个来自接收方的确认报文段（ACK）的到达（更确切地说是一个包含了有效 ACK 字段值的报文段）。当该事件发生时，TCP 将 ACK 的值 y 与它的变量 SendBase 进行比较。TCP 状态变量 SendBase 是最早未被确认的字节的

序号。(因此 $\text{SendBase} - 1$ 是指接收方已正确按序接收到的数据的最后一个字节的序号。) 如前面指出的那样, TCP 采用累积确认, 所以 y 确认了字节编号在 y 之前的所有字节都已经收到。如果 $y > \text{SendBase}$, 则该 ACK 是在确认一个或多个先前未被确认的报文段。因此发送方更新它的 SendBase 变量; 如果当前有未被确认的报文段, TCP 还要重新启动定时器。

```

/* 假设发送方不受TCP流量和拥塞控制的限制, 来自上层的数据长度小于MSS, 且数据传送只在一个
   方向进行。*/

NextSeqNum=InitialSeqNumber
SendBase=InitialSeqNumber

loop (forever) {
    switch(event)

        event: data received from application above
            create TCP segment with sequence number NextSeqNum
            if (timer currently not running)
                start timer
            pass segment to IP
            NextSeqNum=NextSeqNum+length(data)
            break;

        event: timer timeout
            retransmit not-yet-acknowledged segment with
                smallest sequence number
            start timer
            break;

        event: ACK received, with ACK field value of y
            if (y > SendBase) {
                SendBase=y
                if (there are currently any not-yet-acknowledged segments)
                    start timer
            }
            break;

    } /* forever 循环结束 */

```

图 3-33 简化的 TCP 发送方

1. 一些有趣的情况

我们刚刚描述了一个关于 TCP 如何提供可靠数据传输的高度简化的版本。但即使这种高度简化的版本, 仍然存在着许多微妙之处。为了较好地感受该协议的工作过程, 我们来看几种简单情况。图 3-34 描述了第一种情况, 主机 A 向主机 B 发送一个报文段。假设该报文段的序号是 92, 而且包含 8 字节数据。在发出该报文段之后, 主机 A 等待一个来自主机 B 的确认号为 100 的报文段。虽然 A 发出的报文段在主机 B 上被收到, 但从主机 B 发往主机 A 的确认报文丢失了。在这种情况下, 超时事件就会发生, 主机 A 会重传相同的报文段。当然, 当主机 B 收到该重传的报文段时, 它将通过序号发现该报文段包含了早已收到的数据。因此, 主机 B 中的 TCP 将丢弃该重传的报文段中的这些字节。

在第二种情况中, 如图 3-35 所示, 主机 A 连续发回了两个报文段。第一个报文段序号是 92, 包含 8 字节数据; 第二个报文段序号是 100, 包含 20 字节数据。假设两个报文段都完好无损地到达主机 B, 并且主机 B 为每一个报文段分别发送一个确认。第一个确认报

文的确认号是 100，第二个确认报文的确认号是 120。现在假设在超时之前这两个报文段中没有一个确认报文到达主机 A。当超时事件发生时，主机 A 重传序号 92 的第一个报文段，并重启定时器。只要第二个报文段的 ACK 在新的超时发生以前到达，则第二个报文段将不会被重传。

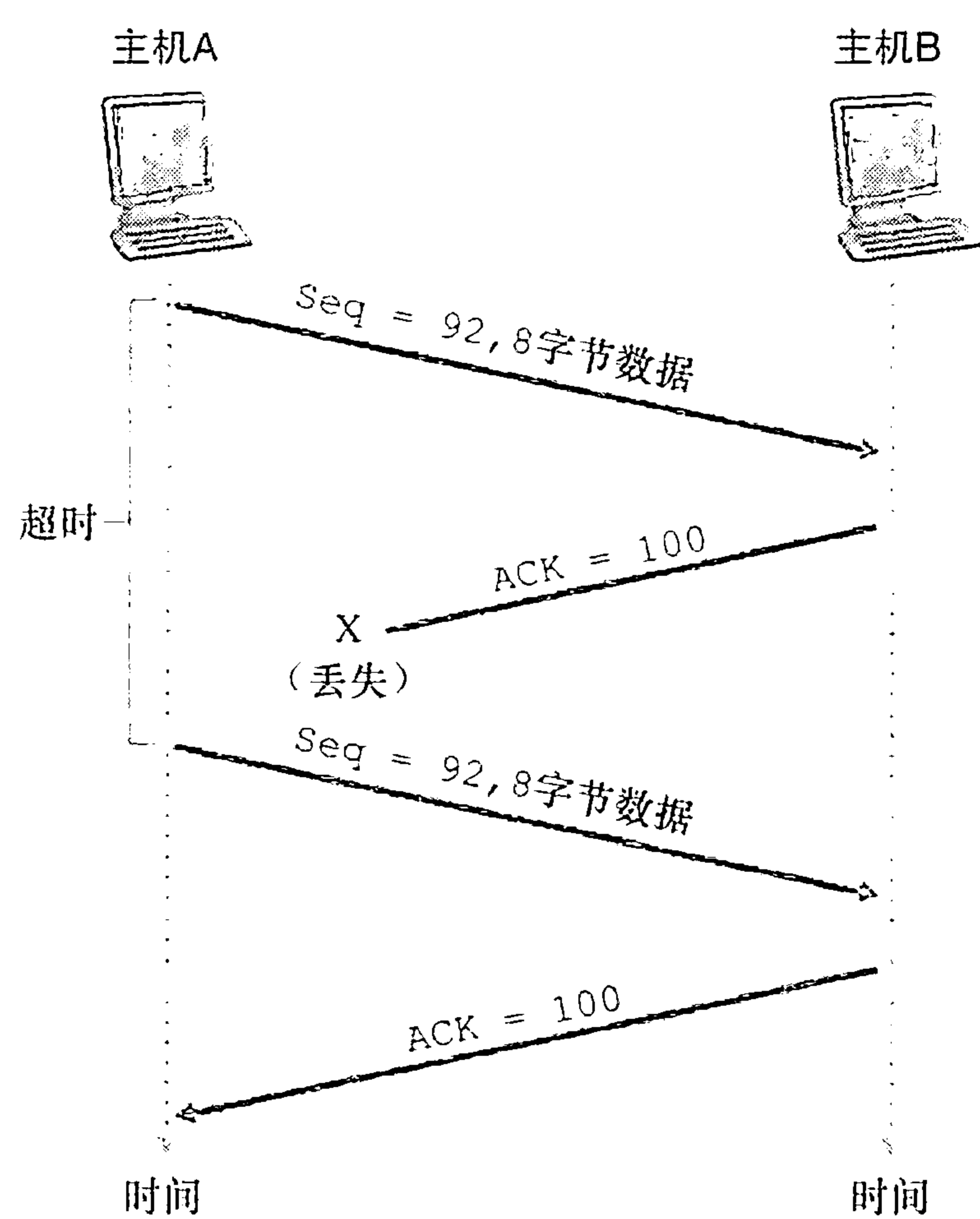


图 3-34 由于确认丢失而重传

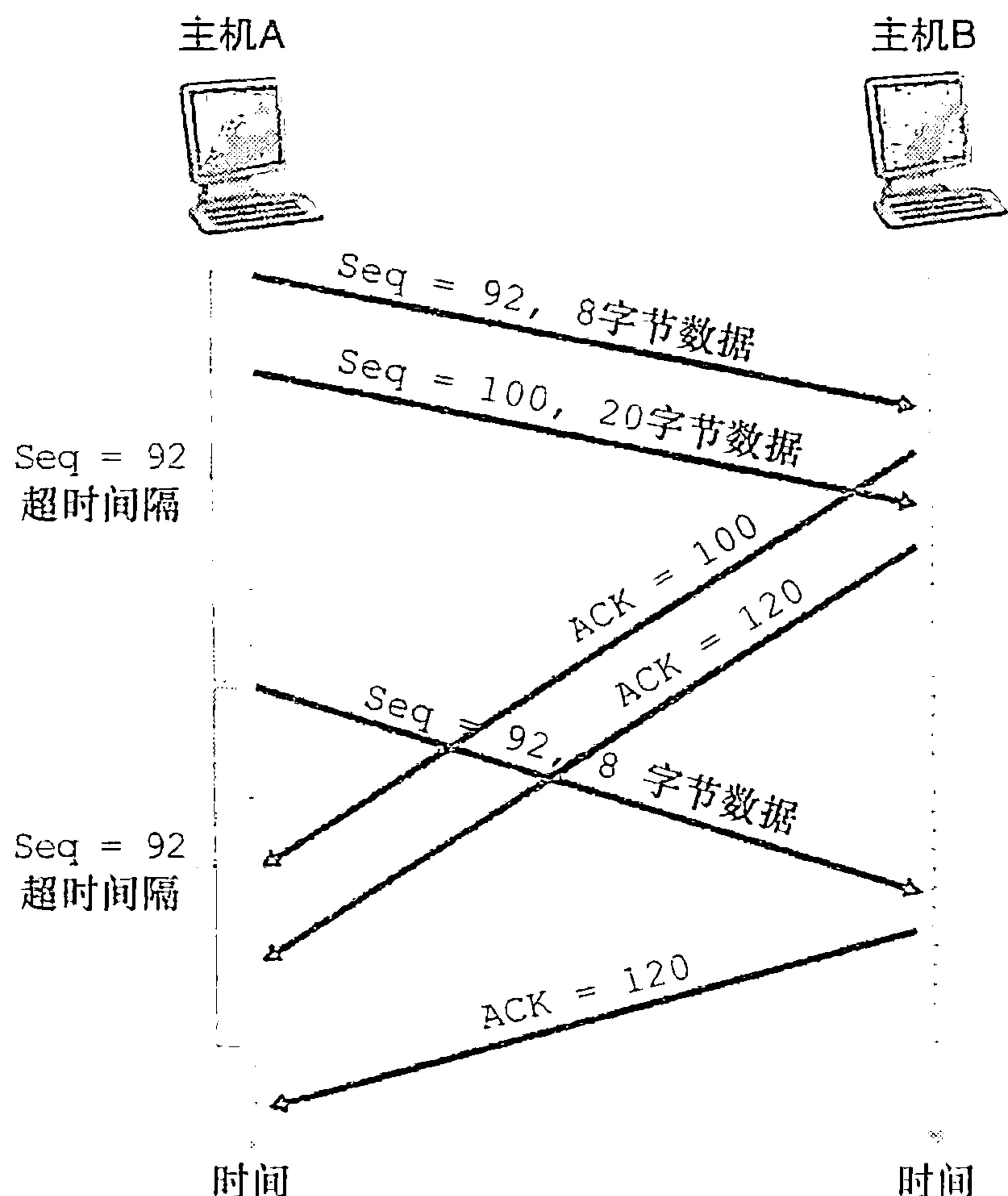


图 3-35 报文段 100 没有重传

在第三种也是最后一种情况中，假设主机 A 与在第二种情况中完全一样，发送两个报文段。第一个报文段的确认报文在网络丢失，但在超时事件发生之前主机 A 收到一个确认号为 120 的确认报文。主机 A 因而知道主机 B 已经收到了序号为 119 及之前的所有字节；所以主机 A 不会重传这两个报文段中的任何一个。这种情况在图 3-36 中进行了图示。

2. 超时间隔加倍

我们现在讨论一下在大多数 TCP 实现中所做的一些修改。首先关注的是在定时器时限过期后超时间隔的长度。在这种修改中，每当超时事件发生时，如前所述，TCP 重传具有最小序号的还未被确认的报文段。只是每次 TCP 重传时都会将下一次的超时间隔设为先前值的两倍，而不是用从 EstimatedRTT 和 DevRTT 推算出的值（如在 3.5.3 节中所描述的）。例如，假设当定时器第一次过期时，与最早的未被确认的报文段相关联的 TimeoutInterval 是 0.75 秒。TCP 就会重传该

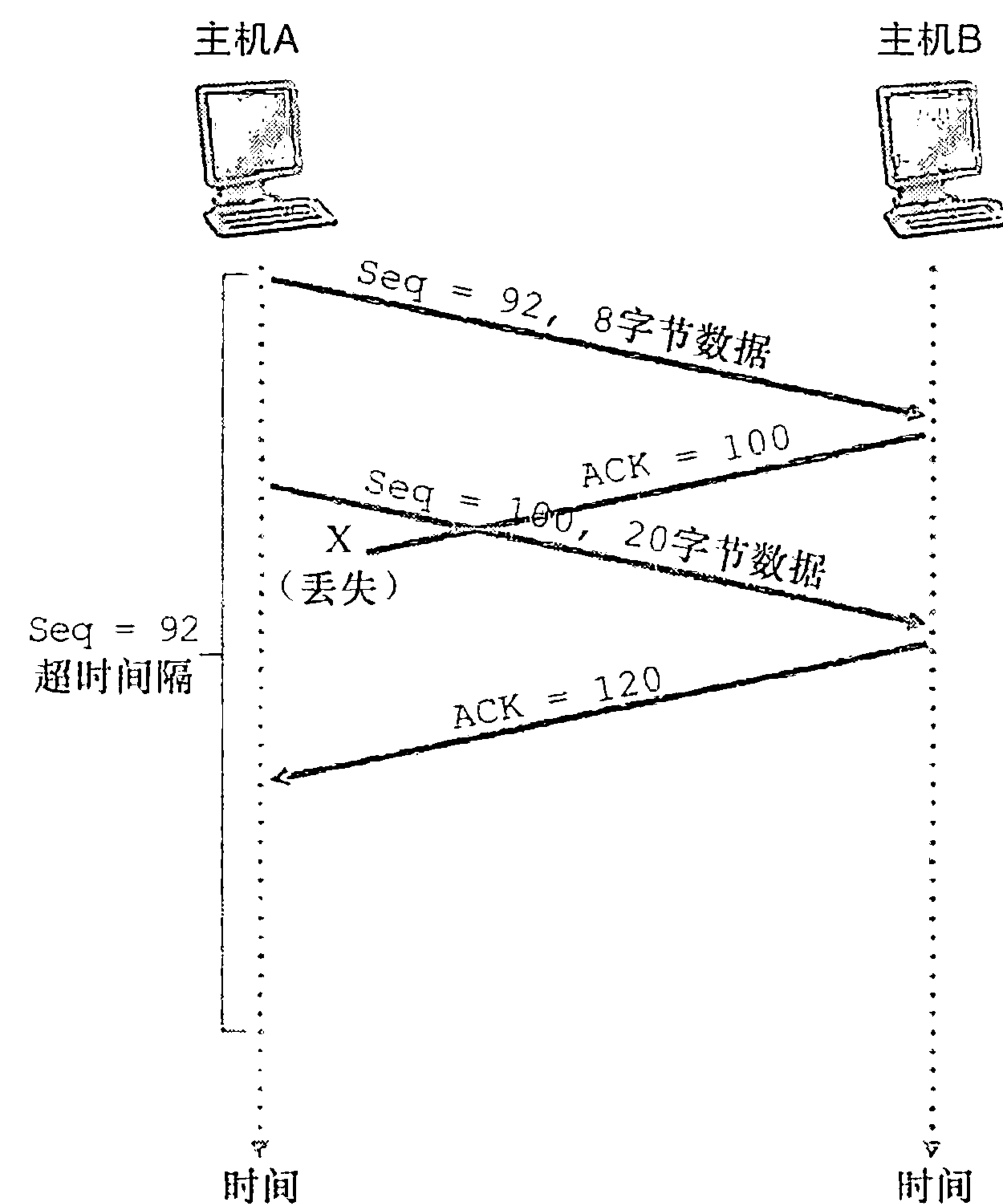


图 3-36 累积确认避免了第一个报文段的重传

报文段，并把新的过期时间设置为 1.5 秒。如果 1.5 秒后定时器又过期了，则 TCP 将再次重传该报文段，并把过期时间设置为 3.0 秒。因此，超时间隔在每次重传后会呈指数型增长。然而，每当定时器在另两个事件（即收到上层应用的数据和收到 ACK）中的任意一个启动时，TimeoutInterval 由最近的 EstimatedRTT 值与 DevRTT 值推算得到。

这种修改提供了一个形式受限的拥塞控制。（更复杂的 TCP 拥塞控制形式将在 3.7 节中学习。）定时器过期很可能是由网络拥塞引起的，即太多的分组到达源与目的地之间路径上的一台（或多台）路由器的队列中，造成分组丢失或长时间的排队时延。在拥塞的时候，如果源持续重传分组，会使拥塞更加严重。相反，TCP 使用更文雅的方式，每个发送方的重传都是经过越来越长的时间间隔后进行的。当我们在第 5 章学习 CSMA/CD 时，将看到以太网采用了类似的思路。

3. 快速重传

超时触发重传存在的问题之一是超时周期可能相对较长。当一个报文段丢失时，这种长超时周期迫使发送方延迟重传丢失的分组，因而增加了端到端时延。幸运的是，发送方通常可在超时事件发生之前通过注意所谓冗余 ACK 来较好地检测到丢包情况。冗余 ACK（duplicate ACK）就是再次确认某个报文段的 ACK，而发送方先前已经收到对该报文段的确认。要理解发送方对冗余 ACK 的响应，我们必须首先看一下接收方为什么会发送冗余 ACK。表 3-2 总结了 TCP 接收方的 ACK 生成策略 [RFC 5681]。当 TCP 接收方收到一个具有这样序号的报文段时，即其序号大于下一个所期望的、按序的报文段，它检测到了数据流中的一个间隔，这就是说有报文段丢失。这个间隔可能是由于在网络中报文段丢失或重新排序造成的。因为 TCP 不使用否定确认，所以接收方不能向发送方发回一个显式的否定确认。相反，它只是对已经接收到的最后一个按序字节数据进行重复确认（即产生一个冗余 ACK）即可。（注意到在表 3-2 中允许接收方不丢弃失序报文段。）

表 3-2 产生 TCP ACK 的建议 [RFC 5681]

事件	TCP 接收方动作
具有所期望序号的按序报文段到达。所有在期望序号及以前的数据都已经被确认	延迟的 ACK。对另一个按序报文段的到达最多等待 500ms。如果下一个按序报文段在这个时间间隔内没有到达，则发送一个 ACK
具有所期望序号的按序报文段到达。另一个按序报文段等待 ACK 传输	立即发送单个累积 ACK，以确认两个按序报文段
比期望序号大的失序报文段到达。检测出间隔	立即发送冗余 ACK，指示下一个期待字节的序号（其为间隔的低端的序号）
能部分或完全填充接收数据间隔的报文段到达	倘若该报文段起始于间隔的低端，则立即发送 ACK

因为发送方经常一个接一个地发送大量的报文段，如果一个报文段丢失，就很可能引起许多一个接一个的冗余 ACK。如果 TCP 发送方接收到对相同数据的 3 个冗余 ACK，它把这当作一种指示，说明跟在这个已被确认过 3 次的报文段之后的报文段已经丢失。（在课后习题中，我们将考虑为什么发送方等待 3 个冗余 ACK，而不是仅仅等待一个冗余 ACK。）一旦收到 3 个冗余 ACK，TCP 就执行快速重传（fast retransmit）[RFC 5681]，即在该报文段的定时器过期之前重传丢失的报文段。对于采用快速重传的 TCP，可用下列代码片段代替图 3-33 中的 ACK 收到事件：

```

event: ACK received, with ACK field value of y
    if (y > SendBase) {
        SendBase=y
        if (there are currently any not yet
            acknowledged segments)
            start timer
    }
else { /* a duplicate ACK for already ACKed
        segment */
        increment number of duplicate ACKs
        received for y
        if (number of duplicate ACKS received
            for y==3)
            /* TCP fast retransmit */
            resend segment with sequence number y
    }
break;

```

前面讲过，当在如 TCP 这样一个实际协议中实现超时/重传机制时，会产生许多微妙的问题。上面的过程是在超过 20 年的 TCP 定时器使用经验的基础上演化而来的，读者应当理解实际情况确实是这样的。

4. 是回退 N 步还是选择重传

考虑下面这个问题来结束有关 TCP 差错恢复机制的学习：TCP 是一个 GBN 协议还是一个 SR 协议？前面讲过，TCP 确认是累积式的，正确接收但失序的报文段是会被接收方逐个确认的。因此，如图 3-33 所示（也可参见图 3-19），TCP 发送方仅需维持已发送过但未被确认的字节的最小序号（SendBase）和下一个要发送的字节的序号（NextSeqNum）。在这种意义下，TCP 看起来更像一个 GBN 风格的协议。但是 TCP 和 GBN 协议之间有着一些显著的区别。许多 TCP 实现会将正确接收但失序的报文段缓存起来 [Stevens 1994]。另外考虑一下，当发

送方发送的一组报文段 1, 2, ..., N ，并且所有的报文段都按序无差错地到达接收方时会发生的情况。进一步假设对分组 $n < N$ 的确认报文丢失，但是其余 $N-1$ 个确认报文在分别超时以前到达发送端，这时又会发生的情况。在该例中，GBN 不仅会重传分组 n ，还会重传所有后继的分组 $n+1, n+2, \dots, N$ 。在另一方面，TCP 将重传至多一个报文段，即报文段 n 。此外，如果对报文段 $n+1$ 的确认报文在报文段 n 超时之前到达，TCP 甚至不会重传报文段 n 。

对 TCP 提出的一种修改意见是所谓的**选择确认**（selective acknowledgment）[RFC 2018]，它允许 TCP 接收方有选择地确认失序报文段，而不是累积地确认最后一个正确接收的有序

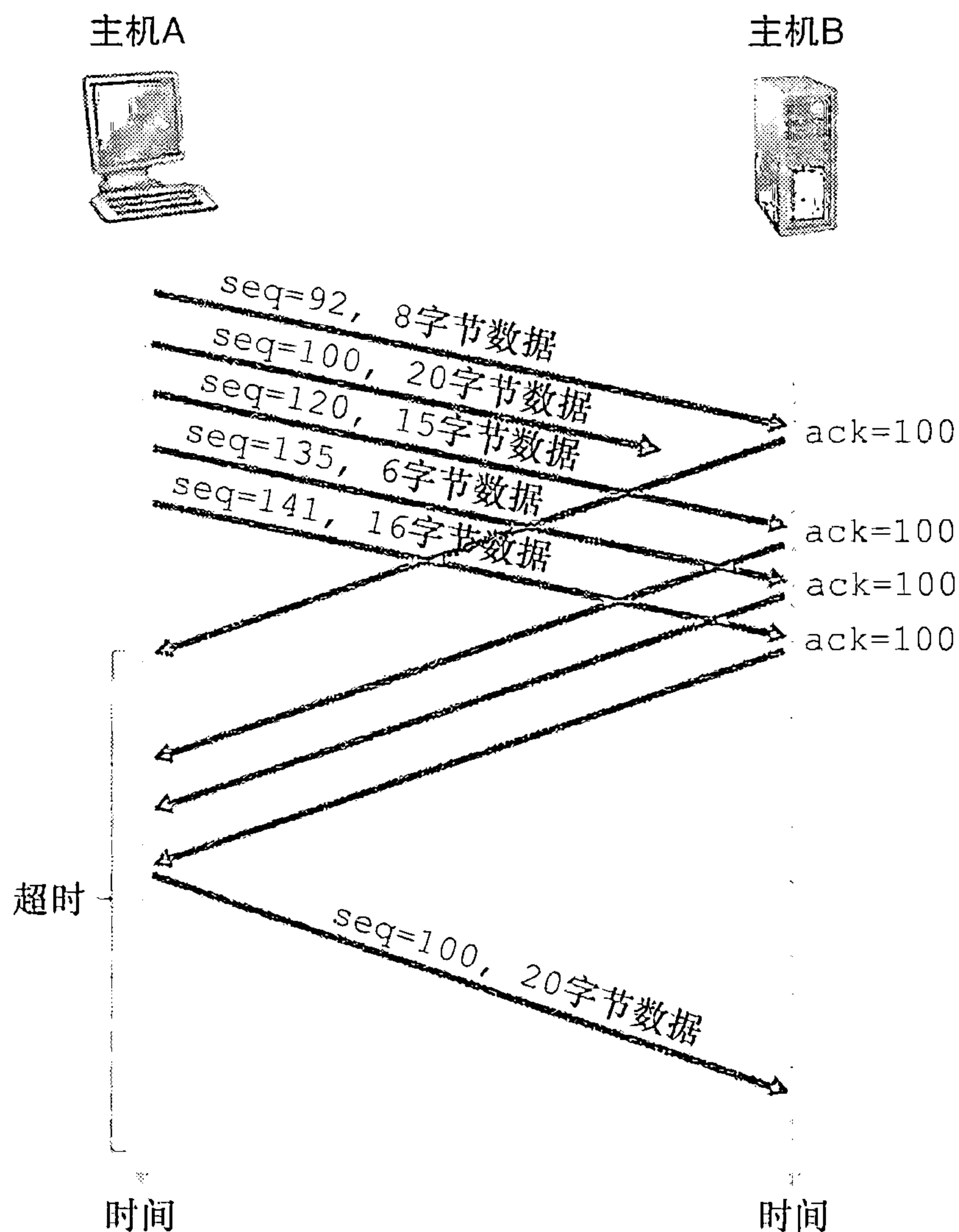


图 3-37 快速重传：在某报文段的定时器过期之前重传丢失的报文段

报文段。当将该机制与选择重传机制结合起来使用时（即跳过重传那些已被接收方选择性地确认过的报文段），TCP 看起来就很像我们通常的 SR 协议。因此，TCP 的差错恢复机制也许最好被分类为 GBN 协议与 SR 协议的混合体。

3.5.5 流量控制

前面讲过，一条 TCP 连接每一侧主机都为该连接设置了接收缓存。当该 TCP 连接收到正确、按序的字节后，它就将数据放入接收缓存。相关联的应用进程会从该缓存中读取数据，但不必是数据刚一到达就立即读取。事实上，接收方应用也许正忙于其他任务，甚至要过很长时间后才去读取该数据。如果某应用程序读取数据时相对缓慢，而发送方发送得太多、太快，发送的数据就会很容易地使该连接的接收缓存溢出。

TCP 为它的应用程序提供了**流量控制服务**（flow-control service）以消除发送方使接收方缓存溢出的可能性。流量控制因此是一个速度匹配服务，即发送方的发送速率与接收方应用程序的读取速率相匹配。前面提到过，TCP 发送方也可能因为 IP 网络的拥塞而被遏制；这种形式的发送方的控制被称为**拥塞控制**（congestion control），我们将在 3.6 节和 3.7 节详细地讨论这个主题。即使流量控制和拥塞控制采取的动作非常相似（对发送方的遏制），但是它们显然是针对完全不同的原因而采取的措施。不幸的是，许多作者把这两个术语混用，理解力强的读者会明智地区分这两种情况。现在我们来讨论 TCP 如何提供流量控制服务的。为了能从整体上看问题，我们在本节都假设 TCP 是这样实现的，即 TCP 接收方丢弃失序的报文段。

TCP 通过让发送方维护一个称为**接收窗口**（receive window）的变量来提供流量控制。通俗地说，接收窗口用于给发送方一个指示——该接收方还有多少可用的缓存空间。因为 TCP 是全双工通信，在连接两端的发送方都各自维护一个接收窗口。我们在文件传输的情况下研究接收窗口。假设主机 A 通过一条 TCP 连接向主机 B 发送一个大文件。主机 B 为该连接分配了一个接收缓存，并用 RcvBuffer 来表示其大小。主机 B 上的应用进程不时地从该缓存中读取数据。我们定义以下变量：

- LastByteRead：主机 B 上的应用进程从缓存读出的数据流的最后一个字节的编号。
- LastByteRcvd：从网络中到达的并且已放入主机 B 接收缓存中的数据流的最后一个字节的编号。

由于 TCP 不允许已分配的缓存溢出，下式必须成立：

$$\text{LastByteRcvd} - \text{LastByteRead} \leq \text{RcvBuffer}$$

接收窗口用 rwnd 表示，根据缓存可用空间的数量来设置：

$$\text{rwnd} = \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$$

由于该空间是随着时间变化的，所以 rwnd 是动态的。图 3-38 对变量 rwnd 进行了图示。

连接是如何使用变量 rwnd 提供流量控制服务的呢？主机 B 通过把当前的 rwnd 值放入它发给主机 A 的报文段接收窗口字段中，通知主机 A 它在该连接的缓存中还有多少可用空间。开始时，主机 B 设定 $\text{rwnd} = \text{RcvBuffer}$ 。注意到为了实现这

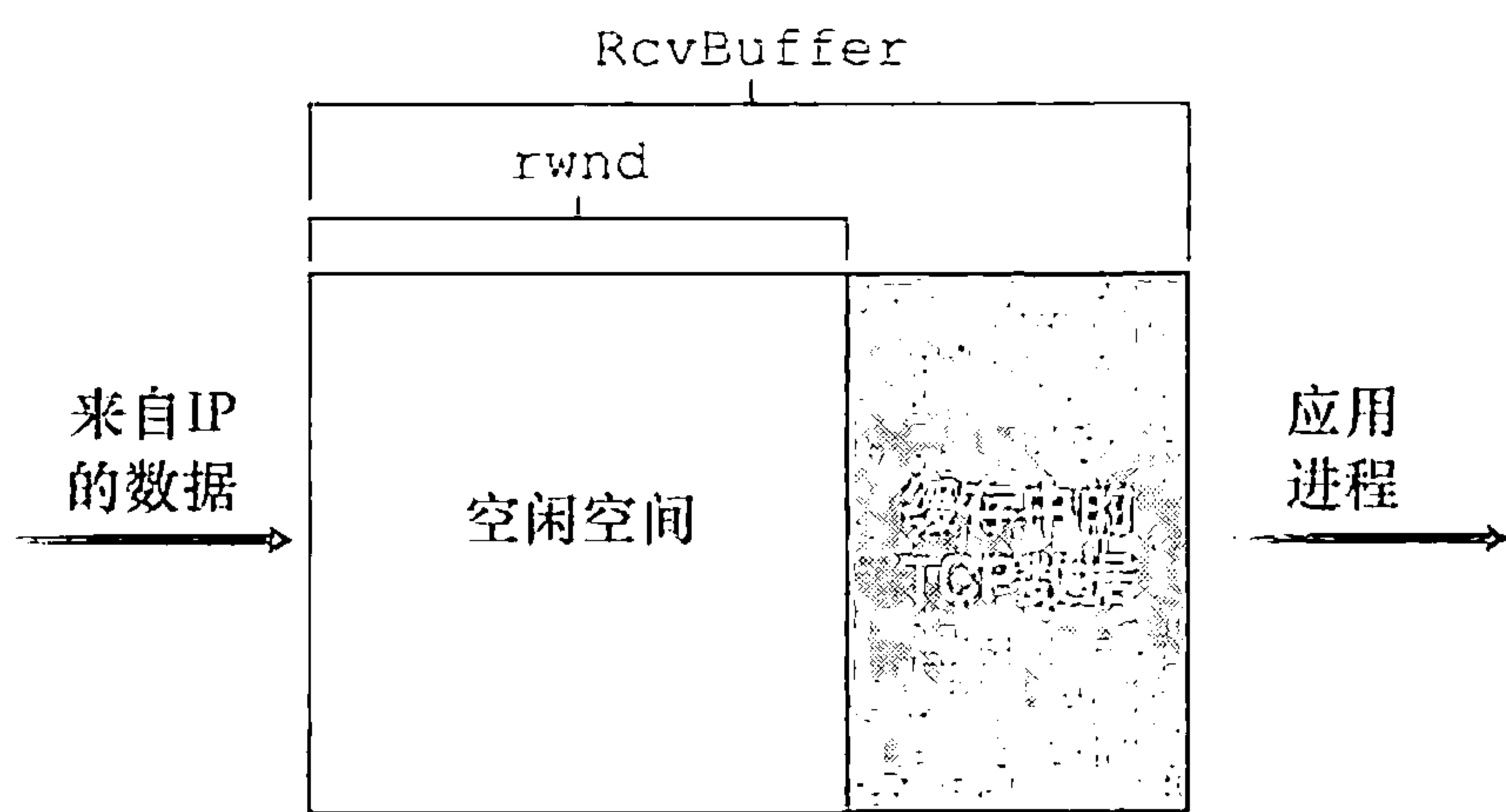


图 3-38 接收窗口（rwnd）和接收缓存（RcvBuffer）

一点，主机 B 必须跟踪几个与连接有关的变量。

主机 A 轮流跟踪两个变量，LastByteSent 和 LastByteAcked，这两个变量的意义很明显。注意到这两个变量之间的差 LastByteSent - LastByteAcked，就是主机 A 发送到连接中但未被确认的数据量。通过将未确认的数据量控制在值 rwnd 以内，就可以保证主机 A 不会使主机 B 的接收缓存溢出。因此，主机 A 在该连接的整个生命周期须保证：

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

对于这个方案还存在一个小小的技术问题。为了理解这一点，假设主机 B 的接收缓存已经存满，使得 rwnd = 0。在将 rwnd = 0 通告给主机 A 之后，还要假设主机 B 没有任何数据要发给主机 A。此时，考虑会发生什么情况。因为主机 B 上的应用进程将缓存清空，TCP 并不向主机 A 发送带有 rwnd 新值的新报文段；事实上，TCP 仅当在它有数据或有确认要发时才会发送报文段给主机 A。这样，主机 A 不可能知道主机 B 的接收缓存已经有新的空间了，即主机 A 被阻塞而不能再发送数据！为了解决这个问题，TCP 规范中要求：当主机 B 的接收窗口为 0 时，主机 A 继续发送只有一个字节数据的报文段。这些报文段将会被接收方确认。最终缓存将开始清空，并且确认报文里将包含一个非 0 的 rwnd 值。

位于 <http://www.awl.com/kurose-ross> 的在线站点为本书提供了一个交互式 Java 小程序，用以说明 TCP 接收窗口的运行情况。

描述了 TCP 的流量控制服务以后，我们在此要简要地提一下 UDP 并不提供流量控制。为了理解这个问题，考虑一下从主机 A 上的一个进程向主机 B 上的一个进程发送一系列 UDP 报文段的情形。对于一个典型的 UDP 实现，UDP 将会把这些报文段添加到相应套接字（进程的门户）“前面”的一个有限大小的缓存中。进程每次从缓存中读取一个完整的报文段。如果进程从缓存中读取报文段的速度不够快，那么缓存将会溢出，并且将丢失报文段。

3.5.6 TCP 连接管理

在本小节中，我们更为仔细地观察如何建立和拆除一条 TCP 连接。尽管这个主题并不特别令人兴奋，但是它很重要，因为 TCP 连接的建立会显著地增加人们感受到的时延（如在 Web 上冲浪时）。此外，许多常见的网络攻击（包括极为流行的 SYN 洪泛攻击）利用了 TCP 连接管理中的弱点。现在我们观察一下一条 TCP 连接是如何建立的。假设运行在一台主机（客户）上的一个进程想与另一台主机（服务器）上的一个进程建立一条连接。客户应用进程首先通知客户 TCP，它想建立一个与服务器上某个进程之间的连接。客户中的 TCP 会用以下方式与服务器中的 TCP 建立一条 TCP 连接：

- 第一步：客户端的 TCP 首先向服务器端的 TCP 发送一个特殊的 TCP 报文段。该报文段中不包含应用层数据。但是在报文段的首部（参见图 3-29）中的一个标志位（即 SYN 比特）被置为 1。因此，这个特殊报文段被称为 SYN 报文段。另外，客户会随机地选择一个初始序号（client_isn），并将此编号放置于该起始的 TCP SYN 报文段的序号字段中。该报文段会被封装在一个 IP 数据报中，并发送给服务器。为了避免某些安全性攻击，在适当地随机化选择 client_isn 方面有着不少有趣的研究 [CERT 2001-09]。

- 第二步：一旦包含 TCP SYN 报文段的 IP 数据报到达服务器主机（假定它的确到达了！），服务器会从该数据报中提取出 TCP SYN 报文段，为该 TCP 连接分配 TCP 缓存和变量，并向该客户 TCP 发送允许连接的报文段。（我们将在第 8 章看到，在完成三次握手的第三步之前分配这些缓存和变量，使得 TCP 易于受到称为 SYN 洪泛的拒绝服务攻击。）这个允许连接的报文段也不包含应用层数据。但是，在报文段的首部却包含 3 个重要的信息。首先，SYN 比特被置为 1。其次，该 TCP 报文段首部的确认号字段被置为 $\text{client_isn} + 1$ 。最后，服务器选择自己的初始序号 (server_isn)，并将其放置到 TCP 报文段首部的序号字段中。这个允许连接的报文段实际上表明了：“我收到了你发起建立连接的 SYN 分组，该分组带有初始序号 client_isn 。我同意建立该连接。我自己的初始序号是 server_isn 。”该允许连接的报文段有时被称为 SYNACK 报文段 (SYNACK segment)。
- 第三步：在收到 SYNACK 报文段后，客户也要给该连接分配缓存和变量。客户主机则向服务器发送另外一个报文段；这最后一个报文段对服务器的允许连接的报文段进行了确认（该客户通过将值 $\text{server_isn} + 1$ 放置到 TCP 报文段首部的确认字段中来完成此项工作）。因为连接已经建立了，所以该 SYN 比特被置为 0。该三次握手的第三个阶段可以在报文段负载中携带客户到服务器的数据。

一旦完成这 3 个步骤，客户和服务器主机就可以相互发送包括数据的报文段了。在以后每一个报文段中，SYN 比特都将被置为 0。注意到为了创建该连接，在两台主机之间发送了 3 个分组，如图 3-39 所示。由于这个原因，这种连接创建过程通常被称为 3 次握手 (three-way handshake)。TCP 3 次握手的几个方面将在课后习题中讨论（为什么需要初始序号？为什么需要 3 次握手，而不是两次握手？）。注意到这样一件事是很有趣的，一个攀岩者和一个保护者（他位于攀岩者的下面，他的任务是处理好攀岩者的安全绳索）就使用了与 TCP 相同的 3 次握手通信协议，以确保在攀岩者开始攀爬前双方都已经准备好了。

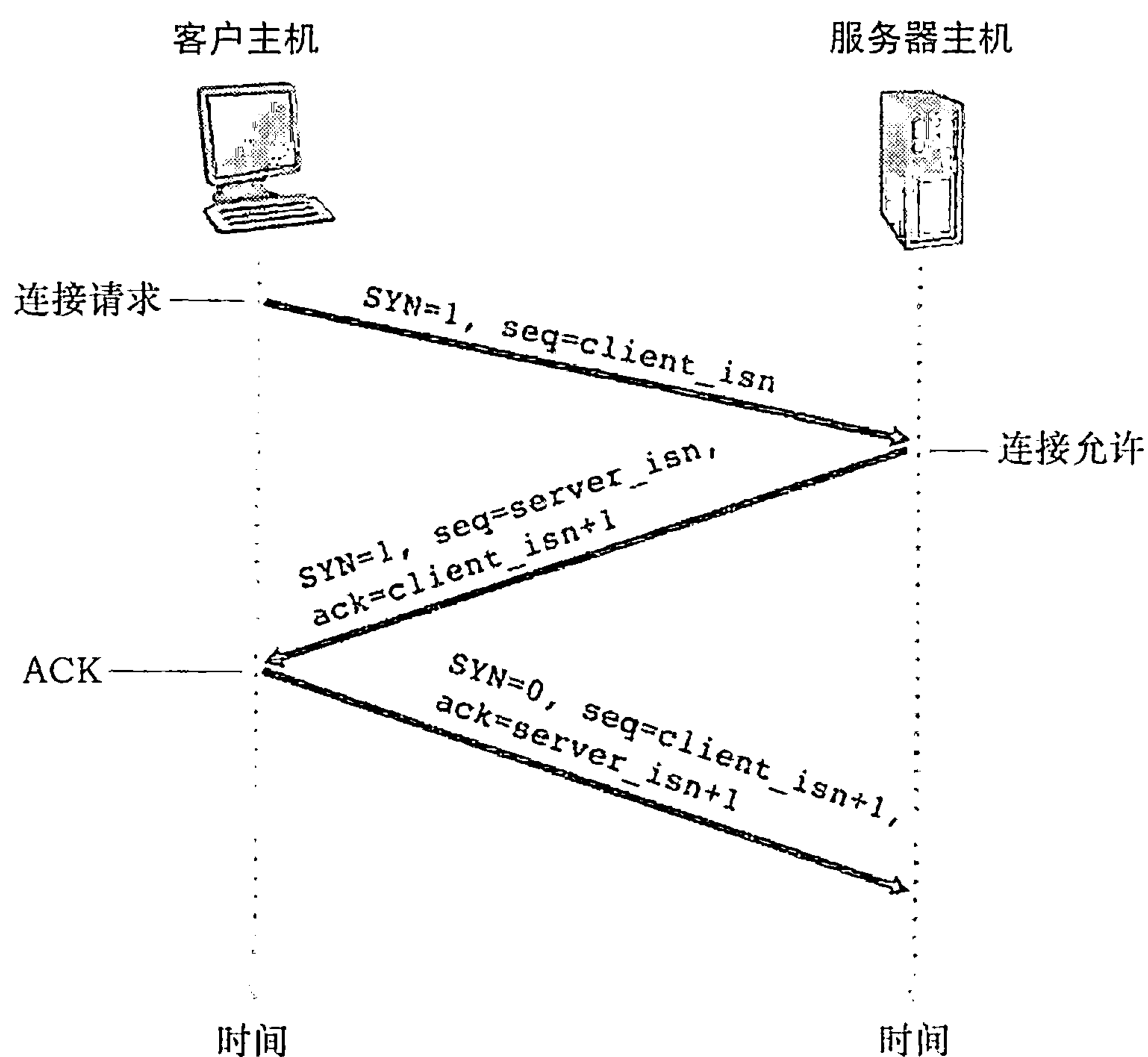


图 3-39 TCP 三次握手：报文段交换

天下没有不散的宴席，对于 TCP 连接也是这样。参与一条 TCP 连接的两个进程中的任何一个都能终止该连接。当连接结束后，主机中的“资源”（即缓存和变量）将被释放。举一个例子，假设某客户打算关闭连接，如图 3-40 所示。客户应用进程发出一个关闭连接命令。这会引引起客户 TCP 向服务器进程发送一个特殊的 TCP 报文段。这个特殊的报文段让其首部中的一个标志位即 FIN 比特（参见图 3-29）被设置为 1。当服务器接收到该报文段后，就向发送方回送一个确认报文段。然后，服务器发送它自己的终止报文段，其 FIN 比特被置为 1。最后，该客户对这个服务器的终止报文段进行确认。此时，在两台主机上用于该连接的所有资源都被释放了。

在一个 TCP 连接的生命周期内，运行在每台主机中的 TCP 协议在各种 TCP 状态（TCP state）之间变迁。图 3-41 说明了客户 TCP 会经历的一系列典型 TCP 状态。客户 TCP 开始时处于 CLOSED（关闭）状态。客户的应用程序发起一个新的 TCP 连接（可通过在第 2 章讲过的 Python 例子中创建一个 Socket 对象来完成）。这引起客户中的 TCP 向服务器中的 TCP 发送一个 SYN 报文段。在发送过 SYN 报文段后，客户 TCP 进入了 SYN_SENT 状态。当客户 TCP 处在 SYN_SENT 状态时，它等待来自服务器 TCP 的对客户所发报文段进行确认且 SYN 比特被置为 1 的一个报文段。收到这样一个报文段之后，客户 TCP 进入 ESTABLISHED（已建立）状态。当处在 ESTABLISHED 状态时，TCP 客户就能发送和接收包含有效载荷数据（即应用层产生的数据）的 TCP 报文段了。

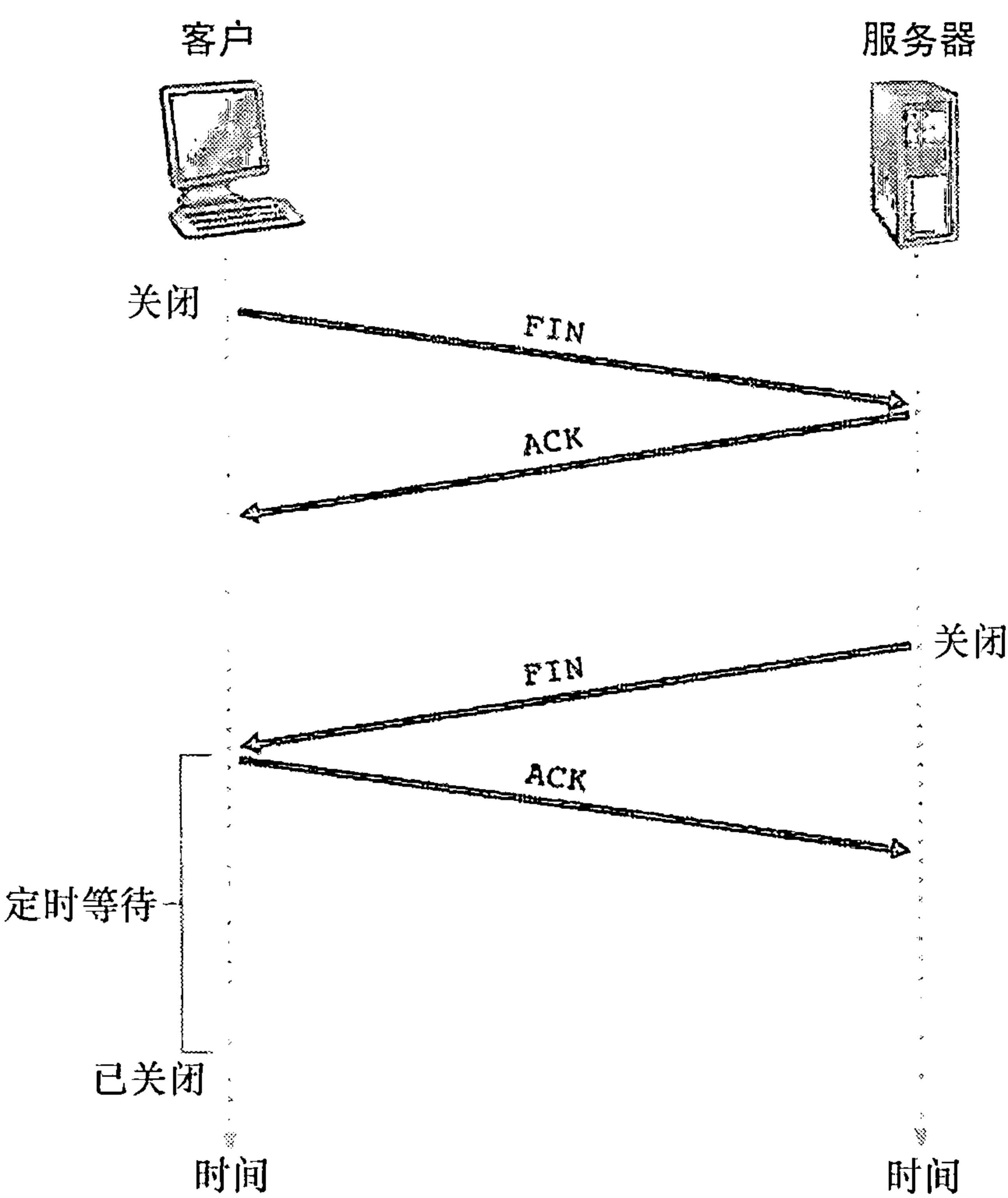


图 3-40 关闭一条 TCP 连接

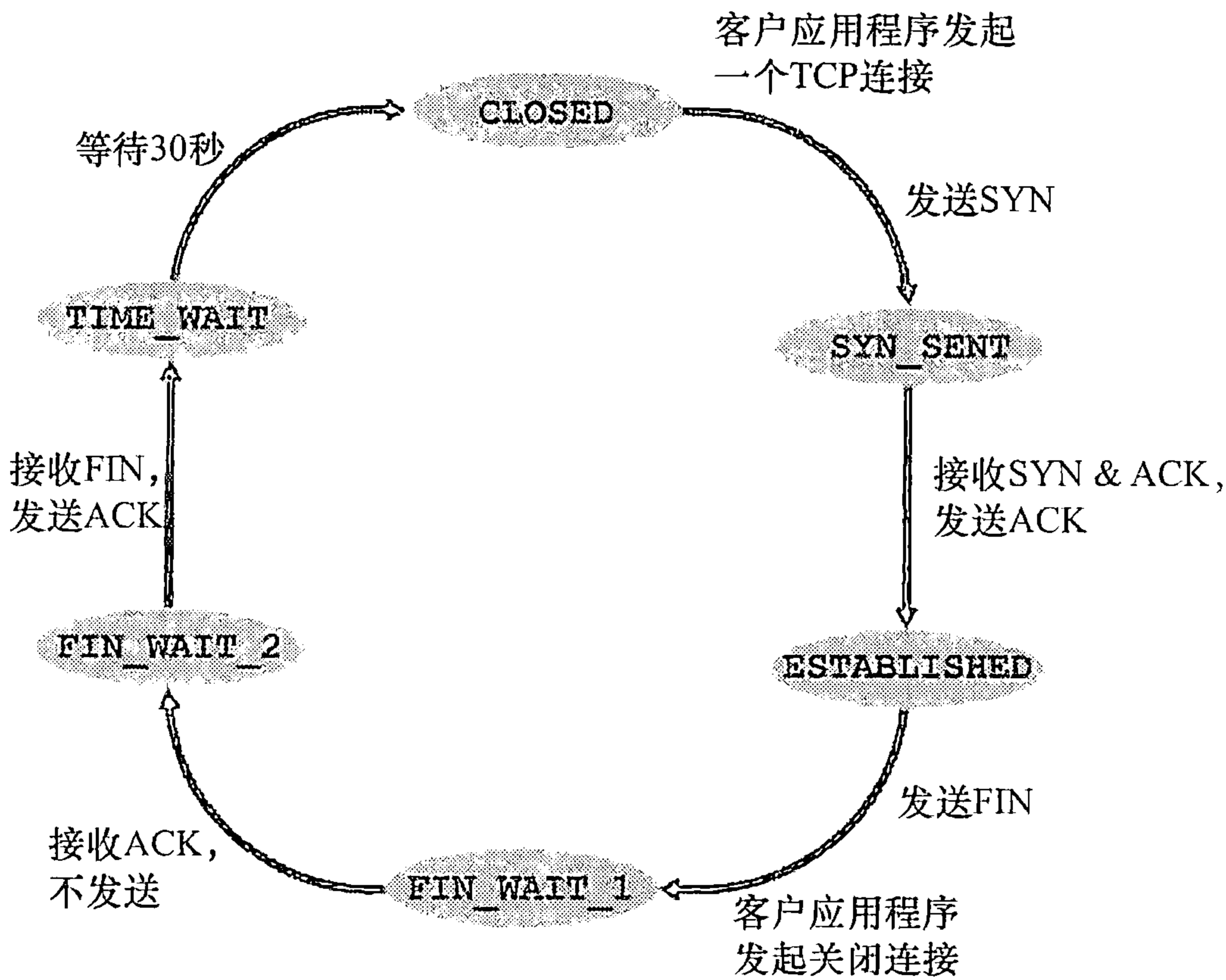


图 3-41 客户 TCP 经历的典型的 TCP 状态序列

假设客户应用程序决定要关闭该连接。（注意到服务器也能选择关闭该连接。）这引起客户 TCP 发送一个带有 FIN 比特被置为 1 的 TCP 报文段，并进入 FIN_WAIT_1 状态。当处在 FIN_WAIT_1 状态时，客户 TCP 等待一个来自服务器的带有确认的 TCP 报文段。当它收到该报文段时，客户 TCP 进入 FIN_WAIT_2 状态。当处在 FIN_WAIT_2 状态时，客户等待来自服务器的 FIN 比特被置为 1 的另一个报文段；当收到该报文段后，客户 TCP 对服务器的报文段进行确认，并进入 TIME_WAIT 状态。假定 ACK 丢失，TIME_WAIT 状态使 TCP 客户重传最后的确认报文。在 TIME_WAIT 状态中所消耗的时间是与具体实现有关的，而典型的值是 30 秒、1 分钟或 2 分钟。经过等待后，连接就正式关闭，客户端所有资源（包括端口号）将被释放。

关注安全性

SYN 洪泛攻击

我们在 TCP 三次握手的讨论中已经看到，服务器为了响应一个收到的 SYN，分配并初始化连接变量和缓存。然后服务器发送一个 SYNACK 进行响应，并等待来自客户的 ACK 报文段。如果某客户不发送 ACK 来完成该三次握手的第三步，最终（通常在一分钟多钟之后）服务器将终止该半开连接并回收资源。

这种 TCP 连接管理协议为经典的 DoS 攻击即 SYN 洪泛攻击（SYN flood attack）提供了环境。在这种攻击中，攻击者发送大量的 TCP SYN 报文段，而不完成第三次握手的步骤。随着这种 SYN 报文段纷至沓来，服务器不断为这些半开连接分配资源（但从未使用），导致服务器的连接资源被消耗殆尽。这种 SYN 洪泛攻击是被记载的众多 DoS 攻击中的第一种 [CERT SYN 1996]。幸运的是，现在有一种有效的防御系统，称为 SYN cookie [RFC 4987]，它们被部署在大多数主流操作系统中。SYN cookie 以下列方式工作：

- 当服务器接收到一个 SYN 报文段时，它并不知道该报文段是来自一个合法的用户，还是一个 SYN 洪泛攻击的一部分。因此服务器不会为该报文段生成一个半开连接。相反，服务器生成一个初始 TCP 序列号，该序列号是 SYN 报文段的源和目的 IP 地址与端口号以及仅有该服务器知道的秘密数的一个复杂函数（散列函数）。这种精心制作的初始序列号被称为“cookie”。服务器则发送具有这种特殊初始序列号的 SYNACK 分组。重要的是，服务器并不记忆该 cookie 或任何对应于 SYN 的其他状态信息。
- 如果客户是合法的，则它将返回一个 ACK 报文段。当服务器收到该 ACK，需要验证该 ACK 是与前面发送的某些 SYN 相对应的。如果服务器没有维护有关 SYN 报文段的记忆，这是怎样完成的呢？正如你可能猜测的那样，它是借助于 cookie 来做到的。前面讲过对于一个合法的 ACK，在确认字段中的值等于在 SYNACK 字段（此时为 cookie 值）中的值加 1（参见图 3-39）。服务器则将使用在 SYNACK 报文段中的源和目的地 IP 地址与端口号（它们与初始的 SYN 中的相同）以及秘密数运行相同的散列函数。如果该函数的结果加 1 与在客户的 SYNACK 中的确认（cookie）值相同的话，服务器认为该 ACK 对应于较早的 SYN 报文段，因此它是合法的。服务器则生成一个具有套接字的全开的连接。

- 在另一方面，如果客户没有返回一个 ACK 报文段，则初始的 SYN 并没有对服务器产生危害，因为服务器没有为它分配任何资源。

图 3-42 图示了服务器端的 TCP 通常要经历的一系列状态，其中假设客户开始连接拆除。这些状态变迁是自解释的。在这两个状态变迁图中，我们只给出了 TCP 连接是如何正常地被建立和拆除的。我们没有描述在某些不正常的情况下（例如当连接的双方同时都要发起或终止一条连接时）发生的事情。如果你对此问题及其他与 TCP 有关的高级问题感兴趣，推荐阅读 Stevens 的内容更全面的书籍 [Stevens 1994]。

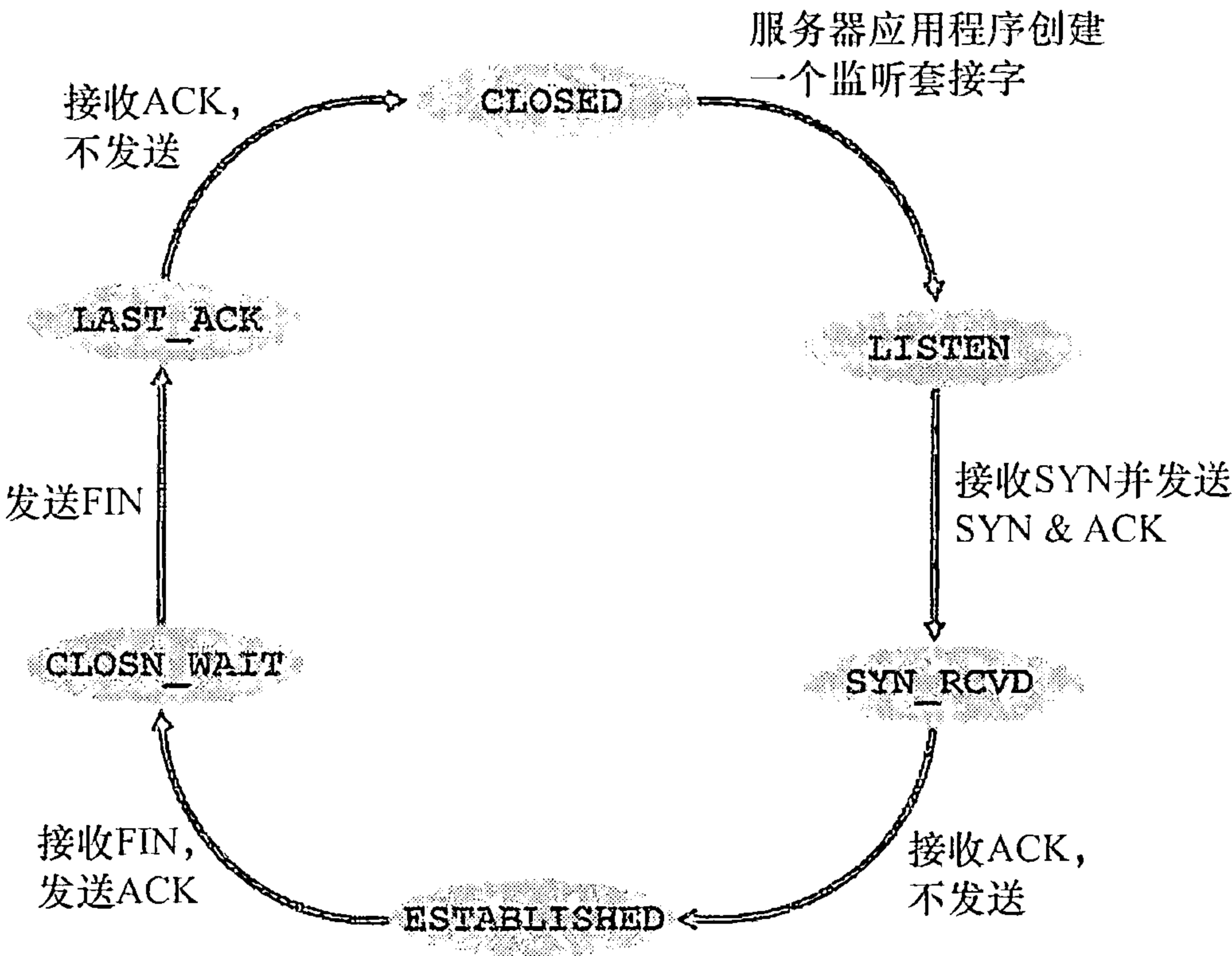


图 3-42 服务器端 TCP 经历的典型的 TCP 状态序列

我们上面的讨论假定了客户和服务端都准备通信，即服务器正在监听客户发送其 SYN 报文段的端口。我们来考虑当一台主机接收到一个 TCP 报文段，其端口号或源 IP 地址与该主机上正在进行的套接字都不匹配的情况。例如，假如一台主机接收了具有目的端口 80 的一个 TCP SYN 分组，但该主机在端口 80 不接受连接（即它不在端口 80 上运行 Web 服务器）。则该主机将向源发送一个特殊重置报文段。该 TCP 报文段将 RST 标志位（参见 3.5.2 节）置为 1。因此，当主机发送一个重置报文段时，它告诉该源“我没有那个报文段的套接字。请不要再发送该报文段了”。当一台主机接收一个 UDP 分组，它的目的端口与正在进行的 UDP 套接字不匹配，该主机发送一个特殊的 ICMP 数据报，这将在第 4 章中讨论。

既然我们已经对 TCP 连接管理有了深入的了解，我们再次回顾 nmap 端口扫描工具，并更为详细地研究它的工作原理。为了探索目标主机上的一个特定的 TCP 端口，如端口 6789，nmap 将对那台主机的目的端口 6789 发送一个特殊的 TCP SYN 报文段。有 3 种可能的输出：

- 源主机从目标主机接收到一个 TCP SYNACK 报文段。因为这意味着在目标主机上一个应用程序使用 TCP 端口 6789 运行，nmap 返回“打开”。

- 源主机从目标主机接收到一个 TCP RST 报文段。这意味着该 SYN 报文段到达了目标主机，但目标主机没有运行一个使用 TCP 端口 6789 的应用程序。但攻击者至少知道发向该主机端口 6789 的报文段没有被源和目标主机之间的任何防火墙所阻挡。（将在第 8 章中讨论防火墙。）
- 源什么也没有收到。这很可能表明该 SYN 报文段被中间的防火墙所阻挡，无法到达目标主机。

nmap 是一个功能强大的工具，该工具不仅能“侦察”打开的 TCP 端口，也能“侦察”打开的 UDP 端口，还能“侦察”防火墙及其配置，甚至能“侦察”应用程序的版本和操作系统。其中的大多数都能通过操作 TCP 连接管理报文段完成 [Skoudis 2006]。读者能够从 <http://www.nmap.org> 下载 nmap。

到此，我们介绍完了 TCP 中的差错控制和流量控制。在 3.7 节中，我们将回到 TCP 并更深入地研究 TCP 拥塞控制问题。然而，在此之前，我们先后退一步，在更广泛环境中讨论拥塞控制问题。

3.6 拥塞控制原理

在前面几节中，我们已经分析了面临分组丢失时用于提供可靠数据传输服务的基本原理及特定的 TCP 机制。我们以前讲过，在实践中，这种丢包一般是当网络变得拥塞时由于路由器缓存溢出引起的。分组重传因此作为网络拥塞的征兆（某个特定的运输层报文段的丢失）来对待，但是却无法处理导致网络拥塞的原因，因为太多的源想以过高的速率发送数据。为了处理网络拥塞原因，需要一些机制以在面临网络拥塞时遏制发送方。

在本节中，我们考虑一般情况下的拥塞控制问题，试图理解为什么网络拥塞是一件坏事情，网络拥塞是如何在上层应用得到的服务性能中明确地显露出来的？如何可用各种方法来避免网络拥塞或对它作出反应？这种对拥塞控制的更一般研究是恰当的，因为就像可靠数据传输一样，它在组网技术中的前 10 个基础性重要问题清单中位居前列。我们通过对异步传递方式（ATM）网络中可用比特率（ABR）服务中的拥塞控制的讨论来总结本节。下面一节包含了 TCP 的拥塞控制算法的详细研究。

3.6.1 拥塞原因与代价

我们通过分析 3 个复杂性越来越高的发生拥塞的情况，开始对拥塞控制的一般性研究。在每种情况下，我们首先将看看出现拥塞的原因以及拥塞的代价（根据资源未被充分利用以及端系统得到的低劣服务性能来评价）。我们暂不关注如何对拥塞作出反应或避免拥塞，而是重点理解一个较为简单的问题，即随着主机增加其发送速率并使网络变得拥塞，这时会发生的情况。

1. 情况 1：两个发送方和一台具有无穷大缓存的路由器

我们先考虑也许是最简单的拥塞情况：两台主机（A 和 B）都有一条连接，且这两条连接共享源与目的地之间的单跳路由，如图 3-43 所示。

我们假设主机 A 中的应用程序以 λ_m 字节/秒的平均速率将数据发送到连接中（例如，通过一个套接字将数据传递给运输层协议）。这些数据是初始数据，这意味着每个数据单元仅向套接字中发送一次。下面的运输层协议是一个简单的协议。数据被封装并发送；不

执行差错恢复（如重传）、流量控制或拥塞控制。忽略由于添加运输层和较低层首部信息产生的额外开销，在第一种情况下，主机 A 向路由器提供流量的速率是 λ_{in} 字节/秒。主机 B 也以同样的方式运行，为了简化问题，我们假设它也是以速率 λ_{in} 字节/秒发送数据。来自主机 A 和主机 B 的分组通过一台路由器，在一段容量为 R 的共享式输出链路上传输。该路由器带有缓存，可用于当分组到达速率超过该输出链路的容量时存储“入分组”。在此第一种情况下，我们将假设路由器有无限大的缓存空间。

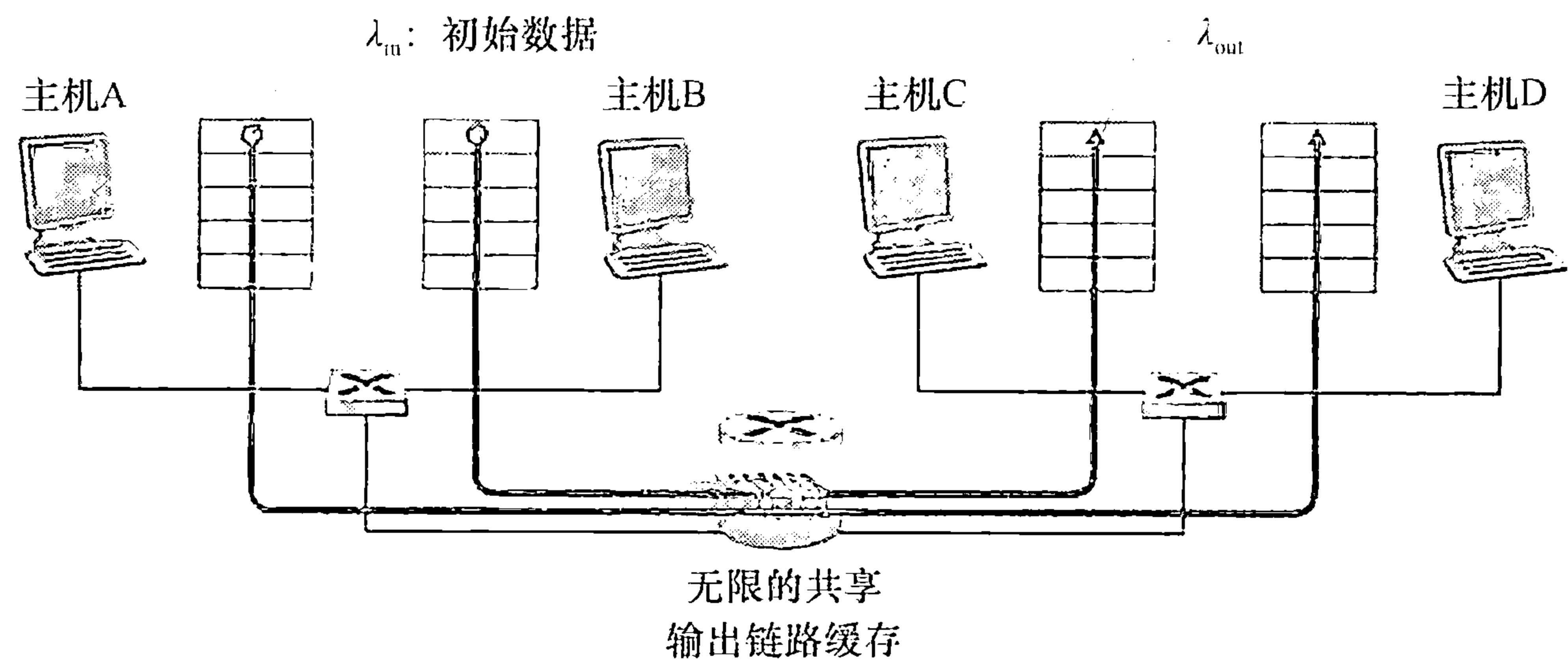


图 3-43 拥塞情况 1：两条连接共享具有无限大缓存的单跳路由

图 3-44 描绘出了第一种情况下主机 A 的连接性能。左边的图形描绘了每连接的吞吐量（per-connection throughput）（接收方每秒接收的字节数）与该连接发送速率之间的函数关系。当发送速率在 $0 \sim R/2$ 之间时，接收方的吞吐量等于发送方的发送速率，即发送方发送的所有数据经有限时延后到达接收方。然而当发送速率超过 $R/2$ 时，它的吞吐量只能达 $R/2$ 。这个吞吐量上限是由两条连接之间共享链路容量造成的。链路完全不能以超过 $R/2$ 的稳定状态速率向接收方交付分组。无论主机 A 和主机 B 将其发送速率设置为多高，它们都不会看到超过 $R/2$ 的吞吐量。

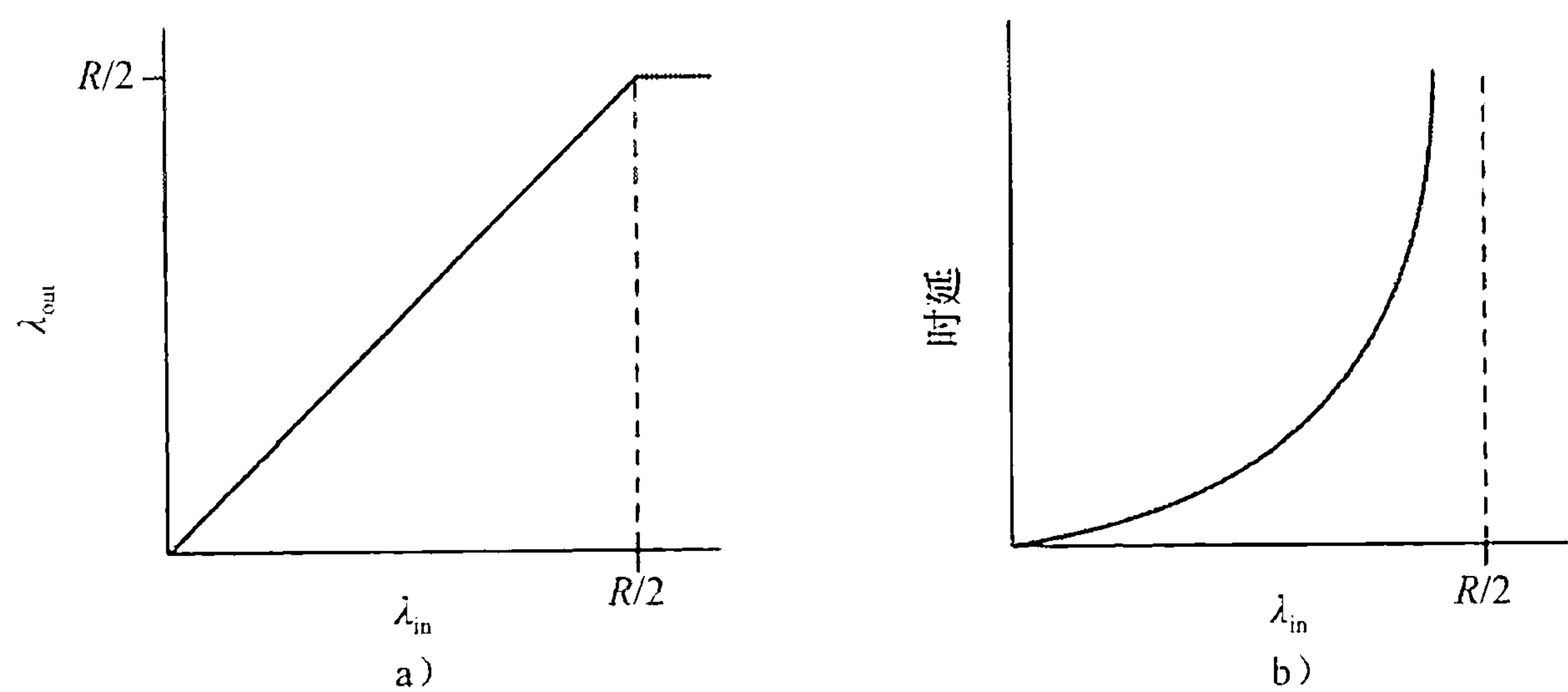


图 3-44 拥塞情况 1：吞吐量、时延与主机发送速率的函数关系

取得每连接 $R/2$ 的吞吐量实际上看起来可能是件好事，因为在将分组交付到目的地的过程中链路被充分利用了。但是，图 3-44b 的图形却显示了以接近链路容量的速率运行时产生的后果。当发送速率接近 $R/2$ 时（从左至右），平均时延就会越来越大。当发送速率超过 $R/2$ 时，路由器中的平均排队分组数就会无限增长，源与目的地之间的平均时延也会变成无穷大（假设这些连接以此发送速率运行无限长时间并且有无限量的缓存可用）。因此，虽然从吞吐量角度看，运行在总吞吐量接近 R 的状态也许是一个理想状态，但从

时延角度看，却远不是一个理想状态。甚至在这种（极端）理想化的情况中，我们已经发现了拥塞网络的一种代价，即当分组的到达速率接近链路容量时，分组经历巨大的排队时延。

2. 情况2：两个发送方和一台具有有限缓存的路由器

现在我们从下列两个方面对情况1稍微做一些修改（参见图3-45）。首先，假定路由器缓存的容量是有限的。这种现实世界的假设的结果是，当分组到达一个已满的缓存时会被丢弃。其次，我们假定每条连接都是可靠的。如果一个包含有运输层报文段的分组在路由器中被丢弃，那么它终将被发送方重传。由于分组可以被重传，所以我们现在必须更小心地使用发送速率这个术语。特别是我们再次以 λ_{in} 字节/秒表示应用程序将初始数据发送到套接字中的速率。运输层向网络中发送报文段（含有初始数据或重传数据）的速率用 λ'_{in} 字节/秒表示。 λ'_{in} 有时被称为网络的供给载荷（offered load）。

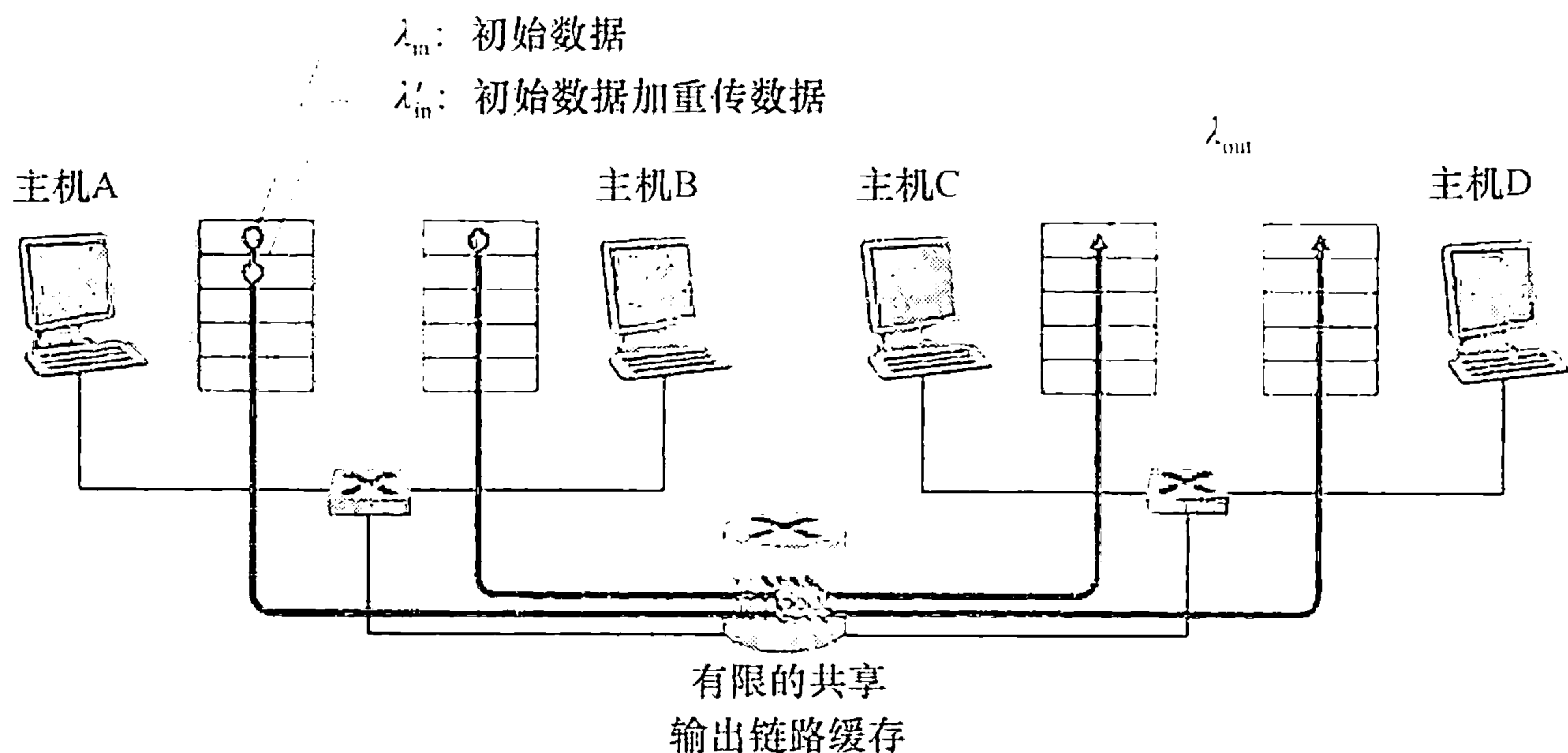


图 3-45 情况 2：（有重传的）两台主机与一台拥有有限缓存的路由器

在情况2下实现的性能将完全取决于重传的方式。首先，考虑一种不真实的情况，即主机A能够以某种方式（不可思议地！）确定路由器中的缓存是否空闲，因而仅当缓存空闲时才发送一个分组。在这种情况下，将不会产生丢包， λ_{in} 与 λ'_{in} 相等，并且连接的吞吐量就等于 λ_{in} 。图3-46a中描述了这种情况。从吞吐量的角度看，性能是理想的，即发送的每个分组都被接收到。注意到在这种情况下，平均主机发送速率不能超过 $R/2$ ，因为假定不会发生分组丢失。

接下来考虑一种更为真实的情况，发送方仅当在确定了一个分组已经丢失时才重传。（同样，所做的假设有一些弹性。然而，发送主机有可能将超时时间设置得足够长，以无形中使其确信一个还没有被确认的分组已经丢失。）在这种情况下，性能就可能与图3-46b所示的情况相似。为了理解这时发生的情况，考虑一下供给载荷 λ'_{in} （初始数据传输加上重传的总速率）等于 $R/2$ 的情况。根据图3-46b，在这一供给载荷值时，数据被交付给接收方应用程序的速率是 $R/3$ 。因此，在所发送的 $0.5R$ 单位数据当中，从平均的角度说， $0.333R$ 字节/秒是初始数据，而 $0.166R$ 字节/秒是重传数据。我们在此看到了另一种网络拥塞的代价，即发送方必须执行重传以补偿因为缓存溢出而丢弃（丢失）的分组。

最后，我们考虑下面一种情况：发送方也许会提前发生超时并重传在队列中已被推迟但还未丢失的分组。在这种情况下，初始数据分组和重传分组都可能到达接收方。当然，

接收方只需要一份这样的分组副本就行了，重传分组将被丢弃。在这种情况下，路由器转发重传的初始分组副本是在做无用功，因为接收方已收到了该分组的初始版本。而路由器本可以利用链路的传输能力去发送另一个分组。这里，我们又看到了网络拥塞的另一种代价，即发送方在遇到大时延时所进行的不必要重传会引起路由器利用其链路带宽来转发不必要的分组副本。图 3-46c 显示了当假定每个分组被路由器转发（平均）两次时，吞吐量与供给载荷的对比情况。由于每个分组被转发两次，当其供给载荷接近 $R/2$ 时，其吞吐量将渐近 $R/4$ 。

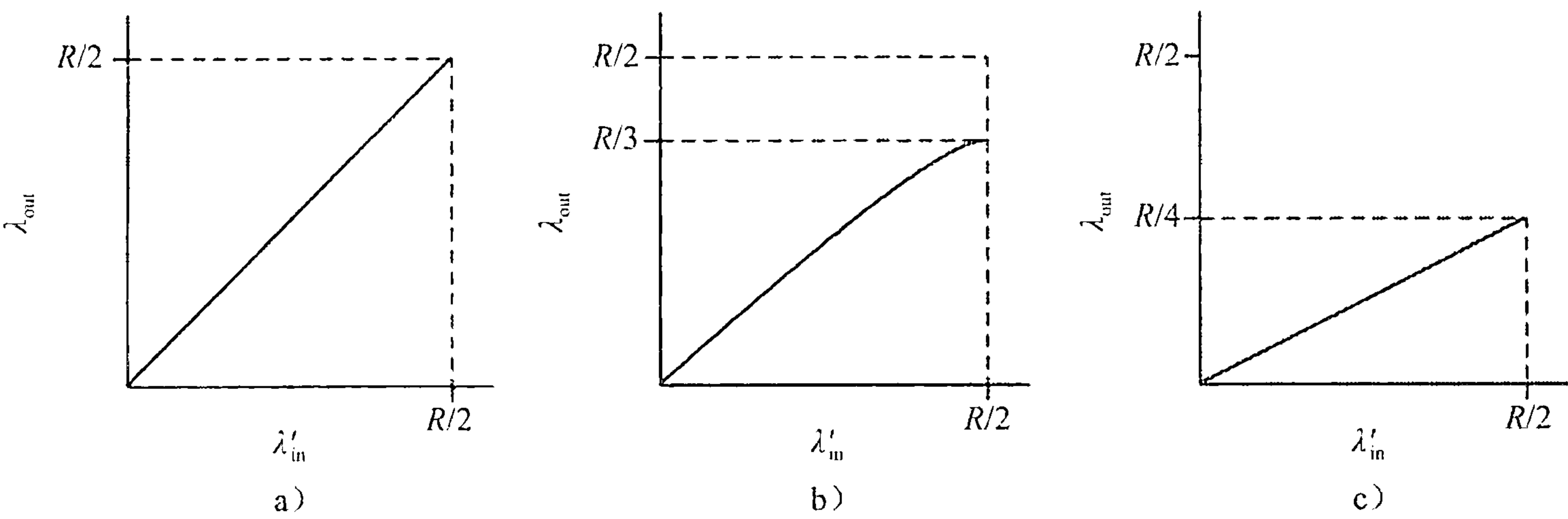


图 3-46 具有有限缓存时情况 2 的性能

3. 情况 3：4 个发送方和具有有限缓存的多台路由器及多跳路径

在最后一种拥塞情况中，有 4 台主机发送分组，每台都通过交叠的两跳路径传输，如图 3-47 所示。我们再次假设每台主机都采用超时/重传机制来实现可靠数据传输服务，所有的主机都有相同的 λ_{in} 值，所有路由器的链路容量都是 R 字节/秒。

λ_{in} ：初始数据
 λ'_{in} ：初始数据加重传数据 λ_{out}

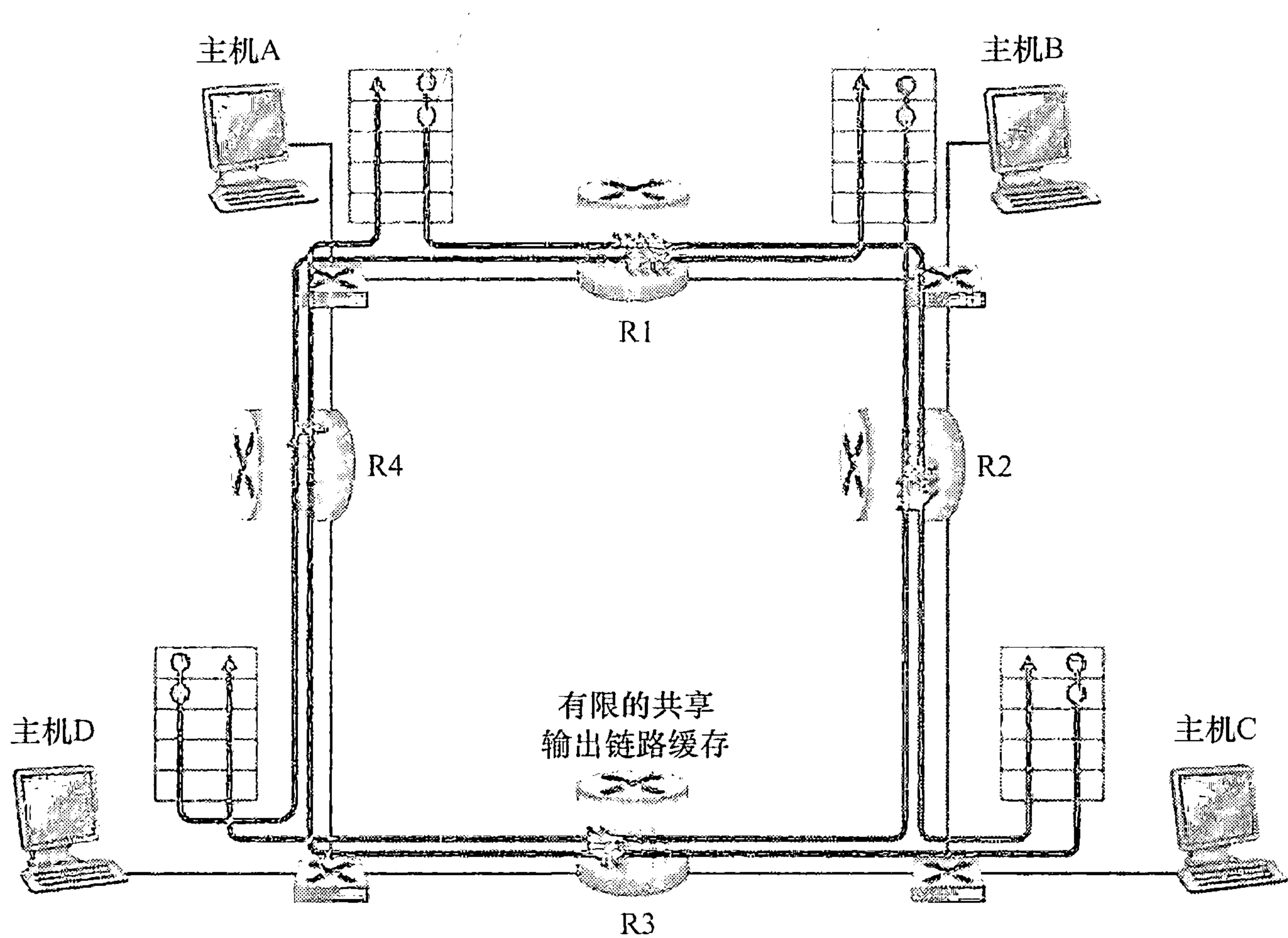


图 3-47 四个发送方和具有有限缓存的多台路由器及多跳路径

我们考虑从主机 A 到主机 C 的连接，该连接经过路由器 R1 和 R2。A - C 连接与 D - B 连接共享路由器 R1，并与 B - D 连接共享路由器 R2。对极小的 λ_{in} 值，路由器缓存的溢出是很少见的（与拥塞情况 1、拥塞情况 2 中的一样），吞吐量大致接近供给载荷。对稍大的 λ_{in} 值，对应的吞吐量也更大，因为有更多的初始数据被发送到网络中并交付到目的地，溢出仍然很少。因此，对于较小的 λ_{in} ， λ_{in} 的增大会导致 λ_{out} 的增大。

在考虑了流量很小的情况后，下面分析当 λ_{in} （因此 λ'_{in} ）很大时的情况。考虑路由器 R2。不管 λ_{in} 的值是多大，到达路由器 R2 的 A - C 流量（在经过路由器 R1 转发后到达路由器 R2）的到达速率至多是 R ，也就是从 R1 到 R2 的链路容量。如果 λ'_{in} 对于所有连接（包括 B - D 连接）来说是极大的值，那么在 R2 上，B - D 流量的到达速率可能会比 A - C 流量的到达速率大得多。因为 A - C 流量与 B - D 流量在路由器 R2 上必须为有限缓存空间而竞争，所以当来自 B - D 连接的供给载荷越来越大时，A - C 连接上成功通过 R2（即由于缓存溢出而未被丢失）的流量会越来越小。在极限情况下，当供给载荷趋近于无穷大时，R2 的空闲缓存会立即被 B - D 连接的分组占满，因而 A - C 连接在 R2 上的吞吐量趋近于 0。这又一次说明在重载的极限情况下，A - C 端到端吞吐量将趋近于 0。这些考虑引发了供给载荷与吞吐量之间的权衡，如图 3-48 所示。

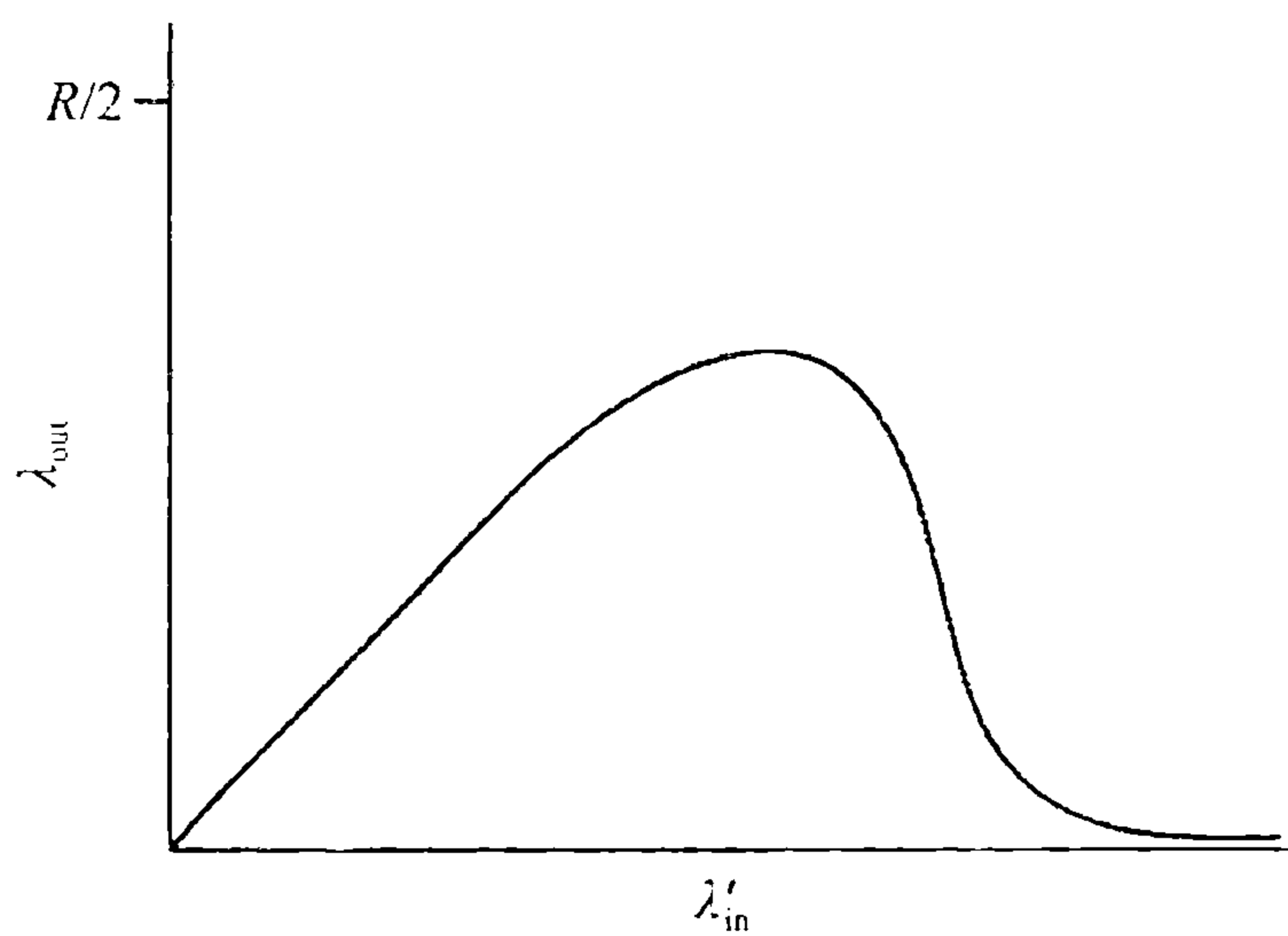


图 3-48 具有有限缓存和多跳路径时的情况 3 性能

当考虑由网络所做的浪费掉的工作量时，随着供给载荷的增加而使吞吐量最终减少的原因是明显的。在上面提到的大流量的情况中，每当有一个分组在第二跳路由器上被丢弃时，第一跳路由器所做的将分组转发到第二跳路由器的工作就是“劳而无功”的。如果第一跳路由器只是丢弃该分组并保持空闲，则网络中的情况是幸运的（更准确地说是糟糕的）。需要指出的是，第一跳路由器所使用的将分组转发到第二跳路由器的传输容量用来传送不同的分组可能更有效益。（例如，当选择一个分组发送时，路由器最好优先考虑那些已经历过一定数量的上游路由器的分组。）所以，我们在此又看到了由于拥塞而丢弃分组的另一种代价，即当一个分组沿一条路径被丢弃时，每个上游路由器用于转发该分组到丢弃该分组而使用的传输容量最终被浪费掉了。

3.6.2 拥塞控制方法

在 3.7 节中，我们将详细研究 TCP 用于拥塞控制的特定方法。这里，我们指出在实践中所采用的两种主要拥塞控制方法，讨论特定的网络体系结构和具体使用这些方法的拥塞控制协议。

在最为宽泛的级别上，我们可根据网络层是否为运输层拥塞控制提供了显式帮助，来区分拥塞控制方法。

- 端到端拥塞控制。在端到端拥塞控制方法中，网络层没有为运输层拥塞控制提供显式支持。即使网络中存在拥塞，端系统也必须通过对网络行为的观察（如分组丢失与时延）来推断之。我们将在 3.7 节中看到，TCP 必须通过端到端的方法解决拥塞控制，因为 IP 层不会向端系统提供有关网络拥塞的反馈信息。TCP 报文

段的丢失（通过超时或 3 次冗余确认而得知）被认为是网络 congestion 的一个迹象，TCP 会相应地减小其窗口长度。我们还将看到关于 TCP congestion 控制的一些最新建议，即使用增加的往返时延值作为网络 congestion 程度增加的指示。

- 网络辅助的 congestion 控制。在网络辅助的 congestion 控制中，网络层构件（即路由器）向发送方提供关于网络中 congestion 状态的显式反馈信息。这种反馈可以简单地用一个比特来指示链路中的 congestion 情况。该方法在早期的 IBM SNA [Schwartz 1982] 和 DEC DECnet [Jain 1989; Ramakrishnan 1990] 等体系结构中被采用，近来被建议用于 TCP/IP 网络 [Floyd TCP 1994; RFC 3168]，而且还用在我们下面要讨论的 ATM 可用比特率（ABR）congestion 控制中。更复杂的网络反馈也是可能的。例如，我们很快将学习的一种 ATM ABR congestion 控制形式，它允许路由器显式地通知发送方，告知它（路由器）能在输出链路上支持的传输速率。关于源端是增加还是降低其传输速率，XCP 协议 [Katabi 2002] 对每个源提供了路由器计算的反馈，该反馈携带在分组首部中。

对于网络辅助的 congestion 控制，congestion 信息从网络反馈到发送方通常有两种方式，如图 3-49 所示。直接反馈信息可以由网络路由器发给发送方。这种方式的通知通常采用了一种阻塞分组（choke packet）的形式（主要是说：“我 congestion 了！”）。第二种形式的通知是，路由器标记或更新从发送方流向接收方的分组中的某个字段来指示 congestion 的产生。一旦收到一个标记的分组后，接收方就会向发送方通知该网络 congestion 指示。注意到后一种形式的通知至少要经过一个完整的往返时间。

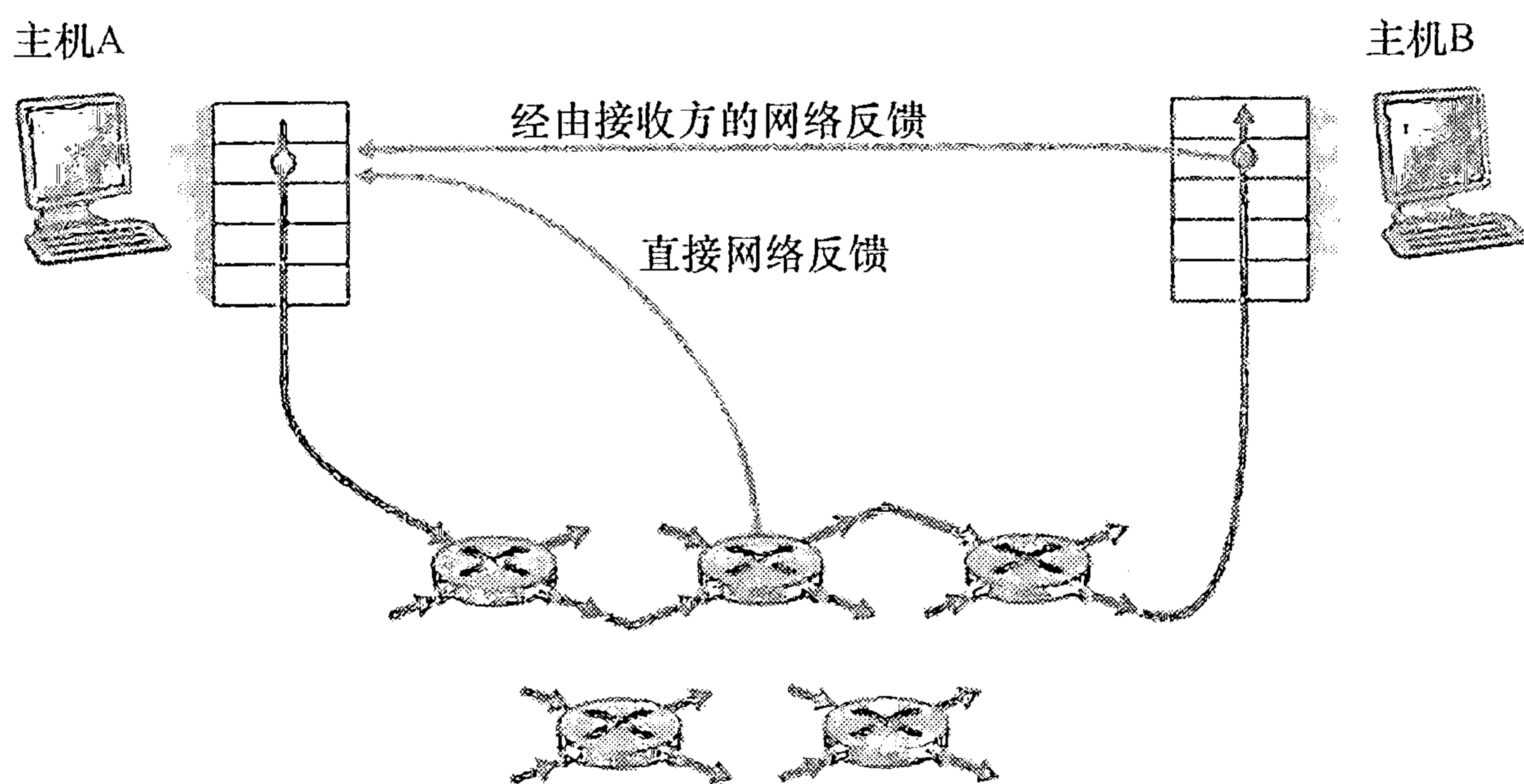


图 3-49 网络指示 congestion 信息的两种反馈路径

3.6.3 网络辅助的 congestion 控制例子：ATM ABR congestion 控制

我们现在通过一个简要的学习案例来结束本节，该案例是 ATM ABR 中的 congestion 控制算法，即一种采用网络辅助方法解决 congestion 控制的协议。我们强调此时的目的不是详细地描述 ATM 体系结构的方方面面，而只是为了说明该协议为 congestion 控制所采用的方法明显不同于因特网 TCP 协议的方法。事实上，我们下面仅给出为了理解 ABR congestion 控制所需要的 ATM 体系结构的几个方面。

ATM 基本上采用一种面向虚电路（VC）的方法来处理分组交换。回想我们在第 1 章中的讨论，这意味着从源到目的地路径上的每台交换机将维护有关源到目的地 VC 的状态。这

种逐个 VC 的状态允许交换机跟踪各个发送方的行为（例如，跟踪它们的平均传输速率），并采取特定源的拥塞控制动作（例如，当交换机变得拥塞时，向发送方发显式信令以减少它的速率）。网络交换机上的这种逐 VC 状态使 ATM 非常适合执行网络辅助拥塞控制。

ABR 已被设计成一种弹性数据传输服务，该服务方式使人联想起 TCP。当网络轻载时，ABR 服务会充分利用空闲的可用带宽；当网络拥塞时，ABR 服务会将其传输速率抑制为某些预先确定的最小传输速率。[Jain 1996] 提供了一个关于 ATM ABR 拥塞控制与流量管理的详细学习指南。

图 3-50 显示了 ATM ABR 拥塞控制框架。在下面的讨论中，我们将采用 ATM 的术语（如使用术语交换机而不使用路由器；使用术语信元（cell）而不使用分组）。对于 ATM ABR 服务，数据信元从源经过一系列中间交换机传输到目的地。在数据信元中夹杂着所谓的资源管理信元（Resource-Management cell, RM 信元）；这些 RM 信元可被用来在主机和交换机之间传递与拥塞相关的信息。当一个 RM 信元到达目的地时，它将被掉转方向并向发送方发送（很可能是在目的地修改了该 RM 信元的内容之后）。交换机也有可能自己产生一个 RM 信元，并将该 RM 信元直接发送给源。因此，RM 信元可用来提供直接网络反馈和经由接收方的网络反馈，如图 3-50 所示。

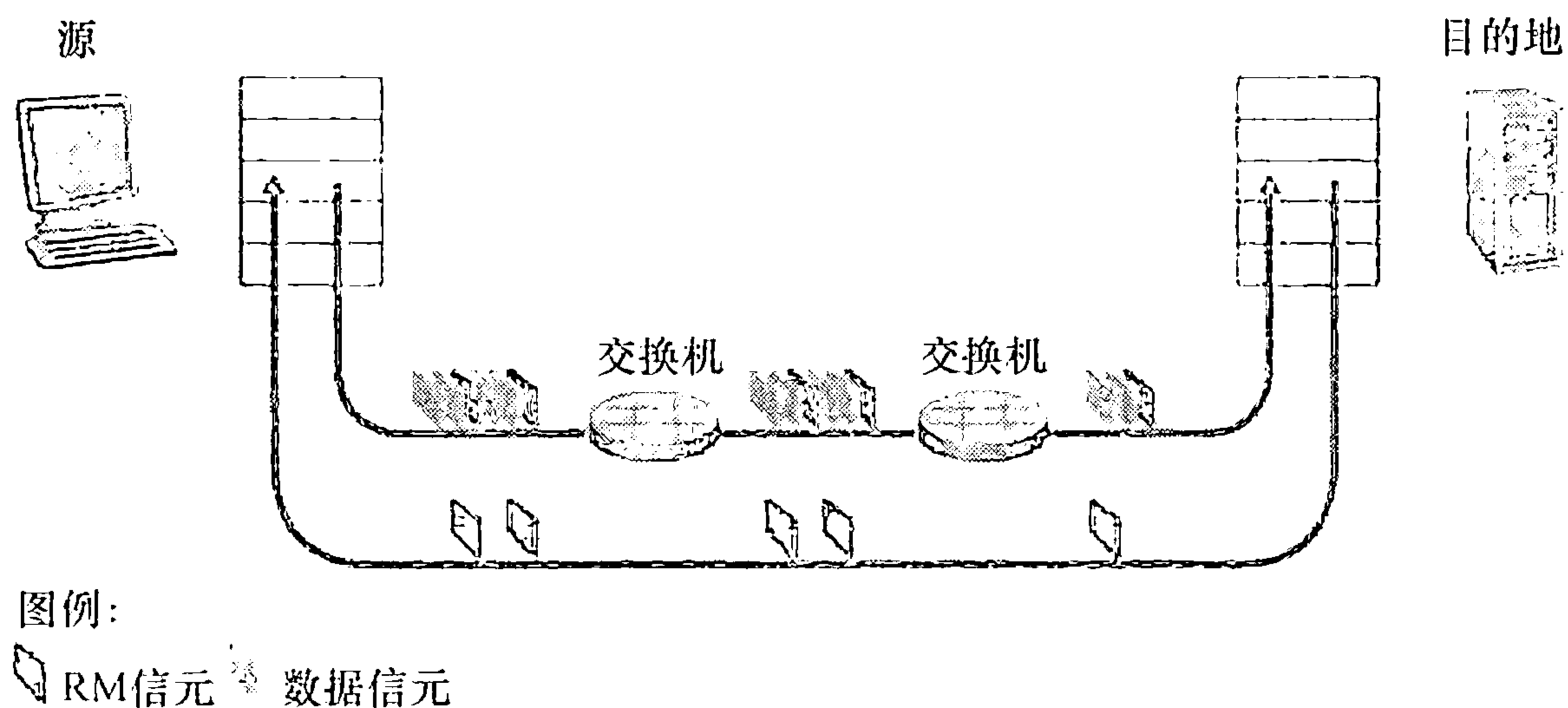


图 3-50 用于 ATM ABR 服务的拥塞控制框架

ATM ABR 拥塞控制是一种基于速率的方法。即发送方明确地计算出它所能发送的最大速率，并据此对自己进行相应的调整。ABR 提供三种机制用于从交换机向接收方发送与拥塞相关的信令信息：

- EFCI 比特。每个数据信元都包含 1 比特的显式转发拥塞指示（Explicit Forward Congestion Indication, EFCI）比特。某拥塞的网络交换机可把一个数据信元中的 EFCI 比特设置为 1 来向目的主机发送网络已经拥塞的信令。其目的地必须检查所有收到的数据信元中的 EFCI 比特。当一个 RM 信元到达目的地时，如果多数近来收到的数据信元的 EFCI 比特都被置为 1，则目的地就会将 RM 信元的拥塞指示比特（CI 比特）置为 1，并将该 RM 信元发送回发送方。使用数据信元中的 EFCI 比特和 RM 信元中的 CI 比特，发送方因而能在网络交换机拥塞时得到通知。
- CI 和 NI 比特。如上所述，发送方到接收方的 RM 信元是夹杂在数据单元当中的。RM 信元的夹杂比率是一个可调参数，默认值是每 32 个数据信元中有一个 RM 信元。这些 RM 信元中有一个拥塞指示（Congestion Indication, CI）比特和无增长（No Increase, NI）比特，这两个比特可被一台拥塞的交换机设置。特别是，交换